

唐诗意象智能分析系统

— 源代码正文 —

(共计 31 个源文件, 6225 行)

文件: config.py (112 行, 5.4 KB)

```
1  #-*- coding: utf-8 -*-
2  """
3  唐诗意象智能分析系统 — 全局配置
4  """
5  import os
6  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
7  # -----
8  # API 配置
9  # -----
10 LLM_API_BASE = "http://127.0.0.1:44787"
11 EMBED_API_URL = f"{LLM_API_BASE}/v1/embeddings"
12 CHAT_API_URL = f"{LLM_API_BASE}/v1/chat/completions"
13 EMBED_MODEL = "text-embedding-qwen3-embedding-4b"
14 CHAT_MODEL = "qwen3.5-35b-a3b"
15 # -----
16 # 路径配置
17 # -----
18 POEMS_JSON_DIR = os.path.join(BASE_DIR, "json_dataa")
19 POEM_JSON_DIR = os.path.join(BASE_DIR, "poem_json")
20 RAG_DB_DIR = os.path.join(BASE_DIR, "rag_db")
21 TEMPLATES_DIR = os.path.join(BASE_DIR, "templates")
22 EXPORT_DIR = os.path.join(BASE_DIR, "exports")
23 LOG_DIR = os.path.join(BASE_DIR, "logs")
24 CACHE_DIR = os.path.join(BASE_DIR, "cache")
25 TESTS_DIR = os.path.join(BASE_DIR, "tests")
26 # -----
27 # RAG 检索配置
28 # -----
29 TOP_K = 5
30 RAG_COLLECTION_NAME = "poems"
31 RAG_SPACE_FUNC = "cosine"
32 # -----
33 # Flask 运行配置
34 # -----
35 FLASK_HOST = "0.0.0.0"
36 FLASK_PORT = 5000
37 FLASK_DEBUG = True
38 FLASK_THREADED = True
39 SECRET_KEY = os.environ.get("FLASK_SECRET_KEY", "tang-poetry-secret-key-change-in-production")
40 # -----
41 # 数据分页
42 # -----
43 PAGE_SIZE = 25
44 RENDER_LIMIT = 5000
45 # -----
46 # 文件处理
47 # -----
48 MAX_BYTES_PER_CHUNK = 1024
49 EMBEDDING_RETRIES = 3
50 EMBEDDING_TIMEOUT = 30
51 CHAT_TIMEOUT = 300
52 # -----
53 # 意象分类映射
54 # -----
55 CATEGORY_NAME_MAP = {
56     "1-1": "自然意象-天文意象",
57     "1-2": "自然意象-地理意象",
58     "1-3": "自然意象-植物意象",
```

```

59     "1-4": "自然意象-动物意象",
60     "2-1": "社会意象-生产生活意象",
61     "2-2": "社会意象-军事战争意象",
62     "2-3": "社会意象-制度观念意象",
63     "3-1": "人文意象-人造器物意象",
64     "3-2": "人文意象-人类自身意象",
65     "3-3": "人文意象-人物角色意象",
66     "3-4": "人文意象-文化意象",
67 }
68 CATEGORY_MAJOR_MAP = {
69     "1": "自然意象",
70     "2": "社会意象",
71     "3": "人文意象",
72 }
73 # -----
74 # 已知诗列表
75 # -----
76 KNOWN_AUTHORS = [
77     "杜甫", "李白", "王维", "白居易", "苏轼", "李商隐", "杜牧",
78     "王昌龄", "孟浩然", "柳宗元", "韩愈", "欧阳修", "辛弃疾",
79     "李清照", "陶渊明", "张九龄", "司空曙", "刘长卿", "韦应物",
80     "岑参", "高适", "王之涣", "元稹", "温庭筠", "晏殊",
81 ]
82 # -----
83 # 日志
84 # -----
85 LOG_LEVEL = "INFO"
86 LOG_MAX_BYTES = 10 * 1024 * 1024 # 10MB
87 LOG_BACKUP_COUNT = 5
88 LOG_FORMAT = "%(asctime)s [%(levelname)s] %(name)s: %(message)s"
89 # -----
90 # 缓存
91 # -----
92 CACHE_DEFAULT_TTL = 300 # 5 分钟
93 CACHE_ENABLED = True
94 # -----
95 # 导出
96 # -----
97 EXPORT_CSV_ENCODING = "utf-8-sig"
-- 文件结束: config.py --
文件: logger.py (93 行, 2.3 KB)
1  #-*- coding: utf-8 -*-
2  """
3  日志模块 — 分级日志、文件滚动、上下文跟踪
4  """
5  import logging
6  import os
7  import sys
8  import traceback
9  from logging.handlers import RotatingFileHandler
10 from config import LOG_DIR, LOG_LEVEL, LOG_MAX_BYTES, LOG_BACKUP_COUNT, LOG_FORMAT
11 def ensure_log_dir():
12     if not os.path.exists(LOG_DIR):
13         os.makedirs(LOG_DIR, exist_ok=True)
14 def _create_console_handler(formatter):
15     handler = logging.StreamHandler(sys.stdout)
16     handler.setFormatter(formatter)
17     handler.setLevel(logging.DEBUG)
18     return handler
19 def _create_file_handler(log_path, formatter):
20     handler = RotatingFileHandler(
21         log_path,
22         maxBytes=LOG_MAX_BYTES,
23         backupCount=LOG_BACKUP_COUNT,

```

```

24         encoding="utf-8",
25     )
26     handler.setFormatter(formatter)
27     handler.setLevel(logging.DEBUG)
28     return handler
29 _loggers = {}
30 def get_logger(name=None):
31     if name is None:
32         name = __name__
33     if name in _loggers:
34         return _loggers[name]
35     ensure_log_dir()
36     level = getattr(logging, LOG_LEVEL.upper(), logging.INFO)
37     formatter = logging.Formatter(LOG_FORMAT)
38     logger = logging.getLogger(name)
39     logger.setLevel(level)
40     if not logger.handlers:
41         logger.addHandler(_create_console_handler(formatter))
42         logger.addHandler(
43             _create_file_handler(os.path.join(LOG_DIR, "app.log"), formatter)
44         )
45     logger.propagate = False
46     _loggers[name] = logger
47     return logger
48 class LoggerMixin:
49     """混入类，为对象提供便捷日志方法"""
50     @property
51     def logger(self):
52         if not hasattr(self, "_logger"):
53             cls_name = type(self).__name__
54             self._logger = get_logger(cls_name)
55         return self._logger
56     def log_debug(self, msg, *args, **kwargs):
57         self.logger.debug(msg, *args, **kwargs)
58     def log_info(self, msg, *args, **kwargs):
59         self.logger.info(msg, *args, **kwargs)
60     def log_warning(self, msg, *args, **kwargs):
61         self.logger.warning(msg, *args, **kwargs)
62     def log_error(self, msg, *args, **kwargs):
63         self.logger.error(msg, *args, **kwargs)
64     def log_exception(self, msg, *args, **kwargs):
65         self.logger.exception(msg, *args, **kwargs)
66     def format_exception(e: Exception) -> str:
67         return "".join(traceback.format_exception(type(e), e, e.__traceback__))
-- 文件结束: logger.py --
文件: errors.py (112 行, 3.0 KB)
1  # -*- coding: utf-8 -*-
2  """
3  统一异常处理模块
4  """
5  import json
6  from flask import Response, jsonify
7  class AppError(Exception):
8      """应用基础异常"""
9      def __init__(self, message: str, status_code: int = 400, details: dict = None):
10         super().__init__(message)
11         self.message = message
12         self.status_code = status_code
13         self.details = details or {}
14 class DataNotFoundError(AppError):
15     """数据未找到"""
16     def __init__(self, message="未找到相关数据", details=None):
17         super().__init__(message, status_code=404, details=details)
18 class EmbeddingError(AppError):

```

```

19     """向量化服务异常"""
20     def __init__(self, message="向量化服务请求失败", details=None):
21         super().__init__(message, status_code=502, details=details)
22 class LLMError(AppError):
23     """大模型服务异常"""
24     def __init__(self, message="大模型服务请求失败", details=None):
25         super().__init__(message, status_code=502, details=details)
26 class ValidationError(AppError):
27     """输入校验异常"""
28     def __init__(self, message="输入参数校验失败", field=None, details=None):
29         d = details or {}
30         if field:
31             d["field"] = field
32         super().__init__(message, status_code=422, details=d)
33 class CacheError(AppError):
34     """缓存异常"""
35     def __init__(self, message="缓存操作失败", details=None):
36         super().__init__(message, status_code=500, details=details)
37 class ExportError(AppError):
38     """导出异常"""
39     def __init__(self, message="数据导出失败", details=None):
40         super().__init__(message, status_code=500, details=details)
41 def sse_error(message: str) -> str:
42     """生成 SSE 格式错误消息"""
43     return f"data: {json.dumps({'type': 'error', 'message': message}, ensure_ascii=False)}\n\n"
44 def json_error_response(error: AppError) -> Response:
45     """将 AppError 转为 Flask JSON 响应"""
46     return jsonify({
47         "error": {
48             "message": error.message,
49             "type": type(error).__name__,
50             "details": error.details,
51         }
52     }, error.status_code)
53 def register_error_handlers(app):
54     """在 Flask app 上注册统一异常处理器"""
55     @app.errorhandler(AppError)
56     def handle_app_error(error):
57         return json_error_response(error)
58     @app.errorhandler(400)
59     def handle_bad_request(e):
60         return jsonify({
61             "error": {"message": "请求格式不正确", "type": "BadRequest"}
62         }), 400
63     @app.errorhandler(404)
64     def handle_not_found(e):
65         return jsonify({
66             "error": {"message": "请求的资源不存在", "type": "NotFound"}
67         }), 404
68     @app.errorhandler(405)
69     def handle_method_not_allowed(e):
70         return jsonify({
71             "error": {"message": "请求方法不允许", "type": "MethodNotAllowed"}
72         }), 405
73     @app.errorhandler(500)
74     def handle_internal_error(e):
75         return jsonify({
76             "error": {"message": "服务器内部错误", "type": "InternalServerError"}
77         }), 500
78     return app

```

-- 文件结束: errors.py --

文件: validators.py (84 行, 2.6 KB)

```

1 # -*- coding: utf-8 -*-
2 """

```

```

3  请求校验模块
4  """
5  import re
6  from typing import Optional
7  from errors import ValidationError
8  def validate_question(question: Optional[str]) -> str:
9      """校验用户提问"""
10     if not question:
11         raise ValidationError("问题不能为空", field="question")
12     q = question.strip()
13     if len(q) > 2000:
14         raise ValidationError("问题长度不能超过 2000 字符", field="question")
15     if len(q) < 1:
16         raise ValidationError("问题不能为空", field="question")
17     return q
18 def validate_item(item: Optional[dict]) -> dict:
19     """校验意象解析请求中的 item"""
20     if not item or not isinstance(item, dict):
21         raise ValidationError("缺少诗歌意象数据", field="item")
22     i = item.get("文本") or item.get("txt") or ""
23     line = item.get("line") or ""
24     if not i and not line:
25         raise ValidationError("缺少诗句或意象上下文", field="item")
26     return item
27 def validate_history(history: Optional[list]) -> list:
28     """校验对话历史"""
29     if not history or not isinstance(history, list):
30         return []
31     valid = []
32     for h in history:
33         if isinstance(h, dict) and h.get("role") in ("user", "assistant") and h.get("content"):
34             valid.append({
35                 "role": h["role"],
36                 "content": str(h["content"][:5000])
37             })
38     return valid
39 def validate_poem_id(poem_id: Optional[str]) -> str:
40     """校验诗歌编号"""
41     if not poem_id or not poem_id.strip():
42         raise ValidationError("诗歌编号不能为空", field="poem_id")
43     pid = poem_id.strip()
44     if not re.match(r"^[P]?[d]{2,6}$", pid):
45         raise ValidationError(f"诗歌编号格式不正确: {pid}", field="poem_id")
46     return pid
47 def validate_filename(filename: Optional[str], allowed_exts=None) -> str:
48     """校验导出文件名"""
49     if not filename or not filename.strip():
50         return "export"
51     allowed = allowed_exts or [".csv", ".json", ".xlsx"]
52     name = filename.strip()
53     has_ext = any(name.lower().endswith(ext) for ext in allowed)
54     if not has_ext:
55         raise ValidationError(
56             f"文件名必须包含合法后缀 ({', '.join(allowed)})",
57             field="filename"
58         )
59     if re.search(r"[\|\\:.*?\"<>|]", name):
60         raise ValidationError("文件名包含非法字符", field="filename")
61     return name
62 def sanitize_html(text: str) -> str:
63     """简单的 HTML 转义"""
64     if not text:
65         return ""
66     return (text.replace("&", "&"))

```

```

67         .replace("<", "&lt;")
68         .replace(">", "&gt;")
69         .replace("'", "&quot;")
70         .replace('"', "&#39;"))

```

-- 文件结束: validators.py --

文件: utils.py (146 行, 3.8 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  通用工具函数模块 — 文本处理、格式化、系统辅助
4  """
5  import json
6  import re
7  import math
8  import hashlib
9  from typing import List, Optional, Any
10 from datetime import datetime
11 def truncate(text: str, max_len: int = 100, suffix: str = "...") -> str:
12     """截断字符串到指定长度"""
13     if not text:
14         return ""
15     if len(text) <= max_len:
16         return text
17     return text[:max_len].rstrip() + suffix
18 def extract_numbers(text: str) -> List[int]:
19     """从文本中提取所有整数"""
20     return [int(x) for x in re.findall(r"\d+", text)]
21 def is_chinese_char(ch: str) -> bool:
22     """判断是否为中文字符"""
23     if len(ch) != 1:
24         return False
25     cp = ord(ch)
26     return (
27         0x4E00 <= cp <= 0x9FFF
28         or 0x3400 <= cp <= 0x4DBF
29         or 0x20000 <= cp <= 0x2A6DF
30         or 0x2A700 <= cp <= 0x2B73F
31     )
32 def chinese_ratio(text: str) -> float:
33     """计算字符串中中文字符占比"""
34     if not text:
35         return 0.0
36     chinese_chars = sum(1 for ch in text if is_chinese_char(ch))
37     return round(chinese_chars / len(text), 4)
38 def split_sentences(text: str) -> List[str]:
39     """按中文句号、问号、感叹号、分句"""
40     if not text:
41         return []
42     parts = re.split(r"([。！？\n])", text)
43     sentences = []
44     buf = ""
45     for part in parts:
46         buf += part
47         if part in ("。", "!", "?", "\n"):
48             buf = buf.strip()
49             if buf:
50                 sentences.append(buf)
51             buf = ""
52     buf = buf.strip()
53     if buf:
54         sentences.append(buf)
55     return sentences
56 def format_file_size(size_bytes: int) -> str:
57     """格式化文件大小 (B/KB/MB/GB)"""
58     if size_bytes <= 0:

```

```

59         return "0 B"
60     units = ["B", "KB", "MB", "GB"]
61     i = int(math.floor(math.log(size_bytes, 1024))) if size_bytes > 0 else 0
62     i = min(i, len(units) - 1)
63     size = size_bytes / (1024 ** i)
64     return f"{size:.1f} {units[i]}"
65 def format_timestamp(ts: float, fmt: str = "%Y-%m-%d %H:%M:%S") -> str:
66     """格式化时间戳"""
67     return datetime.fromtimestamp(ts).strftime(fmt)
68 def safe_get(data: dict, key: str, default: Any = "") -> Any:
69     """安全获取字典值"""
70     if not isinstance(data, dict):
71         return default
72     value = data.get(key)
73     if value is None:
74         return default
75     return value
76 def dict_pick(data: dict, keys: List[str]) -> dict:
77     """从字典中提取指定键的子集"""
78     return {k: data[k] for k in keys if k in data}
79 def dict_omit(data: dict, keys: List[str]) -> dict:
80     """从字典中排除指定键"""
81     return {k: v for k, v in data.items() if k not in keys}
82 def md5_hash(text: str) -> str:
83     """计算 MD5 哈希"""
84     return hashlib.md5(text.encode("utf-8")).hexdigest()
85 def safe_json_loads(text: str, default: Any = None) -> Any:
86     """安全解析 JSON，失败返回默认值"""
87     try:
88         return json.loads(text)
89     except (json.JSONDecodeError, TypeError):
90         return default
91 def list_chunk(items: list, size: int) -> List[list]:
92     """将列表按指定大小分块"""
93     return [items[i : i + size] for i in range(0, len(items), size)]
94 def deduplicate_by_key(items: List[dict], key: str) -> List[dict]:
95     """按字典的某个键去重（保留第一个）"""
96     seen = set()
97     result = []
98     for item in items:
99         val = item.get(key)
100         if val not in seen:
101             seen.add(val)
102             result.append(item)
103     return result
104 def merge_dicts(base: dict, override: dict) -> dict:
105     """合并字典，override 覆盖 base"""
106     result = base.copy()
107     result.update(override)
108     return result
109 def normalize_whitespace(text: str) -> str:
110     """规范化空白字符"""
111     if not text:
112         return ""
113     return re.sub(r"\s+", " ", text).strip()

```

— 文件结束: utils.py —

文件: middleware.py (188 行, 5.6 KB)

```

1  #-*- coding: utf-8 -*-
2  """
3  中间件模块 — 请求标识、计时、CORS、日志追踪
4  """
5  import time
6  import uuid
7  import threading

```

```

8 from functools import wraps
9 from flask import request, g, jsonify
10 from config import BASE_DIR
11 from logger import get_logger
12 logger = get_logger("middleware")
13 # —— 请求标识 ——
14 class RequestContext:
15     """线程级请求上下文"""
16     def __init__(self):
17         self.request_id = None
18         self.start_time = None
19         self.client_ip = None
20         self.endpoint = None
21         self.method = None
22     def init_from_flask(self):
23         self.request_id = request.headers.get("X-Request-ID") or \
24             request.args.get("_rid") or \
25             uuid.uuid4().hex[:12]
26         self.start_time = time.time()
27         self.client_ip = request.remote_addr or "unknown"
28         self.endpoint = request.endpoint or request.path
29         self.method = request.method
30         return self
31     def elapsed(self):
32         if self.start_time:
33             return round((time.time() - self.start_time) * 1000, 2)
34         return 0
35     def to_dict(self):
36         return {
37             "request_id": self.request_id,
38             "client_ip": self.client_ip,
39             "endpoint": self.endpoint,
40             "method": self.method,
41             "elapsed_ms": self.elapsed(),
42         }
43 _request_ctx = threading.local()
44 def get_request_ctx() -> RequestContext:
45     if not hasattr(_request_ctx, "current"):
46         _request_ctx.current = RequestContext()
47     return _request_ctx.current
48 # —— 装饰器 ——
49 def request_context_middleware(app):
50     """为 Flask app 注册请求生命周期钩子"""
51     @app.before_request
52     def before_request():
53         ctx = get_request_ctx()
54         ctx.init_from_flask()
55         g.request_id = ctx.request_id
56         logger.info(f"[{ctx.request_id}] {ctx.method} {request.path} from {ctx.client_ip}")
57     @app.after_request
58     def after_request(response):
59         ctx = get_request_ctx()
60         elapsed = ctx.elapsed()
61         status = response.status_code
62         logger.info(f"[{ctx.request_id}] {status} ({elapsed}ms)")
63         response.headers.set("X-Request-ID", ctx.request_id)
64         response.headers.set("X-Response-Time", f"{elapsed}ms")
65         return response
66     return app
67 def cors_middleware(app, origins=None):
68     """为 Flask app 添加 CORS 头"""
69     if origins is None:
70         origins = ["*"]
71     @app.after_request

```



```

72     def add_cors_headers(response):
73         origin = request.headers.get("Origin", "")
74         if "*" in origins or origin in origins:
75             response.headers.set("Access-Control-Allow-Origin", origin)
76             response.headers.set("Access-Control-Allow-Methods",
77                                 "GET, POST, PUT, DELETE, OPTIONS")
78             response.headers.set("Access-Control-Allow-Headers",
79                                 "Content-Type, Authorization, X-Request-ID")
80             response.headers.set("Access-Control-Allow-Credentials", "true")
81         return response
82     return app
83 def rate_limit(max_per_minute=60):
84     """简易速率限制装饰器（进程内计数）"""
85     records = {}
86     def decorator(f):
87         @wraps(f)
88         def wrapper(*args, **kwargs):
89             now = time.time()
90             ip = request.remote_addr or "unknown"
91             window_start = now - 60
92             if ip not in records:
93                 records[ip] = []
94             records[ip] = [t for t in records[ip] if t > window_start]
95             if len(records[ip]) >= max_per_minute:
96                 logger.warning(f"速率限制触发: {ip}")
97                 return jsonify({"error": "请求过于频繁, 请稍后再试"}), 429
98             records[ip].append(now)
99             return f(*args, **kwargs)
100        return wrapper
101    return decorator
102 def timer(f):
103     """函数执行计时装饰器"""
104     @wraps(f)
105     def wrapper(*args, **kwargs):
106         start = time.time()
107         result = f(*args, **kwargs)
108         elapsed = round((time.time() - start) * 1000, 2)
109         logger.debug(f"{f.__name__} 执行耗时: {elapsed}ms")
110         return result
111    return wrapper
112 class PerformanceTracker:
113     """性能追踪器——记录接口调用统计"""
114     def __init__(self):
115         self._lock = threading.Lock()
116         self._stats = {}
117     def record(self, endpoint: str, elapsed_ms: float, status: int):
118         with self._lock:
119             if endpoint not in self._stats:
120                 self._stats[endpoint] = {
121                     "count": 0, "total_time": 0, "max_time": 0,
122                     "min_time": float("inf"), "error_count": 0,
123                 }
124             s = self._stats[endpoint]
125             s["count"] += 1
126             s["total_time"] += elapsed_ms
127             s["max_time"] = max(s["max_time"], elapsed_ms)
128             s["min_time"] = min(s["min_time"], elapsed_ms)
129             if status >= 400:
130                 s["error_count"] += 1
131     def summary(self):
132         result = {}
133         with self._lock:
134             for ep, s in self._stats.items():
135                 avg = round(s["total_time"] / s["count"], 2) if s["count"] else 0

```

```

136         err_rate = round(s["error_count"] / s["count"] * 100, 1) if s["count"] else 0
137         result[ep] = {
138             "calls": s["count"],
139             "avg_ms": avg,
140             "max_ms": round(s["max_time"], 2),
141             "min_ms": round(s["min_time"], 2) if s["min_time"] != float("inf") else 0,
142             "errors": s["error_count"],
143             "error_rate_pct": err_rate,
144         }
145     return result
146     def reset(self):
147         with self._lock:
148             self._stats.clear()
149     tracker = PerformanceTracker()

```

-- 文件结束: middleware.py --

文件: cache.py (144 行, 4.3 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  缓存模块 — 文件缓存 + 内存缓存, 支持 TTL
4  """
5  import json
6  import os
7  import time
8  import threading
9  from config import CACHE_DIR, CACHE_DEFAULT_TTL, CACHE_ENABLED
10 from logger import get_logger
11 logger = get_logger("cache")
12 class MemoryCache:
13     """线程安全的内存缓存"""
14     def __init__(self, default_ttl: int = CACHE_DEFAULT_TTL):
15         self._store = {}
16         self._lock = threading.RLock()
17         self.default_ttl = default_ttl
18     def get(self, key: str):
19         if not CACHE_ENABLED:
20             return None
21         with self._lock:
22             entry = self._store.get(key)
23             if entry is None:
24                 return None
25             if entry["expires"] is not None and time.time() > entry["expires"]:
26                 del self._store[key]
27                 return None
28             return entry["value"]
29     def set(self, key: str, value, ttl: int = None):
30         if not CACHE_ENABLED:
31             return
32         expires = None
33         if ttl is None and self.default_ttl > 0:
34             expires = time.time() + self.default_ttl
35         elif ttl and ttl > 0:
36             expires = time.time() + ttl
37         with self._lock:
38             self._store[key] = {"value": value, "expires": expires}
39     def delete(self, key: str):
40         with self._lock:
41             self._store.pop(key, None)
42     def clear(self):
43         with self._lock:
44             self._store.clear()
45     def size(self) -> int:
46         with self._lock:
47             return len(self._store)
48     def keys(self):

```

```

49         with self._lock:
50             return list(self._store.keys())
51 class FileCache:
52     """文件缓存 — 持久化到磁盘"""
53     def __init__(self, cache_dir: str = None, default_ttl: int = CACHE_DEFAULT_TTL):
54         self.cache_dir = cache_dir or CACHE_DIR
55         self.default_ttl = default_ttl
56         self._lock = threading.RLock()
57         if not os.path.exists(self.cache_dir):
58             os.makedirs(self.cache_dir, exist_ok=True)
59     def _path(self, key: str) -> str:
60         safe = key.replace("/", "_").replace("\\", "_").replace(":", "_")
61         return os.path.join(self.cache_dir, f"{safe}.json")
62     def get(self, key: str):
63         if not CACHE_ENABLED:
64             return None
65         path = self._path(key)
66         try:
67             with open(path, "r", encoding="utf-8") as f:
68                 entry = json.load(f)
69                 if entry.get("expires") and time.time() > entry["expires"]:
70                     os.remove(path)
71                     return None
72                 return entry["value"]
73         except (FileNotFoundError, json.JSONDecodeError, KeyError):
74             return None
75     def set(self, key: str, value, ttl: int = None):
76         if not CACHE_ENABLED:
77             return
78         expires = None
79         if ttl is None and self.default_ttl > 0:
80             expires = time.time() + self.default_ttl
81         elif ttl and ttl > 0:
82             expires = time.time() + ttl
83         with self._lock:
84             path = self._path(key)
85             with open(path, "w", encoding="utf-8") as f:
86                 json.dump({"value": value, "expires": expires}, f, ensure_ascii=False)
87     def delete(self, key: str):
88         path = self._path(key)
89         try:
90             os.remove(path)
91         except FileNotFoundError:
92             pass
93     def clear(self):
94         with self._lock:
95             for fname in os.listdir(self.cache_dir):
96                 if fname.endswith(".json"):
97                     os.remove(os.path.join(self.cache_dir, fname))
98     def size(self) -> int:
99         return len([f for f in os.listdir(self.cache_dir) if f.endswith(".json")])
100 # 全局单例
101 memory_cache = MemoryCache()
102 file_cache = FileCache()
103 class cached:
104     """装饰器：缓存函数返回值"""
105     def __init__(self, ttl: int = None, cache_backend: str = "memory"):
106         self.ttl = ttl
107         self.cache = memory_cache if cache_backend == "memory" else file_cache
108     def __call__(self, func):
109         def wrapper(*args, **kwargs):
110             key = f"{func.__module__}:{func.__name__}:{repr(args)}:{repr(sorted(kwargs.items()))}"
111             result = self.cache.get(key)
112             if result is not None:

```

```

113         return result
114         result = func(*args, **kwargs)
115         self.cache.set(key, result, ttl=self.ttl)
116         return result
117     return wrapper

```

-- 文件结束: cache.py --

文件: models.py (193 行, 5.1 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  数据模型层 — 诗歌、意象、分析单元的数据类定义
4  """
5  import json
6  from typing import List, Optional, Dict, Any
7  from dataclasses import dataclass, field, asdict
8  @dataclass
9  class PoemLine:
10     """诗行"""
11     诗行编号: str = ""
12     原文: str = ""
13     注释: str = ""
14     def to_dict(self):
15         return asdict(self)
16 @dataclass
17 class AnalysisUnit:
18     """分析单元（意象标注单元）"""
19     单元编号: str = ""
20     诗行编号: str = ""
21     文本: str = ""
22     行内位置: str = ""
23     词性: str = ""
24     成分类型: str = ""
25     是否意象: str = "0"
26     大类编码: str = ""
27     子类编码: str = ""
28     感知通道: str = ""
29     素材类型: str = ""
30     内部结构: str = ""
31     指涉来源: str = ""
32     表现功能: str = ""
33     文化流通性: str = ""
34     跨文化性: str = ""
35     认知强度: str = ""
36     核心意象: str = ""
37     结构功能组: str = ""
38     情感极性: str = ""
39     情感类别: str = ""
40     情感置信度: str = ""
41     def to_dict(self):
42         return asdict(self)
43     @property
44     def is_image(self) -> bool:
45         return self.是否意象 == "1"
46 @dataclass
47 class EmotionTrajectory:
48     """情感轨迹"""
49     诗行编号: str = ""
50     诗行原文: str = ""
51     平均情感极性: str = ""
52     主导情感: str = ""
53     情感波动值: str = ""
54     def to_dict(self):
55         return asdict(self)
56 @dataclass
57 class ImageRelation:

```

```

58     """意象关系"""
59     关系编号: str = ""
60     来源单元编号: str = ""
61     来源文本: str = ""
62     目标单元编号: str = ""
63     目标文本: str = ""
64     关系类型: str = ""
65     关系强度: str = ""
66     def to_dict(self):
67         return asdict(self)
68 @dataclass
69 class Poem:
70     """诗歌完整数据模型"""
71     诗歌编号: str = ""
72     标题: str = ""
73     作者: str = ""
74     朝代: str = "唐"
75     分类标签: str = ""
76     体裁: str = ""
77     原文: str = ""
78     诗行: List[PoeLine] = field(default_factory=list)
79     分析单元: List[AnalysisUnit] = field(default_factory=list)
80     情感轨迹: List[EmotionTrajectory] = field(default_factory=list)
81     意象关系: List[ImageRelation] = field(default_factory=list)
82     def to_dict(self):
83         return {
84             "诗歌编号": self.诗歌编号,
85             "标题": self.标题,
86             "作者": self.作者,
87             "分类标签": self.分类标签,
88             "体裁": self.体裁,
89             "原文": self.原文,
90             "诗行": [l.to_dict() for l in self.诗行],
91             "分析单元": [u.to_dict() for u in self.分析单元],
92             "情感轨迹": [t.to_dict() for t in self.情感轨迹],
93             "意象关系": [r.to_dict() for r in self.意象关系],
94         }
95     @property
96     def image_units(self) -> List[AnalysisUnit]:
97         return [u for u in self.分析单元 if u.is_image]
98     @property
99     def image_count(self) -> int:
100         return len(self.image_units)
101     @property
102     def summary(self) -> str:
103         return f"《{self.标题}》{self.作者} ({self.诗歌编号}) — {self.image_count}个意象"
104     @classmethod
105     def from_dict(cls, data: dict) -> "Poem":
106         lines = [PoeLine(**l) for l in data.get("诗行", []) if isinstance(l, dict)]
107         units = [AnalysisUnit(**u) for u in data.get("分析单元", []) if isinstance(u, dict)]
108         trajs = [EmotionTrajectory(**t) for t in data.get("情感轨迹", []) if isinstance(t, dict)]
109         rels = [ImageRelation(**r) for r in data.get("意象关系", []) if isinstance(r, dict)]
110         return cls(
111             诗歌编号=data.get("诗歌编号", ""),
112             标题=data.get("标题", ""),
113             作者=data.get("作者", ""),
114             分类标签=data.get("分类标签", ""),
115             体裁=data.get("体裁", ""),
116             原文=data.get("原文", ""),
117             诗行=lines, 分析单元=units,
118             情感轨迹=trajs, 意象关系=rels,
119         )
120 @dataclass
121 class TracebackItem:

```

```

122     """溯源表条目（前台展示用）"""
123     poem_id: str = ""
124     title: str = ""
125     author: str = ""
126     genre: str = ""
127     cat: str = ""
128     txt: str = ""
129     text: str = ""
130     dimensions: str = ""
131     emotion: str = ""
132     emo_cat: str = ""
133     line: str = ""
134     id: str = ""
135     词性: str = ""
136     成分类型: str = ""
137     感知通道: str = ""
138     素材类型: str = ""
139     内部结构: str = ""
140     指涉来源: str = ""
141     表现功能: str = ""
142     文化流通性: str = ""
143     跨文化性: str = ""
144     认知强度: str = ""
145     核心意象: str = ""
146     结构功能组: str = ""
147     情感极性: str = ""
148     情感类别: str = ""
149     情感置信度: str = ""
150     大类编码: str = ""
151     子类编码: str = ""
152     def to_dict(self):
153         return asdict(self)
154 @dataclass
155 class AnalysisResult:
156     """解析结果"""
157     item: Dict[str, Any] = field(default_factory=dict)
158     messages: List[Dict] = field(default_factory=list)
159     done: bool = False
160     is_streaming: bool = False
161     def to_dict(self):
162         return {
163             "done": self.done,
164             "is_streaming": self.is_streaming,
165             "message_count": len(self.messages),
166         }

```

-- 文件结束: models.py --

文件: analytics.py (313 行, 11.0 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  数据统计分析服务 — 多维度聚合统计
4  """
5  import json
6  from collections import Counter, defaultdict
7  from typing import List, Dict, Any
8  from config import CATEGORY_NAME_MAP, CATEGORY_MAJOR_MAP
9  from logger import get_logger
10 logger = get_logger("analytics")
11 class StatsService:
12     """意象数据统计分析服务"""
13     def __init__(self, traceback_data: List[Dict] = None):
14         self.data = traceback_data or []
15         self._cache = {}
16     def set_data(self, data: List[Dict]):
17         self.data = data

```

```

18     self._cache.clear()
19 # —— 基础统计 ——
20 def total_images(self) -> int:
21     return len(self.data)
22 def total_poems(self) -> int:
23     return len({item.get("poem_id") for item in self.data if item.get("poem_id")})
24 def total_authors(self) -> int:
25     return len({item.get("author") for item in self.data if item.get("author")})
26 # —— 意象频次统计 ——
27 def top_images(self, n: int = 50) -> List[Dict]:
28     counter = Counter()
29     for item in self.data:
30         txt = item.get("文本") or item.get("txt") or ""
31         if txt:
32             counter[txt] += 1
33     return [{"text": txt, "count": cnt} for txt, cnt in counter.most_common(n)]
34 # —— 分类统计 ——
35 def category_distribution(self) -> List[Dict]:
36     counter = Counter()
37     for item in self.data:
38         cat = item.get("cat", "未分类")
39         counter[cat] += 1
40     total = sum(counter.values()) or 1
41     return [
42         {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
43         for cat, cnt in counter.most_common()
44     ]
45 def major_category_distribution(self) -> List[Dict]:
46     counter = Counter()
47     for item in self.data:
48         code = str(item.get("大类编码", ""))
49         major = CATEGORY_MAJOR_MAP.get(code, "其他")
50         counter[major] += 1
51     total = sum(counter.values()) or 1
52     return [
53         {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
54         for cat, cnt in counter.most_common()
55     ]
56 def sub_category_distribution(self, major_code: str = None) -> List[Dict]:
57     counter = Counter()
58     for item in self.data:
59         code = str(item.get("子类编码", ""))
60         if major_code and not code.startswith(major_code):
61             continue
62         cat = CATEGORY_NAME_MAP.get(code, f"未知子类 ({code})")
63         counter[cat] += 1
64     total = sum(counter.values()) or 1
65     return [
66         {"category": cat, "count": cnt, "percentage": round(cnt / total * 100, 2)}
67         for cat, cnt in counter.most_common()
68     ]
69 # —— 诗人统计 ——
70 def author_statistics(self) -> List[Dict]:
71     author_images = defaultdict(list)
72     for item in self.data:
73         author = item.get("author") or item.get("作者") or "未知"
74         txt = item.get("文本") or item.get("txt") or ""
75         if txt:
76             author_images[author].append(txt)
77     results = []
78     for author, images in sorted(author_images.items(), key=lambda x: -len(x[1])):
79         unique_images = set(images)
80         top = Counter(unique_images).most_common(5)
81         results.append({

```

```

82         "author": author,
83         "image_count": len(images),
84         "unique_image_count": len(unique_images),
85         "poem_count": len({
86             item.get("poem_id") for item in self.data
87             if (item.get("author") or item.get("作者")) == author
88         }),
89         "top_images": [{"text": t, "count": c} for t, c in top],
90     })
91     return results
92 # —— 情感分析 ——
93 def emotion_distribution(self) -> List[Dict]:
94     counter = Counter()
95     for item in self.data:
96         emo = item.get("情感类别") or item.get("emo_cat") or "未知"
97         counter[emo] += 1
98     total = sum(counter.values()) or 1
99     return [
100         {"emotion": emo, "count": cnt, "percentage": round(cnt / total * 100, 2)}
101         for emo, cnt in counter.most_common()
102     ]
103 def emotion_polarity_distribution(self) -> List[Dict]:
104     counter = Counter()
105     for item in self.data:
106         pol = item.get("情感极性") or "未标注"
107         counter[pol] += 1
108     total = sum(counter.values()) or 1
109     return [
110         {"polarity": pol, "count": cnt, "percentage": round(cnt / total * 100, 2)}
111         for pol, cnt in counter.most_common()
112     ]
113 # —— 感知通道统计 ——
114 def perception_channel_distribution(self) -> List[Dict]:
115     counter = Counter()
116     for item in self.data:
117         ch = item.get("感知通道") or "未标注"
118         counter[ch] += 1
119     total = sum(counter.values()) or 1
120     return [
121         {"channel": ch, "count": cnt, "percentage": round(cnt / total * 100, 2)}
122         for ch, cnt in counter.most_common()
123     ]
124 # —— 诗词体裁统计 ——
125 def genre_distribution(self) -> List[Dict]:
126     """按诗歌体裁分类统计"""
127     counter = Counter()
128     for item in self.data:
129         genre = item.get("genre", "未分类")
130         if genre:
131             counter[genre] += 1
132     total = sum(counter.values()) or 1
133     return [
134         {"genre": g, "count": cnt, "percentage": round(cnt / total * 100, 2)}
135         for g, cnt in counter.most_common()
136     ]
137 # —— 内部结构统计 ——
138 def internal_structure_distribution(self) -> List[Dict]:
139     """内部结构维度统计"""
140     counter = Counter()
141     for item in self.data:
142         val = item.get("内部结构") or "未标注"
143         counter[val] += 1
144     total = sum(counter.values()) or 1
145     return [

```



```

146         {"structure": s, "count": cnt, "percentage": round(cnt / total * 100, 2)}
147     for s, cnt in counter.most_common()
148 ]
149 # —— 表现功能统计 ——
150 def function_distribution(self) -> List[Dict]:
151     """表现功能维度统计"""
152     counter = Counter()
153     for item in self.data:
154         val = item.get("表现功能") or "未标注"
155         counter[val] += 1
156     total = sum(counter.values()) or 1
157     return [
158         {"function": f, "count": cnt, "percentage": round(cnt / total * 100, 2)}
159         for f, cnt in counter.most_common()
160     ]
161 # —— 核心意象统计 ——
162 def core_image_distribution(self) -> List[Dict]:
163     """核心意象标注统计"""
164     counter = Counter()
165     for item in self.data:
166         val = item.get("核心意象") or "未标注"
167         counter[val] += 1
168     total = sum(counter.values()) or 1
169     return [
170         {"core_image": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
171         for c, cnt in counter.most_common()
172     ]
173 # —— 指涉来源统计 ——
174 def reference_source_distribution(self) -> List[Dict]:
175     """指涉来源维度统计"""
176     counter = Counter()
177     for item in self.data:
178         val = item.get("指涉来源") or "未标注"
179         counter[val] += 1
180     total = sum(counter.values()) or 1
181     return [
182         {"source": s, "count": cnt, "percentage": round(cnt / total * 100, 2)}
183         for s, cnt in counter.most_common()
184     ]
185 # —— 认知强度统计 ——
186 def cognitive_intensity_distribution(self) -> List[Dict]:
187     """认知强度维度分布"""
188     counter = Counter()
189     for item in self.data:
190         val = item.get("认知强度") or "未标注"
191         counter[val] += 1
192     total = sum(counter.values()) or 1
193     return [
194         {"intensity": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
195         for c, cnt in counter.most_common()
196     ]
197 # —— 素材类型统计 ——
198 def material_type_distribution(self) -> List[Dict]:
199     """素材类型维度统计"""
200     counter = Counter()
201     for item in self.data:
202         val = item.get("素材类型") or "未标注"
203         counter[val] += 1
204     total = sum(counter.values()) or 1
205     return [
206         {"material": m, "count": cnt, "percentage": round(cnt / total * 100, 2)}
207         for m, cnt in counter.most_common()
208     ]
209 # —— 跨文化性统计 ——

```

```

210 def cross_cultural_distribution(self) -> List[Dict]:
211     """跨文化性维度统计"""
212     counter = Counter()
213     for item in self.data:
214         val = item.get("跨文化性") or "未标注"
215         counter[val] += 1
216     total = sum(counter.values()) or 1
217     return [
218         {"level": c, "count": cnt, "percentage": round(cnt / total * 100, 2)}
219         for c, cnt in counter.most_common()
220     ]
221 # —— 综合报告 ——
222 def summary_report(self) -> Dict:
223     return {
224         "total_images": self.total_images(),
225         "total_poems": self.total_poems(),
226         "total_authors": self.total_authors(),
227         "category_count": len(self.category_distribution()),
228         "emotion_categories": len(self.emotion_distribution()),
229         "top_images": self.top_images(20),
230         "category_stats": self.category_distribution(),
231         "major_category_stats": self.major_category_distribution(),
232         "emotion_stats": self.emotion_distribution(),
233         "emotion_polarity_stats": self.emotion_polarity_distribution(),
234         "perception_channel_stats": self.perception_channel_distribution(),
235         "genre_stats": self.genre_distribution(),
236         "internal_structure_stats": self.internal_structure_distribution(),
237         "function_stats": self.function_distribution(),
238         "core_image_stats": self.core_image_distribution(),
239         "reference_source_stats": self.reference_source_distribution(),
240         "cognitive_intensity_stats": self.cognitive_intensity_distribution(),
241         "material_type_stats": self.material_type_distribution(),
242         "cross_cultural_stats": self.cross_cultural_distribution(),
243     }
244 # —— 跨表分析 ——
245 def cross_analysis(self) -> Dict:
246     """情感 × 分类域 交叉分析"""
247     cross = defaultdict(lambda: defaultdict(int))
248     for item in self.data:
249         emo = item.get("情感类别") or "未知"
250         cat = item.get("cat", "未分类")
251         cross[cat][emo] += 1
252     result = {}
253     for cat, emotions in cross.items():
254         total = sum(emotions.values())
255         result[cat] = [
256             {"emotion": emo, "count": cnt, "percentage": round(cnt / total * 100, 2)}
257             for emo, cnt in sorted(emotions.items(), key=lambda x: -x[1])
258         ]
259     return result
260 def build_analytics_api(stats_service: StatsService) -> Dict:
261     """生成供 API 返回的分析数据"""
262     return {
263         "summary": stats_service.summary_report(),
264         "cross_analysis": stats_service.cross_analysis(),
265     }

```

— 文件结束: analytics.py —

文件: export_service.py (118 行, 4.0 KB)

```

1  #-*- coding: utf-8 -*-
2  """
3  数据导出服务 — CSV / JSON / 报告导出
4  """
5  import csv
6  import json

```

```

7 import os
8 import time
9 from typing import List, Dict, Optional
10 from config import EXPORT_DIR, EXPORT_CSV_ENCODING
11 from errors import ExportError
12 from logger import get_logger
13 logger = get_logger("export")
14 def ensure_export_dir():
15     if not os.path.exists(EXPORT_DIR):
16         os.makedirs(EXPORT_DIR, exist_ok=True)
17 class ExportService:
18     """数据导出服务"""
19     @staticmethod
20     def export_traceback_to_csv(data: List[Dict], filename: str = None) -> str:
21         """将溯源数据导出为 CSV"""
22         ensure_export_dir()
23         filename = filename or f"traceback_export_{int(time.time())}.csv"
24         filepath = os.path.join(EXPORT_DIR, filename)
25         if not data:
26             raise ExportError("没有数据可导出")
27         fieldnames = [
28             "poem_id", "title", "genre", "cat", "txt",
29             "emotion", "line", "词性", "成分类型", "感知通道",
30             "素材类型", "内部结构", "指涉来源", "表现功能",
31             "文化流通性", "跨文化性", "认知强度", "核心意象",
32             "结构功能组", "情感极性", "情感类别", "情感置信度",
33             "大类编码", "子类编码",
34         ]
35         try:
36             with open(filepath, "w", encoding=EXPORT_CSV_ENCODING, newline="") as f:
37                 writer = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
38                 writer.writeheader()
39                 for row in data:
40                     writer.writerow(row)
41                 logger.info(f"CSV 导出成功: {filepath} ({len(data)} 条)")
42                 return filepath
43         except IOError as e:
44             raise ExportError(f"CSV 写入失败: {e}")
45     @staticmethod
46     def export_to_json(data, filename: str = None, indent: int = 2) -> str:
47         """将数据导出为 JSON"""
48         ensure_export_dir()
49         filename = filename or f"export_{int(time.time())}.json"
50         filepath = os.path.join(EXPORT_DIR, filename)
51         try:
52             with open(filepath, "w", encoding="utf-8") as f:
53                 json.dump(data, f, ensure_ascii=False, indent=indent)
54                 logger.info(f"JSON 导出成功: {filepath}")
55                 return filepath
56         except (IOError, TypeError) as e:
57             raise ExportError(f"JSON 写入失败: {e}")
58     @staticmethod
59     def export_summary_report(report: Dict, filename: str = None) -> str:
60         """导出统计分析报告 (JSON 格式)"""
61         return ExportService.export_to_json(report, filename or f"report_{int(time.time())}.json")
62     @staticmethod
63     def export_filtered_slice(
64         data: List[Dict],
65         start: int = 0,
66         end: int = None,
67         filename: str = None,
68     ) -> str:
69         """导出数据子集"""
70         subset = data[start:end] if end else data[start:]

```

```

71         return ExportService.export_to_json(
72             {"total": len(data), "start": start, "end": end or len(data),
73              "count": len(subset), "data": subset},
74             filename or f"slice_{start}_{end or len(data)}.json",
75         )
76     @staticmethod
77     def list_exports() -> List[Dict]:
78         """列出已导出的文件"""
79         ensure_export_dir()
80         files = []
81         for fname in os.listdir(EXPORT_DIR):
82             fpath = os.path.join(EXPORT_DIR, fname)
83             if os.path.isfile(fpath):
84                 files.append({
85                     "name": fname,
86                     "path": fpath,
87                     "size": os.path.getsize(fpath),
88                     "mtime": os.path.getmtime(fpath),
89                 })
90         return sorted(files, key=lambda x: -x["mtime"])
91     @staticmethod
92     def clear_exports() -> int:
93         """清空导出目录"""
94         ensure_export_dir()
95         count = 0
96         for fname in os.listdir(EXPORT_DIR):
97             fpath = os.path.join(EXPORT_DIR, fname)
98             if os.path.isfile(fpath):
99                 os.remove(fpath)
100             count += 1
101         logger.info(f"清空导出目录: 移除 {count} 个文件")
102         return count

```

-- 文件结束: export_service.py --

文件: preprocessor.py (149 行, 5.1 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  数据预处理模块 — JSON 清洗、格式校验、批量转换
4  """
5  import json
6  import os
7  import re
8  import glob
9  import shutil
10 from typing import List, Dict, Optional, Tuple
11 from config import POEM_JSON_DIR, BASE_DIR
12 from logger import get_logger
13 from models import Poem, AnalysisUnit
14 logger = get_logger("preprocessor")
15 def clean_json_content(content: str) -> str:
16     """清理 JSON 文本中的 Markdown 包裹和不规范字符"""
17     content = re.sub(r"```\s*", "", content, flags=re.MULTILINE)
18     content = re.sub(r"```\s*", "", content, flags=re.MULTILINE)
19     content = re.sub(r"\s*", "", content)
20     content = re.sub(r"\s*", "", content)
21     content = content.strip()
22     return content
23 def validate_poem_json(data: dict) -> List[str]:
24     """验证诗歌 JSON 结构, 返回错误列表"""
25     errors = []
26     if not isinstance(data, dict):
27         return ["根节点不是字典"]
28     poems = data.get("诗歌集", [data]) if isinstance(data, dict) else data
29     if not isinstance(poems, list):
30         poems = [poems]

```

```

31     for i, poem in enumerate(poems):
32         prefix = f"[诗歌 #{i+1}]"
33         if not poem.get("标题"):
34             errors.append(f"{prefix} 缺少标题字段")
35         if not poem.get("诗歌编号") and not poem.get("编号"):
36             errors.append(f"{prefix} 缺少诗歌编号")
37         if not poem.get("原文") and not poem.get("诗行"):
38             errors.append(f"{prefix} 缺少原文或诗行")
39         if "分析单元" in poem:
40             for j, unit in enumerate(poem["分析单元"]):
41                 if not isinstance(unit, dict):
42                     errors.append(f"{prefix} 分析单元 #{j+1} 不是字典")
43                     continue
44                 if not unit.get("文本"):
45                     errors.append(f"{prefix} 分析单元 #{j+1} 缺少文本")
46                 if not unit.get("诗行编号"):
47                     errors.append(f"{prefix} 分析单元 #{j+1} 缺少诗行编号")
48     return errors
49 def parse_json_file(filepath: str) -> Tuple[Optional[dict], List[str]]:
50     """安全解析 JSON 文件, 返回 (data, errors)"""
51     try:
52         with open(filepath, "r", encoding="utf-8") as f:
53             content = f.read()
54     except IOError as e:
55         return None, [f"读取失败: {e}"]
56     content = clean_json_content(content)
57     try:
58         data = json.loads(content)
59     except json.JSONDecodeError as e:
60         return None, [f"JSON 解析失败: {e}"]
61     errors = validate_poem_json(data)
62     return data, errors
63 def batch_validate(directory: str = None) -> Dict:
64     """批量校验目录下所有 JSON 文件"""
65     directory = directory or POEM_JSON_DIR
66     if not os.path.exists(directory):
67         return {"total": 0, "valid": 0, "invalid": 0, "details": []}
68     files = glob.glob(os.path.join(directory, "*.json"))
69     results = {"total": len(files), "valid": 0, "invalid": 0, "details": []}
70     for fpath in sorted(files):
71         fname = os.path.basename(fpath)
72         data, errors = parse_json_file(fpath)
73         if errors:
74             results["invalid"] += 1
75             results["details"].append({"file": fname, "status": "invalid",
76                                         "errors": errors})
77             logger.warning(f"校验失败: {fname} — {'; '.join(errors)}")
78         else:
79             results["valid"] += 1
80             results["details"].append({"file": fname, "status": "valid",
81                                         "poem_count": len(
82                                             data.get("诗歌集", [data])
83                                             if isinstance(data, dict) else data
84                                         )))
85             logger.info(f"校验通过: {fname}")
86     return results
87 def backup_data(directory: str = None) -> str:
88     """备份数据目录"""
89     src = directory or POEM_JSON_DIR
90     if not os.path.exists(src):
91         logger.warning(f"备份源目录不存在: {src}")
92         return ""
93     backup_name = f"backup_poem_json_{__import__('time').time():.0f}"
94     dst = os.path.join(BASE_DIR, backup_name)

```

```

95     shutil.copytree(src, dst)
96     logger.info(f"数据备份完成: {src} -> {dst}")
97     return dst
98 def convert_raw_poem_to_model(poem_dict: dict) -> Poem:
99     """将原始 JSON 字典转为 Poem 模型"""
100     return Poem.from_dict(poem_dict)
101 def extract_image_statistics(poems: List[Poem]) -> Dict:
102     """从 Poem 模型列表提取意象统计"""
103     total_images = 0
104     image_texts = []
105     author_images = {}
106     for poem in poems:
107         images = poem.image_units
108         total_images += len(images)
109         for u in images:
110             image_texts.append(u.文本)
111         author = poem.作者 or "未知"
112         if author not in author_images:
113             author_images[author] = []
114         author_images[author].extend(u.文本 for u in images)
115     from collections import Counter
116     top = Counter(image_texts).most_common(20)
117     return {
118         "total_poems": len(poems),
119         "total_images": total_images,
120         "unique_images": len(set(image_texts)),
121         "top_images": [{"text": t, "count": c} for t, c in top],
122         "authors": len(author_images),
123     }
-- 文件结束: preprocessor.py --
文件: monitor.py (186 行, 5.6 KB)
1  # -*- coding: utf-8 -*-
2  """
3  系统监控模块 — 资源使用、接口统计、健康检查
4  """
5  import os
6  import time
7  import threading
8  from config import BASE_DIR
9  from logger import get_logger, LoggerMixin
10 logger = get_logger("monitor")
11 class SystemMonitor(LoggerMixin):
12     """系统资源监控"""
13     def __init__(self, interval: int = 60):
14         self.interval = interval
15         self._running = False
16         self._thread = None
17         self._snapshots = []
18         self._lock = threading.Lock()
19     @property
20     def disk_usage(self) -> dict:
21         """磁盘使用情况"""
22         try:
23             import shutil
24             total, used, free = shutil.disk_usage(BASE_DIR)
25             return {
26                 "total_gb": round(total / (1024**3), 2),
27                 "used_gb": round(used / (1024**3), 2),
28                 "free_gb": round(free / (1024**3), 2),
29                 "usage_pct": round(used / total * 100, 1) if total else 0,
30             }
31         except Exception:
32             return {"error": "无法获取磁盘信息"}
33     @property

```

```

34 def memory_info(self) -> dict:
35     """内存使用信息"""
36     try:
37         import psutil
38         mem = psutil.virtual_memory()
39         return {
40             "total_gb": round(mem.total / (1024**3), 2),
41             "available_gb": round(mem.available / (1024**3), 2),
42             "usage_pct": mem.percent,
43         }
44     except ImportError:
45         return {"note": "需安装 psutil (pip install psutil)"}
46     except Exception as e:
47         return {"error": str(e)}
48 @property
49 def cpu_info(self) -> dict:
50     """CPU 使用信息"""
51     try:
52         import psutil
53         return {
54             "cpu_percent": psutil.cpu_percent(interval=0.5),
55             "cpu_count": psutil.cpu_count(),
56             "load_avg": os.getloadavg() if hasattr(os, "getloadavg") else None,
57         }
58     except ImportError:
59         return {"note": "需安装 psutil"}
60     except Exception as e:
61         return {"error": str(e)}
62 @property
63 def python_info(self) -> dict:
64     """Python 运行时信息"""
65     import sys
66     return {
67         "version": sys.version,
68         "executable": sys.executable,
69         "platform": sys.platform,
70     }
71 @property
72 def project_info(self) -> dict:
73     """项目文件信息"""
74     py_files = 0
75     py_lines = 0
76     for root, dirs, files in os.walk(BASE_DIR):
77         dirs[:] = [d for d in dirs if d not in ("__pycache__", "rag_db",
78             "node_modules", ".git")]
79         for f in files:
80             if f.endswith(".py"):
81                 py_files += 1
82                 fpath = os.path.join(root, f)
83                 try:
84                     with open(fpath, "r", encoding="utf-8") as fh:
85                         py_lines += sum(1 for _ in fh)
86                 except Exception:
87                     pass
88     return {
89         "python_files": py_files,
90         "python_lines": py_lines,
91         "project_dir": BASE_DIR,
92     }
93 def snapshot(self) -> dict:
94     """获取当前系统快照"""
95     snap = {
96         "timestamp": time.time(),
97         "time_str": time.strftime("%Y-%m-%d %H:%M:%S"),

```

```

98         "disk": self.disk_usage,
99         "memory": self.memory_info,
100        "cpu": self.cpu_info,
101        "python": self.python_info,
102        "project": self.project_info,
103    }
104    self.log_debug(f"系统快照已记录")
105    return snap
106    def collect(self) -> dict:
107        """收集并记录快照"""
108        snap = self.snapshot()
109        with self._lock:
110            self._snapshots.append(snap)
111            if len(self._snapshots) > 100:
112                self._snapshots = self._snapshots[-100:]
113        return snap
114    def start_background_collection(self):
115        """启动后台定时收集"""
116        if self._running:
117            return
118        self._running = True
119        self._thread = threading.Thread(target=self._run_loop, daemon=True)
120        self._thread.start()
121        self.log_info(f"后台监控已启动 (间隔 {self.interval}s)")
122    def stop_background_collection(self):
123        """停止后台收集"""
124        self._running = False
125        if self._thread:
126            self._thread.join(timeout=5)
127            self._thread = None
128        self.log_info("后台监控已停止")
129    def _run_loop(self):
130        while self._running:
131            try:
132                self.collect()
133            except Exception as e:
134                self.log_error(f"监控采集异常: {e}")
135            time.sleep(self.interval)
136    def get_history(self, count: int = 10) -> list:
137        """获取最近 N 次快照"""
138        with self._lock:
139            return self._snapshots[-count:]
140    def health_check(self) -> dict:
141        """健康检查"""
142        snap = self.collect()
143        issues = []
144        disk = snap.get("disk", {})
145        if disk.get("usage_pct", 0) > 90:
146            issues.append(f"磁盘使用率过高: {disk['usage_pct']}%")
147        mem = snap.get("memory", {})
148        if mem.get("usage_pct", 0) > 90:
149            issues.append(f"内存使用率过高: {mem['usage_pct']}%")
150        return {
151            "status": "unhealthy" if issues else "healthy",
152            "timestamp": snap["timestamp"],
153            "time_str": snap["time_str"],
154            "issues": issues,
155            "disk_usage_pct": disk.get("usage_pct"),
156            "memory_usage_pct": mem.get("usage_pct"),
157        }
158    # 全局单例
159    system_monitor = SystemMonitor()
160    def get_system_monitor() -> SystemMonitor:
161        return system_monitor

```


-- 文件结束: monitor.py --

文件: build_rag.py (122 行, 3.6 KB)

```
1  # -*- coding: utf-8 -*-
2  """
3  RAG 向量数据库构建脚本
4  """
5  import os
6  import json
7  import time
8  import shutil
9  import chromadb
10 import requests
11 from config import (
12     EMBED_API_URL, EMBED_MODEL, RAG_DB_DIR, RAG_COLLECTION_NAME,
13     POEMS_JSON_DIR, EMBEDDING_RETRIES, EMBEDDING_TIMEOUT,
14 )
15 from errors import EmbeddingError
16 from logger import get_logger
17 logger = get_logger("build_rag")
18 POEMS_DIR = POEMS_JSON_DIR
19 def get_embedding(text, retries=EMBEDDING_RETRIES):
20     """获取文本向量, 带重试"""
21     for attempt in range(retries):
22         try:
23             response = requests.post(
24                 EMBED_API_URL,
25                 json={"model": EMBED_MODEL, "input": text},
26                 timeout=EMBEDDING_TIMEOUT,
27             )
28             if response.status_code == 200:
29                 return response.json()["data"][0]["embedding"]
30             else:
31                 logger.warning(f"API 异常: {response.status_code}, 重试 {attempt+1}")
32                 time.sleep(2)
33         except requests.RequestException as e:
34             logger.warning(f"请求失败: {e}, 重试 {attempt+1}")
35             time.sleep(2)
36     raise EmbeddingError(f"向量化失败, 已重试 {retries} 次")
37 def main():
38     """主构建流程"""
39     if os.path.exists(RAG_DB_DIR):
40         shutil.rmtree(RAG_DB_DIR)
41         logger.info(f"已清除旧向量库: {RAG_DB_DIR}")
42     logger.info(f"RAG 向量库构建启动")
43     logger.info(f"读取目录: {POEMS_DIR}")
44     logger.info(f"保存至: {RAG_DB_DIR}")
45     client = chromadb.PersistentClient(path=RAG_DB_DIR)
46     collection = client.get_or_create_collection(
47         name=RAG_COLLECTION_NAME,
48         metadata={"hnsw:space": "cosine"},
49     )
50     if not os.path.exists(POEMS_DIR):
51         logger.error(f"数据目录不存在: {POEMS_DIR}")
52         return
53     all_files = [f for f in os.listdir(POEMS_DIR) if f.endswith(".json")]
54     logger.info(f"共发现 {len(all_files)} 个文件")
55     success_count = 0
56     error_count = 0
57     for filename in all_files:
58         file_path = os.path.join(POEMS_DIR, filename)
59         try:
60             with open(file_path, "r", encoding="utf-8") as f:
61                 data = json.load(f)
62             except Exception as e:
```

```

63         logger.error(f"读取失败: {filename} | {e}")
64         error_count += 1
65         continue
66     poems_in_file = data.get("诗歌集", [data]) if isinstance(data, dict) else data
67     for poem in poems_in_file:
68         pid = poem.get("诗歌编号", "")
69         title = poem.get("标题", "未知")
70         author = poem.get("作者", "未知")
71         text = poem.get("原文", "")
72         if not text or not pid:
73             logger.warning(f"跳过空内容或无编号: {title}")
74             continue
75         logger.info(f"[{pid}] 《{title}》 | {author} | {len(text)}字")
76         try:
77             vector = get_embedding(text)
78         except EmbeddingError:
79             logger.error(f"向量化失败, 跳过: {pid}")
80             error_count += 1
81             continue
82         try:
83             collection.add(
84                 ids=[pid],
85                 embeddings=[vector],
86                 documents=[text],
87                 metadatas=[{"标题": title, "作者": author, "诗歌编号": pid}],
88             )
89             logger.info(f"入库成功: {pid}")
90             success_count += 1
91         except Exception as e:
92             logger.error(f"入库失败: {pid} | {e}")
93             error_count += 1
94         time.sleep(0.3)
95     logger.info(f"构建完成! 成功: {success_count} | 失败: {error_count}")
96 if __name__ == "__main__":
97     main()

```

-- 文件结束: build_rag.py --

文件: query_rag.py (238 行, 8.0 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  RAG 检索与流式问答引擎
4  """
5  import json
6  import chromadb
7  import requests
8  from config import (
9      EMBED_API_URL, CHAT_API_URL, EMBED_MODEL, CHAT_MODEL,
10     RAG_DB_DIR, RAG_COLLECTION_NAME, TOP_K, KNOWN_AUTHORS,
11     EMBEDDING_RETRIES, EMBEDDING_TIMEOUT, CHAT_TIMEOUT,
12 )
13 from errors import EmbeddingError, LLMError
14 from logger import get_logger
15 logger = get_logger("query_rag")
16 def get_embedding(text):
17     """获取文本向量"""
18     for attempt in range(EMBEDDING_RETRIES):
19         try:
20             response = requests.post(
21                 EMBED_API_URL,
22                 json={"model": EMBED_MODEL, "input": text},
23                 timeout=EMBEDDING_TIMEOUT,
24             )
25             if response.status_code == 200:
26                 return response.json()[0]["data"][0]["embedding"]
27             else:

```

```

28         logger.warning(f"Embedding API 异常: {response.status_code}, 重试 {attempt+1}")
29     except requests.RequestException as e:
30         logger.warning(f"Embedding 请求失败: {e}, 重试 {attempt+1}")
31     raise EmbeddingError(f"向量化失败, 已重试 {EMBEDDING_RETRIES} 次")
32 def extract_intent(question):
33     """从问题中提取意图 (检测诗人、清理无关词)"""
34     detected_author = None
35     for author in KNOWN_AUTHORS:
36         if author in question:
37             detected_author = author
38             break
39     clean_question = question
40     if detected_author:
41         clean_question = question.replace(detected_author, "").strip()
42     for word in ["写过", "写了", "哪些", "有哪些", "的诗", "哪首", "什么诗", "请问", "帮我找"]:
43         clean_question = clean_question.replace(word, "").strip()
44     if not clean_question:
45         clean_question = question
46     return detected_author, clean_question
47 def search_poems(question, top_k=TOP_K):
48     """在向量库中检索相关诗歌"""
49     client = chromadb.PersistentClient(path=RAG_DB_DIR)
50     collection = client.get_or_create_collection(
51         name=RAG_COLLECTION_NAME,
52         metadata={"hnsw:space": "cosine"},
53     )
54     detected_author, semantic_query = extract_intent(question)
55     vector = get_embedding(semantic_query)
56     if detected_author:
57         try:
58             results = collection.query(
59                 query_embeddings=[vector],
60                 n_results=top_k,
61                 where={"作者": detected_author},
62             )
63         except Exception:
64             logger.warning(f"按作者过滤失败, 回退到无过滤检索")
65             results = collection.query(
66                 query_embeddings=[vector],
67                 n_results=top_k,
68             )
69     else:
70         results = collection.query(
71             query_embeddings=[vector],
72             n_results=top_k,
73         )
74     poems = []
75     for i in range(len(results["documents"][0])):
76         poems.append({
77             "原文": results["documents"][0][i],
78             "标题": results["metadatas"][0][i].get("标题", ""),
79             "作者": results["metadatas"][0][i].get("作者", ""),
80             "相似度": round(1 - results["distances"][0][i], 3),
81         })
82     return poems
83 def _stream_chat_completion(messages, temperature=0.7, max_tokens=20000):
84     """流式调用大模型"""
85     try:
86         response = requests.post(
87             CHAT_API_URL,
88             json={
89                 "model": CHAT_MODEL,
90                 "messages": messages,
91                 "temperature": temperature,

```

```

92         "max_tokens": max_tokens,
93         "stream": True,
94     },
95     timeout=CHAT_TIMEOUT,
96     stream=True,
97 )
98 response.raise_for_status()
99 except requests.RequestException as e:
100     raise LLMError(f"大模型服务请求失败: {e}")
101 for line in response.iter_lines():
102     if not line:
103         continue
104     line_text = line.decode("utf-8")
105     if line_text.startswith("data: "):
106         line_text = line_text[6:]
107     if line_text == "[DONE]":
108         break
109     try:
110         chunk = json.loads(line_text)
111         if not chunk.get("choices"):
112             continue
113         delta = chunk["choices"][0].get("delta", {})
114         content = delta.get("content", "")
115         if content:
116             yield content
117     except json.JSONDecodeError:
118         continue
119 def build_context_block(poems):
120     """构建检索到的诗歌上下文"""
121     parts = []
122     for p in poems:
123         parts.append(f"《{p['标题']}》{p['作者']}\n{p['原文']}\n")
124     return "\n".join(parts)
125 def _build_rag_messages(question, history):
126     """构建 RAG 问答的消息列表"""
127     history = history or []
128     system = (
129         "你是一位精通中国古典诗歌意象分析的学者，回答详尽准确，"
130         "严格基于提供的资料，结合诗句原文进行分析。"
131     )
132     if not history:
133         poems = search_poems(question)
134         if not poems:
135             return None, None
136         ctx = build_context_block(poems)
137         user_content = (
138             f"你是一位精通中国古典诗歌的学者，请基于以下检索到的诗歌资料详细回答问题。\\n"
139             f"回答时请结合具体诗句分析，给出深入的文学解读。\\n"
140             f"注意：只基于提供的诗歌资料回答，不要编造不在资料中的诗歌。\\n\\n"
141             f"诗歌资料：\\n{ctx}\\n\\n问题：{question}\\n\\n请详细回答："
142         )
143     messages = [
144         {"role": "system", "content": system},
145         {"role": "user", "content": user_content},
146     ]
147     return messages, poems
148 messages = [{"role": "system", "content": system}]
149 for h in history:
150     if h.get("role") in ("user", "assistant") and h.get("content"):
151         messages.append({"role": h["role"], "content": h["content"]})
152 messages.append({"role": "user", "content": question})
153 return messages, None
154 def stream_rag_answer_events(question, history=None):
155     """

```

```

156     首轮: 先 yield {"type": "poems", "poems": [...]}, 再 yield {"type": "chunk", "text": "..."}
157     后续轮: 仅 yield chunk。无检索结果时仅 yield poems 空列表后结束。
158     """
159     messages, poems = _build_rag_messages(question, history)
160     history = history or []
161     if not history:
162         yield {"type": "poems", "poems": poems or []}
163         if not messages:
164             return
165     else:
166         if not messages:
167             return
168     for c in _stream_chat_completion(messages):
169         yield {"type": "chunk", "text": c}
170 def stream_rag_answer(question, history=None):
171     """纯文本流, 供命令行使用"""
172     for ev in stream_rag_answer_events(question, history):
173         if ev.get("type") == "chunk" and ev.get("text"):
174             yield ev["text"]
175 def stream_cognitive_analysis(system_prompt, history=None, followup=None):
176     """
177     认知诗学意象解析:
178     首轮: user 为 followup, 缺省则使用默认解析指令。
179     追问: history 为完整对话; followup 为新一轮用户输入。
180     """
181     history = history or []
182     default_first_user = "请从认知诗学角度深度解析这个意象在诗歌中的多维功能。"
183     if not history:
184         user_msg = (followup or "").strip() or default_first_user
185         messages = [
186             {"role": "system", "content": system_prompt},
187             {"role": "user", "content": user_msg},
188         ]
189     else:
190         messages = [{"role": "system", "content": system_prompt}]
191         for h in history:
192             if h.get("role") in ("user", "assistant") and h.get("content"):
193                 messages.append({"role": h["role"], "content": h["content"]})
194         q = (followup or "").strip() or "请继续从认知诗学角度补充分析。"
195         messages.append({"role": "user", "content": q})
196     yield from _stream_chat_completion(messages, temperature=0.65)
197 def ask(question):
198     """命令行交互入口"""
199     logger.info(f"问答: {question}")
200     print(f"\n 问题: {question}")
201     print("-" * 60)
202     out = []
203     for c in stream_rag_answer(question, None):
204         out.append(c)
205     print("\n".join(out))
206     return "\n".join(out)
207 if __name__ == "__main__":
208     ask("杜甫写过哪些关于秋天的诗? ")

```

-- 文件结束: query_rag.py --

文件: cli.py (240 行, 7.5 KB)

```

1  # -*- coding: utf-8 -*-
2  """
3  CLI 命令行工具 — 系统运维与数据管理
4  """
5  import argparse
6  import json
7  import os
8  import sys
9  import time

```

```

10 from config import BASE_DIR, POEM_JSON_DIR, RAG_DB_DIR, EXPORT_DIR
11 from logger import get_logger
12 logger = get_logger("cli")
13 def cmd_status(args):
14     """显示系统状态"""
15     from config import (
16         EMBED_MODEL, CHAT_MODEL, FLASK_HOST, FLASK_PORT,
17         CATEGORY_NAME_MAP, KNOWN_AUTHORS,
18     )
19     print("=" * 60)
20     print("  唐诗意象智能分析系统 — 状态检查")
21     print("=" * 60)
22     print(f"\n  配置概览:")
23     print(f"    Embedding 模型: {EMBED_MODEL}")
24     print(f"    Chat 模型:      {CHAT_MODEL}")
25     print(f"    服务地址:       http://{FLASK_HOST}:{FLASK_PORT}")
26     print(f"    分类维度:       {len(CATEGORY_NAME_MAP)}")
27     print(f"    已知诗人:       {len(KNOWN_AUTHORS)} 位")
28     print(f"\n  目录状态:")
29     for name, path in [("数据目录", POEM_JSON_DIR), ("向量库", RAG_DB_DIR),
30                        ("导出目录", EXPORT_DIR)]:
31         exists = os.path.exists(path)
32         size = ""
33         if exists:
34             fcount = sum(1 for f in os.listdir(path)
35                           if os.path.isfile(os.path.join(path, f)))
36             size = f"{fcount} 个文件"
37         print(f"    {name}: {'V' if exists else 'X'} {size}")
38     print()
39 def cmd_scan(args):
40     """扫描数据目录"""
41     if not os.path.exists(POEM_JSON_DIR):
42         print(f"数据目录不存在: {POEM_JSON_DIR}")
43         return
44     files = [f for f in os.listdir(POEM_JSON_DIR)
45              if f.endswith((".json", ".txt"))]
46     total_poems = 0
47     total_units = 0
48     print(f"\n扫描数据目录: {POEM_JSON_DIR}")
49     print(f"找到 {len(files)} 个数据文件\n")
50     for fname in sorted(files):
51         fpath = os.path.join(POEM_JSON_DIR, fname)
52         size_kb = os.path.getsize(fpath) / 1024
53         try:
54             with open(fpath, "r", encoding="utf-8") as f:
55                 data = json.load(f)
56                 poems = data.get("诗歌集", [data]) if isinstance(data, dict) else data
57                 pcount = len(poems)
58                 ucount = sum(len(p.get("分析单元", [])) for p in poems)
59                 total_poems += pcount
60                 total_units += ucount
61                 print(f"  {fname:40s} {size_kb:>8.1f} KB {pcount:>3d} 首 {ucount:>4d} 单元")
62         except Exception as e:
63             print(f"  {fname:40s} {size_kb:>8.1f} KB 解析失败: {e}")
64     print(f"\n总计: {total_poems} 首诗歌, {total_units} 个分析单元")
65 def cmd_export(args):
66     """导出数据"""
67     from app import get_data
68     from export_service import ExportService
69     data = get_data()
70     traceback = data.get("traceback", [])
71     fmt = args.format or "csv"
72     if fmt == "csv":
73         path = ExportService.export_traceback_to_csv(traceback)

```

```

74     elif fmt == "json":
75         path = ExportService.export_to_json(data)
76     elif fmt == "report":
77         from analytics import StatsService
78         service = StatsService(traceback)
79         path = ExportService.export_summary_report(service.summary_report())
80     else:
81         print(f"不支持格式: {fmt}")
82         return
83     fsize = os.path.getsize(path) / 1024
84     print(f"导出成功: {path} ({fsize:.1f} KB)")
85 def cmd_clear_cache(args):
86     """清理缓存"""
87     from cache import memory_cache, file_cache
88     memory_cache.clear()
89     file_cache.clear()
90     from app import clear_data_cache
91     clear_data_cache()
92     print("缓存已清理")
93 def cmd_list_exports(args):
94     """列出导出文件"""
95     from export_service import ExportService
96     files = ExportService.list_exports()
97     if not files:
98         print("暂无导出文件")
99         return
100    print(f"\n导出目录: {EXPORT_DIR}")
101    print(f"{'文件名':40s} {'大小':>10s} {'修改时间'}")
102    print("-" * 70)
103    for f in files:
104        size = f["size"] / 1024 if f["size"] > 1024 else f["size"]
105        unit = "KB" if f["size"] > 1024 else "B"
106        mtime = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(f["mtime"]))
107        print(f"    {f['name']:38s} {size:>8.2f} {unit} {mtime}")
108 def cmd_check_rag(args):
109     """检查向量库状态"""
110     import chromadb
111     if not os.path.exists(RAG_DB_DIR):
112         print("向量库不存在, 请先运行 build_rag.py")
113         return
114     client = chromadb.PersistentClient(path=RAG_DB_DIR)
115     try:
116         collection = client.get_collection(name="poems")
117         count = collection.count()
118         print(f"\n向量库状态:")
119         print(f"    路径: {RAG_DB_DIR}")
120         print(f"    集合: poems")
121         print(f"    向量数: {count}")
122         if count > 0:
123             sample = collection.peek()
124             print(f"    示例文档:")
125             for i, doc in enumerate(sample["documents"][:3]):
126                 meta = sample["metadatas"][i]
127                 print(f"        [{i+1}] 《{meta.get('标题', '')}》 {meta.get('作者', '')} ({len(doc)}字)")
128     except Exception as e:
129         print(f"读取向量库失败: {e}")
130 def cmd_build_rag(args):
131     """构建向量库"""
132     from build_rag import main as build_main
133     build_main()
134 def cmd_test(args):
135     """运行单元测试"""
136     import subprocess
137     test_path = os.path.join(BASE_DIR, "tests")

```

```

138 cmd = [sys.executable, "-m", "pytest", test_path, "-v"]
139 if args.verbose:
140     cmd.append("-v")
141 print(f"运行测试: {' '.join(cmd)}")
142 result = subprocess.run(cmd, cwd=BASE_DIR)
143 sys.exit(result.returncode)
144 def main():
145     parser = argparse.ArgumentParser(
146         description="唐诗意象智能分析系统 — CLI 管理工具",
147         formatter_class=argparse.RawDescriptionHelpFormatter,
148         epilog="""
149 使用示例:
150  python cli.py status          查看系统状态
151  python cli.py scan           扫描数据目录
152  python cli.py export --format csv  导出 CSV
153  python cli.py test           运行单元测试
154  python cli.py check-rag      检查向量库
155  python cli.py build-rag      构建向量库
156  """
157     )
158     subparsers = parser.add_subparsers(dest="command", help="可用命令")
159     # status
160     p = subparsers.add_parser("status", help="显示系统状态")
161     p.set_defaults(func=cmd_status)
162     # scan
163     p = subparsers.add_parser("scan", help="扫描数据目录")
164     p.set_defaults(func=cmd_scan)
165     # export
166     p = subparsers.add_parser("export", help="导出数据")
167     p.add_argument("--format", "-f", choices=["csv", "json", "report"],
168                   default="csv", help="导出格式")
169     p.set_defaults(func=cmd_export)
170     # clear-cache
171     p = subparsers.add_parser("clear-cache", help="清理缓存")
172     p.set_defaults(func=cmd_clear_cache)
173     # list-exports
174     p = subparsers.add_parser("list-exports", help="列出导出文件")
175     p.set_defaults(func=cmd_list_exports)
176     # check-rag
177     p = subparsers.add_parser("check-rag", help="检查向量库状态")
178     p.set_defaults(func=cmd_check_rag)
179     # build-rag
180     p = subparsers.add_parser("build-rag", help="构建向量库")
181     p.set_defaults(func=cmd_build_rag)
182     # test
183     p = subparsers.add_parser("test", help="运行单元测试")
184     p.add_argument("--verbose", "-v", action="store_true", help="详细输出")
185     p.set_defaults(func=cmd_test)
186     args = parser.parse_args()
187     if args.command is None:
188         parser.print_help()
189         return
190     try:
191         args.func(args)
192     except KeyboardInterrupt:
193         print("\n操作已取消")
194     except Exception as e:
195         logger.exception("CLI 命令执行失败")
196         print(f"错误: {e}")
197         sys.exit(1)
198 if __name__ == "__main__":
199     main()

```

-- 文件结束: cli.py --

文件: app.py (407 行, 15.9 KB)


```

1  #-*- coding: utf-8 -*-
2  """
3  唐诗意象智能分析系统 — Flask Web 主程序
4  """
5  import json
6  import os
7  import glob
8  import re
9  from flask import Flask, render_template, request, Response
10 from config import (
11     BASE_DIR, POEM_JSON_DIR, FLASK_HOST, FLASK_PORT, FLASK_DEBUG, FLASK_THREADED,
12     SECRET_KEY, CATEGORY_NAME_MAP, PAGE_SIZE, RENDER_LIMIT,
13 )
14 from errors import register_error_handlers, AppError, ValidationError, sse_error
15from validators import validate_question, validate_item, validate_history
16from logger import get_logger, LoggerMixin
17from cache import memory_cache, file_cache, cached
18logger = get_logger("app")
19app = Flask(__name__)
20app.secret_key = SECRET_KEY
21register_error_handlers(app)
22from admin import admin_bp
23app.register_blueprint(admin_bp)
24# -----
25# 数据加载
26# -----
27def clean_and_load_json(file_path):
28    with open(file_path, "r", encoding="utf-8") as f:
29        content = f.read().strip()
30        content = re.sub(r"````json\s*", "", content, flags=re.MULTILINE)
31        content = re.sub(r"````\s*", "", content, flags=re.MULTILINE)
32    try:
33        return json.loads(content)
34    except json.JSONDecodeError:
35        return None
36def extract_poems(node):
37    found_poems = []
38    if isinstance(node, dict):
39        if "分析单元" in node and isinstance(node.get("分析单元"), list):
40            found_poems.append(node)
41        else:
42            for value in node.values():
43                found_poems.extend(extract_poems(value))
44    elif isinstance(node, list):
45        for item in node:
46            found_poems.extend(extract_poems(item))
47    return found_poems
48def build_traceback_dataset():
49    """从 ./poem_json/ 扫描 JSON/TXT，生成溯源表与统计"""
50    json_folder = POEM_JSON_DIR
51    if not os.path.exists(json_folder):
52        os.makedirs(json_folder, exist_ok=True)
53    target_files = []
54    for ext in ("*.json", "*.txt"):
55        target_files.extend(glob.glob(os.path.join(json_folder, ext)))
56    logger.info(f"扫描到 {len(target_files)} 个数据文件")
57    traceback_data_list = []
58    total_poems = 0
59    seen_poem_fingerprints = set()
60    skip_names = {
61        "all_data.json", "dashboard_interactive_pro.html",
62        "dashboard_FINAL_DASHBOARD.html", "Thesis_Dashboard.html",
63    }
64    for file_path in target_files:

```

```

65     file_name = os.path.basename(file_path)
66     if file_name in skip_names:
67         continue
68     data = clean_and_load_json(file_path)
69     if not data:
70         continue
71     poems = extract_poems(data)
72     if not poems:
73         continue
74     for p in poems:
75         if not isinstance(p, dict):
76             continue
77         title = str(p.get("标题", "未知诗歌")).strip()
78         poem_id = str(p.get("诗歌编号", p.get("编号", "-"))).strip()
79         lines_data = p.get("诗行", [])
80         first_line = ""
81         if lines_data and isinstance(lines_data[0], dict):
82             first_line = str(lines_data[0].get("原文", "")).strip()
83         fingerprint = f"{title}_{first_line}"
84         if fingerprint in seen_poem_fingerprints:
85             continue
86         seen_poem_fingerprints.add(fingerprint)
87         total_poems += 1
88         genre = str(p.get("分类标签", p.get("体裁", "未分类"))).strip()
89         line_map = {}
90         for l in lines_data:
91             if isinstance(l, dict):
92                 line_map[str(l.get("诗行编号", ""))] = l.get("原文", "")
93         for u in p.get("分析单元", []):
94             if not isinstance(u, dict):
95                 continue
96             if str(u.get("是否意象", "0")).strip() == "1":
97                 text = str(u.get("文本", "")).strip()
98                 sub_code = str(u.get("子类编码", "")).strip()
99                 line_id = str(u.get("诗行编号", "")).strip()
100                 line_text = line_map.get(line_id, "未知诗句")
101                 cat_name = CATEGORY_NAME_MAP.get(sub_code, f"其他分类 ({sub_code})")
102                 if text:
103                     emo_cat = str(u.get("情感类别", "未知")).strip()
104                     emo_pol = str(u.get("情感极性", "")).strip()
105                     emotion_str = f"{emo_cat} ({emo_pol})" if emo_pol else emo_cat
106                     dim_parts = []
107                     for key in ["感知通道", "素材类型", "指涉来源", "表现功能"]:
108                         val = str(u.get(key, "")).strip()
109                         if val and val != "None":
110                             dim_parts.append(val)
111                     dimensions_str = " | ".join(dim_parts) if dim_parts else "-"
112                     major_code = u.get("大类编码", "")
113                     if major_code == "" or major_code is None:
114                         major_code = (
115                             sub_code.split("-")[0]
116                             if sub_code and "-" in sub_code
117                             else ""
118                         )
119                 traceback_data_list.append({
120                     "poem_id": poem_id,
121                     "title": title,
122                     "author": p.get("作者", ""),
123                     "genre": genre,
124                     "cat": cat_name,
125                     "txt": text,
126                     "文本": text,
127                     "dimensions": dimensions_str,
128                     "emotion": emotion_str,

```

```

129         "emo_cat": emo_cat,
130         "line": line_text,
131         "id": line_id,
132         "词性": u.get("词性", ""),
133         "成分类型": u.get("成分类型", ""),
134         "感知通道": u.get("感知通道", ""),
135         "素材类型": u.get("素材类型", ""),
136         "内部结构": u.get("内部结构", ""),
137         "指涉来源": u.get("指涉来源", ""),
138         "表现功能": u.get("表现功能", ""),
139         "文化流通性": u.get("文化流通性", ""),
140         "跨文化性": u.get("跨文化性", ""),
141         "认知强度": u.get("认知强度", ""),
142         "核心意象": u.get("核心意象", ""),
143         "结构功能组": u.get("结构功能组", ""),
144         "情感极性": u.get("情感极性", ""),
145         "情感类别": u.get("情感类别", emo_cat),
146         "情感置信度": u.get("情感置信度", ""),
147         "大类编码": major_code,
148         "子类编码": sub_code,
149     })
150     logger.info(f"数据加载完成: {total_poems} 首诗歌, {len(traceback_data_list)} 条意象")
151     return {
152         "total_poems": total_poems,
153         "total_images": len(traceback_data_list),
154         "traceback": traceback_data_list,
155     }
156 _DATA_CACHE = None
157 def get_data():
158     global _DATA_CACHE
159     if _DATA_CACHE is None:
160         _DATA_CACHE = build_traceback_dataset()
161     return _DATA_CACHE
162 def clear_data_cache():
163     global _DATA_CACHE
164     _DATA_CACHE = None
165     logger.info("数据缓存已清除")
166 # -----
167 # SSE 工具函数
168 # -----
169 def sse_json(obj):
170     return f"data: {json.dumps(obj, ensure_ascii=False)}\n\n"
171 def build_analyze_system_prompt(item):
172     """认知诗学解析: 严格基于 JSON 结构化字段"""
173     return f"""你是专精认知诗学与中国古典文学的学者。
174 当前意象数据:
175 - 意象文本: {item.get('文本', '')}
176 - 所属诗歌: {item.get('title', '')}
177 - 所在诗句: {item.get('line', '')}
178 - 词性: {item.get('词性', '')} | 成分类型: {item.get('成分类型', '')}
179 - 感知通道: {item.get('感知通道', '')} | 素材类型: {item.get('素材类型', '')}
180 - 内部结构: {item.get('内部结构', '')} | 指涉来源: {item.get('指涉来源', '')}
181 - 表现功能: {item.get('表现功能', '')} | 结构功能组: {item.get('结构功能组', '')}
182 - 文化流通性: {item.get('文化流通性', '')} | 跨文化性: {item.get('跨文化性', '')}
183 - 认知强度: {item.get('认知强度', '')} | 核心意象: {item.get('核心意象', '')}
184 - 情感极性: {item.get('情感极性', '')} | 情感类别: {item.get('情感类别', '')} | 置信度: {item.get('情感置信度', '')}
185 请按以下维度解析:
186 1. 感知层: 触发的感官体验和通感效果
187 2. 文化层: 典故来源和文化符号意义
188 3. 情感层: 基于情感极性{item.get('情感极性', '')}分析情感功能
189 4. 结构层: 作为{item.get('结构功能组', '')}在诗中的结构作用
190 5. 跨诗比较: 此意象在其他诗歌中的异同
191 严格基于提供的数据分析, 不编造。"""
192 # -----

```

```
193 # 页面路由
194 # -----
195 @app.route("/")
196 def index():
197     return render_template("index.html")
198 # -----
199 # 数据 API
200 # -----
201 @app.route("/api/data")
202 def api_data():
203     return Response(
204         json.dumps(get_data(), ensure_ascii=False),
205         mimetype="application/json; charset=utf-8",
206     )
207 @app.route("/api/stats")
208 def api_stats_summary():
209     """统计概览 API"""
210     from analytics import StatsService, build_analytics_api
211     data = get_data()
212     traceback = data.get("traceback", [])
213     service = StatsService(traceback)
214     return Response(
215         json.dumps(build_analytics_api(service), ensure_ascii=False),
216         mimetype="application/json; charset=utf-8",
217     )
218 @app.route("/api/export", methods=["POST"])
219 def api_export():
220     """数据导出 API"""
221     from export_service import ExportService
222     payload = request.get_json(force=True, silent=True) or {}
223     fmt = payload.get("format", "csv")
224     data = get_data()
225     traceback = data.get("traceback", [])
226     try:
227         if fmt == "csv":
228             path = ExportService.export_traceback_to_csv(traceback)
229         elif fmt == "json":
230             path = ExportService.export_to_json(data)
231         else:
232             return Response(
233                 json.dumps({"error": f"不支持格式: {fmt}"}, ensure_ascii=False),
234                 mimetype="application/json; charset=utf-8",
235                 status=400,
236             )
237         return Response(
238             json.dumps({"path": path, "file": os.path.basename(path)}, ensure_ascii=False),
239             mimetype="application/json; charset=utf-8",
240         )
241     except Exception as e:
242         logger.exception("导出失败")
243         return Response(
244             json.dumps({"error": str(e)}, ensure_ascii=False),
245             mimetype="application/json; charset=utf-8",
246             status=500,
247         )
248 # -----
249 # AI 问答 API (RAG)
250 # -----
251 @app.route("/api/ask", methods=["POST"])
252 def api_ask():
253     payload = request.get_json(force=True, silent=True) or {}
254     question = (payload.get("question") or "").strip()
255     history = validate_history(payload.get("history"))
256     try:
```

```

257         question = validate_question(question)
258     except ValidationError as e:
259         def empty():
260             yield sse_error(e.message)
261         return Response(empty(), mimetype="text/event-stream")
262 from query_rag import stream_rag_answer_events
263 def generate():
264     try:
265         for ev in stream_rag_answer_events(question, history):
266             if ev.get("type") == "poems":
267                 yield sse_json({"type": "poems", "poems": ev.get("poems") or []})
268             if not history and not (ev.get("poems") or []):
269                 yield sse_json({
270                     "type": "error",
271                     "message": "未检索到相关诗歌, 请调整提问或检查向量库。",
272                 })
273             return
274             elif ev.get("type") == "chunk" and ev.get("text"):
275                 yield sse_json({"type": "chunk", "text": ev["text"]})
276     except Exception as e:
277         logger.exception("RAG 问答异常")
278         yield sse_json({"type": "error", "message": str(e)})
279 return Response(
280     generate(),
281     mimetype="text/event-stream",
282     headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
283 )
284 # -----
285 # 认知诗学解析 API
286 # -----
287 @app.route("/api/analyze", methods=["POST"])
288 def api_analyze():
289     payload = request.get_json(force=True, silent=True) or {}
290     item = payload.get("item") or {}
291     history = validate_history(payload.get("history"))
292     followup = payload.get("followup")
293     try:
294         item = validate_item(item)
295     except ValidationError as e:
296         def err():
297             yield sse_error(e.message)
298         return Response(err(), mimetype="text/event-stream")
299     system_prompt = build_analyze_system_prompt(item)
300     from query_rag import stream_cognitive_analysis
301     def generate():
302         try:
303             for chunk in stream_cognitive_analysis(
304                 system_prompt,
305                 history=history,
306                 followup=followup,
307             ):
308                 if chunk:
309                     yield sse_json({"type": "chunk", "text": chunk})
310         except Exception as e:
311             logger.exception("认知解析异常")
312             yield sse_json({"type": "error", "message": str(e)})
313     return Response(
314         generate(),
315         mimetype="text/event-stream",
316         headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
317     )
318 # -----
319 # 缓存管理 API
320 # -----

```

```

321 @app.route("/api/cache/status")
322 def api_cache_status():
323     return Response(
324         json.dumps({
325             "memory_entries": memory_cache.size(),
326             "file_entries": file_cache.size(),
327             "data_cache_active": _DATA_CACHE is not None,
328         }),
329         mimetype="application/json; charset=utf-8",
330     )
331 # -----
332 # 启动入口
333 # -----
334 if __name__ == "__main__":
335     logger.info(f"启动唐诗意象智能分析系统 (http://{FLASK_HOST}:{FLASK_PORT})")
336     app.run(host=FLASK_HOST, port=FLASK_PORT, debug=FLASK_DEBUG, threaded=FLASK_THREADED)
-- 文件结束: app.py --
文件: admin.py (176 行, 5.1 KB)
1 # -*- coding: utf-8 -*-
2 """
3 管理后台 Blueprint — 系统状态、数据管理、导出操作
4 """
5 import json
6 import os
7 import time
8 from flask import Blueprint, render_template, jsonify, request, Response
9 from config import (
10     BASE_DIR, RAG_DB_DIR, POEM_JSON_DIR, EXPORT_DIR,
11     EMBED_MODEL, CHAT_MODEL, FLASK_PORT,
12     CATEGORY_NAME_MAP, KNOWN_AUTHORS, PAGE_SIZE, TOP_K,
13 )
14 from analytics import StatsService
15 from cache import memory_cache, file_cache
16 from export_service import ExportService
17 from logger import get_logger
18 logger = get_logger("admin")
19 admin_bp = Blueprint("admin", __name__, url_prefix="/admin")
20 def _get_data_provider():
21     """延迟导入以避免循环引用"""
22     from app import get_data
23     return get_data()
24 @admin_bp.route("/")
25 def admin_index():
26     return render_template("admin.html")
27 @admin_bp.route("/api/status")
28 def api_status():
29     """系统状态总览"""
30     data = _get_data_provider()
31     total_images = len(data.get("traceback", []))
32     total_poems = data.get("total_poems", 0)
33     rag_exists = os.path.exists(RAG_DB_DIR) and os.path.isdir(RAG_DB_DIR)
34     rag_file_count = 0
35     if rag_exists:
36         for root, dirs, files in os.walk(RAG_DB_DIR):
37             rag_file_count += len(files)
38     chunk_count = 0
39     if os.path.exists(POEM_JSON_DIR):
40         chunk_count = len([f for f in os.listdir(POEM_JSON_DIR)
41                             if f.endswith((".json", ".txt"))])
42     cache_info = {
43         "memory_entries": memory_cache.size(),
44         "file_entries": file_cache.size(),
45     }
46     export_files = ExportService.list_exports()

```

```

47     return jsonify({
48         "status": "running",
49         "timestamp": time.time(),
50         "data": {
51             "total_poems": total_poems,
52             "total_images": total_images,
53             "category_count": len(CATEGORY_NAME_MAP),
54             "known_authors_count": len(KNOWN_AUTHORS),
55         },
56         "rag_database": {
57             "exists": rag_exists,
58             "file_count": rag_file_count,
59             "path": RAG_DB_DIR,
60         },
61         "poem_json": {
62             "file_count": chunk_count,
63             "path": POEM_JSON_DIR,
64         },
65         "cache": cache_info,
66         "exports": {
67             "file_count": len(export_files),
68             "directory": EXPORT_DIR,
69         },
70         "config": {
71             "embed_model": EMBED_MODEL,
72             "chat_model": CHAT_MODEL,
73             "port": FLASK_PORT,
74             "top_k": TOP_K,
75             "page_size": PAGE_SIZE,
76         },
77     })
78 @admin_bp.route("/api/stats")
79 def api_stats():
80     """统计分析数据"""
81     data = _get_data_provider()
82     traceback = data.get("traceback", [])
83     service = StatsService(traceback)
84     report = service.summary_report()
85     report["cross_analysis"] = service.cross_analysis()
86     report["author_stats"] = service.author_statistics()
87     report["perception_stats"] = service.perception_channel_distribution()
88     return jsonify(report)
89 @admin_bp.route("/api/exports", methods=["GET"])
90 def api_list_exports():
91     """列出导出文件"""
92     return jsonify({"exports": ExportService.list_exports()})
93 @admin_bp.route("/api/exports", methods=["POST"])
94 def api_create_export():
95     """创建导出"""
96     payload = request.get_json(force=True, silent=True) or {}
97     fmt = payload.get("format", "csv")
98     data = _get_data_provider()
99     traceback = data.get("traceback", [])
100     try:
101         if fmt == "csv":
102             path = ExportService.export_traceback_to_csv(traceback)
103         elif fmt == "json":
104             path = ExportService.export_to_json(data)
105         elif fmt == "report":
106             service = StatsService(traceback)
107             path = ExportService.export_summary_report(service.summary_report())
108         else:
109             return jsonify({"error": f"不支持的导出格式: {fmt}"}) , 400
110         fname = os.path.basename(path)

```

```

111         fsize = os.path.getsize(path)
112         return jsonify({"message": "导出成功", "file": fname, "size": fsize, "path": path})
113     except Exception as e:
114         logger.exception("导出失败")
115         return jsonify({"error": str(e)}), 500
116 @admin_bp.route("/api/exports/clear", methods=["POST"])
117 def api_clear_exports():
118     """清空导出目录"""
119     count = ExportService.clear_exports()
120     return jsonify({"message": f"已移除 {count} 个导出文件", "count": count})
121 @admin_bp.route("/api/cache/clear", methods=["POST"])
122 def api_clear_cache():
123     """清理缓存"""
124     cache_type = (request.get_json(force=True, silent=True) or {}).get("type", "all")
125     if cache_type in ("memory", "all"):
126         memory_cache.clear()
127     if cache_type in ("file", "all"):
128         file_cache.clear()
129     from app import clear_data_cache
130     clear_data_cache()
131     logger.info(f"缓存已清理 (type={cache_type})")
132     return jsonify({"message": "缓存已清理", "type": cache_type})
133 @admin_bp.route("/api/refresh", methods=["POST"])
134 def api_refresh_data():
135     """刷新数据缓存"""
136     from app import clear_data_cache, get_data
137     clear_data_cache()
138     data = get_data()
139     return jsonify({
140         "message": "数据已刷新",
141         "total_poems": data.get("total_poems", 0),
142         "total_images": len(data.get("traceback", [])),
143     })

```

-- 文件结束: admin.py --

文件: templates/index.html (1547 行, 55.1 KB)

```

1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>唐诗意象智能分析系统</title>
7      <script src="https://cdn.jsdelivr.net/npm/echarts@5.5.0/dist/echarts.min.js"></script>
8      <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
9      <style>
10         :root {
11             --primary: #2c3e50;
12             --accent: #3498db;
13             --bg: #f4f7f6;
14             --text: #333;
15             --card: #ffffff;
16             --success: #2ecc71;
17             --warn: #e67e22;
18         }
19         * { box-sizing: border-box; }
20         body {
21             font-family: 'Helvetica Neue', Arial, sans-serif;
22             background-color: var(--bg);
23             margin: 0;
24             padding: 0;
25             color: var(--text);
26             padding-top: 64px;
27         }
28         .navbar {
29             position: fixed;

```



```
30     top: 0;
31     left: 0;
32     right: 0;
33     height: 64px;
34     background: var(--card);
35     border-bottom: 1px solid #e0e0e0;
36     box-shadow: 0 2px 10px rgba(0,0,0,0.04);
37     display: flex;
38     align-items: center;
39     justify-content: space-between;
40     padding: 0 24px;
41     z-index: 900;
42 }
43 .brand {
44     display: flex;
45     align-items: center;
46     gap: 12px;
47     font-size: 18px;
48     font-weight: 700;
49     color: var(--primary);
50 }
51 .logo {
52     width: 36px;
53     height: 36px;
54     border-radius: 8px;
55     background: linear-gradient(135deg, var(--accent), #2980b9);
56     display: flex;
57     align-items: center;
58     justify-content: center;
59     color: #fff;
60     font-size: 14px;
61     font-weight: 800;
62 }
63 .tabs {
64     display: flex;
65     gap: 8px;
66 }
67 .tab-btn {
68     border: 1px solid #dce4ec;
69     background: #fff;
70     color: var(--primary);
71     padding: 10px 22px;
72     border-radius: 8px;
73     cursor: pointer;
74     font-weight: 600;
75     transition: background 0.25s, color 0.25s, border-color 0.25s, transform 0.15s;
76 }
77 .tab-btn:hover { border-color: var(--accent); color: var(--accent); transform: translateY(-1px); }
78 .tab-btn.active {
79     background: var(--accent);
80     color: #fff;
81     border-color: var(--accent);
82 }
83 .tab-pane {
84     display: none;
85     opacity: 0;
86     transition: opacity 0.35s ease;
87     max-width: 1600px;
88     margin: 24px auto;
89     padding: 0 20px 40px;
90 }
91 .tab-pane.visible {
92     display: block;
93     opacity: 1;
```

```

94     }
95     .stats-bar-wrap {
96         text-align: center;
97         margin-bottom: 24px;
98     }
99     .stats-bar {
100         display: inline-flex;
101         flex-wrap: wrap;
102         gap: 24px;
103         justify-content: center;
104         background: #f8f9fa;
105         padding: 14px 28px;
106         border-radius: 50px;
107         border: 1px solid #eee;
108     }
109     .stat-item { font-size: 15px; font-weight: bold; color: #555; }
110     .stat-num { color: var(--accent); font-size: 19px; margin-left: 6px; }
111     .card {
112         background: var(--card);
113         border-radius: 10px;
114         padding: 24px;
115         margin-bottom: 24px;
116         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.04);
117         transition: 0.25s;
118         position: relative;
119     }
120     .card-header {
121         display: flex;
122         justify-content: space-between;
123         align-items: center;
124         margin-bottom: 16px;
125         border-bottom: 2px solid #f0f0f0;
126         padding-bottom: 12px;
127         flex-wrap: wrap;
128         gap: 12px;
129     }
130     .card-title {
131         margin: 0;
132         font-size: 19px;
133         color: var(--primary);
134         border-left: 5px solid var(--accent);
135         padding-left: 12px;
136     }
137     .btn-action {
138         background: #ecf0f1;
139         color: var(--primary);
140         border: none;
141         padding: 8px 16px;
142         border-radius: 8px;
143         cursor: pointer;
144         font-weight: bold;
145         transition: 0.2s;
146         display: inline-flex;
147         align-items: center;
148         gap: 6px;
149     }
150     .btn-action:hover { background: var(--accent); color: #fff; }
151     .grid-2 {
152         display: grid;
153         grid-template-columns: 1fr 1fr;
154         gap: 24px;
155     }
156     @media (max-width: 960px) {
157         .grid-2 { grid-template-columns: 1fr; }

```

```
158     .navbar { flex-wrap: wrap; height: auto; padding: 12px; gap: 12px; }
159     body { padding-top: 100px; }
160 }
161 .chart-container { height: 400px; width: 100%; }
162 .filter-bar {
163     display: flex;
164     gap: 12px;
165     margin-bottom: 16px;
166     flex-wrap: wrap;
167     background: #f8f9fa;
168     padding: 14px;
169     border-radius: 8px;
170 }
171 .filter-input, .filter-select {
172     padding: 10px 14px;
173     border: 1px solid #ddd;
174     border-radius: 8px;
175     font-size: 14px;
176     outline: none;
177     transition: 0.25s;
178 }
179 .filter-input:focus, .filter-select:focus {
180     border-color: var(--accent);
181     box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
182 }
183 .filter-input { flex: 1; min-width: 200px; }
184 .table-wrapper {
185     max-height: 520px;
186     overflow: auto;
187     border: 1px solid #eee;
188     border-radius: 8px;
189 }
190 table {
191     width: 100%;
192     border-collapse: collapse;
193     font-size: 14px;
194     table-layout: fixed;
195 }
196 th, td {
197     border: 1px solid #eee;
198     padding: 10px 12px;
199     text-align: left;
200     vertical-align: middle;
201     word-wrap: break-word;
202     line-height: 1.5;
203 }
204 th {
205     background: #fafafa;
206     position: sticky;
207     top: 0;
208     z-index: 5;
209     font-weight: 600;
210 }
211 tbody tr.data-row {
212     transition: background 0.2s;
213     cursor: default;
214 }
215 tbody tr.data-row:hover { background-color: #eaf4fc; }
216 .tag {
217     background: #ecf0f1;
218     padding: 3px 8px;
219     border-radius: 4px;
220     font-size: 12px;
221     color: #555;
```

```
222     display: inline-block;
223 }
224 .recycle-btn {
225     background: var(--warn);
226     color: #fff;
227     padding: 10px 18px;
228     border-radius: 30px;
229     border: none;
230     font-weight: bold;
231     cursor: pointer;
232     transition: 0.25s;
233     box-shadow: 0 4px 10px rgba(230, 126, 34, 0.25);
234 }
235 .recycle-btn:hover { filter: brightness(1.05); transform: translateY(-2px); }
236 .pager {
237     display: flex;
238     align-items: center;
239     gap: 12px;
240     justify-content: flex-end;
241     margin-top: 12px;
242     flex-wrap: wrap;
243 }
244 .pager button {
245     padding: 8px 14px;
246     border-radius: 8px;
247     border: 1px solid #dce4ec;
248     background: #fff;
249     cursor: pointer;
250     transition: 0.2s;
251 }
252 .pager button:hover:not(:disabled) { border-color: var(--accent); color: var(--accent); }
253 .pager button:disabled { opacity: 0.45; cursor: not-allowed; }
254 .modal-overlay {
255     display: none;
256     position: fixed;
257     inset: 0;
258     background: rgba(0, 0, 0, 0.55);
259     z-index: 950;
260     backdrop-filter: blur(3px);
261 }
262 .modal-content {
263     position: absolute;
264     top: 50%;
265     left: 50%;
266     transform: translate(-50%, -50%);
267     background: #fff;
268     width: min(1000px, 92vw);
269     max-height: 82vh;
270     border-radius: 12px;
271     padding: 28px;
272     box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
273     display: flex;
274     flex-direction: column;
275 }
276 .close-btn {
277     position: absolute;
278     top: 16px;
279     right: 16px;
280     background: none;
281     border: none;
282     font-size: 22px;
283     cursor: pointer;
284     color: #999;
285 }
```

```
286 .close-btn:hover { color: #e74c3c; }
287 /* 意象解析 — 右侧抽屉 */
288 .drawer-overlay {
289     position: fixed;
290     inset: 0;
291     background: rgba(0, 0, 0, 0.45);
292     z-index: 1000;
293     opacity: 0;
294     visibility: hidden;
295     transition: opacity 0.3s ease, visibility 0.3s ease;
296 }
297 .drawer-overlay.show {
298     opacity: 1;
299     visibility: visible;
300 }
301 .analyze-drawer {
302     position: fixed;
303     top: 0;
304     right: 0;
305     width: 420px;
306     max-width: 100vw;
307     height: 100vh;
308     background: #fff;
309     z-index: 1001;
310     box-shadow: -4px 0 20px rgba(0, 0, 0, 0.1);
311     transform: translateX(100%);
312     transition: transform 0.3s ease;
313     display: flex;
314     flex-direction: column;
315 }
316 .analyze-drawer.open {
317     transform: translateX(0);
318 }
319 /* 抽屉拖拽边框 */
320 .drawer-resizer {
321     position: absolute;
322     left: 0;
323     top: 0;
324     bottom: 0;
325     width: 6px;
326     cursor: ew-resize;
327     z-index: 1005;
328     background: transparent;
329     transition: background 0.2s;
330 }
331 .drawer-resizer:hover, .drawer-resizer.active {
332     background: rgba(52, 152, 219, 0.4);
333 }
334 .analyze-drawer.no-transition {
335     transition: none !important;
336 }
337 .drawer-head {
338     flex-shrink: 0;
339     padding: 16px 44px 14px 18px;
340     border-bottom: 1px solid #eee;
341     position: relative;
342 }
343 .drawer-close {
344     position: absolute;
345     top: 12px;
346     right: 12px;
347     width: 36px;
348     height: 36px;
349     border: none;
```

```
350     background: transparent;
351     font-size: 22px;
352     line-height: 1;
353     cursor: pointer;
354     color: #999;
355     border-radius: 8px;
356 }
357 .drawer-close:hover { color: #e74c3c; background: #f5f5f5; }
358 .drawer-title-row {
359     font-size: 16px;
360     font-weight: 700;
361     color: var(--primary);
362     margin-bottom: 8px;
363 }
364 .drawer-title-row .hl { color: var(--accent); }
365 .drawer-meta {
366     font-size: 13px;
367     color: #666;
368     margin-bottom: 6px;
369 }
370 .drawer-line {
371     font-family: 楷体, KaiTi, serif;
372     font-size: 15px;
373     line-height: 1.55;
374     color: #333;
375 }
376 .drawer-tags {
377     flex-shrink: 0;
378     padding: 10px 14px;
379     border-bottom: 1px solid #eee;
380     background: #fafbfc;
381     max-height: 120px;
382     overflow-y: auto;
383 }
384 .tag-chip {
385     display: inline-block;
386     padding: 3px 10px;
387     border-radius: 20px;
388     background: #eaf4fd;
389     color: #3498db;
390     font-size: 12px;
391     margin: 3px;
392 }
393 .drawer-chat-scroll {
394     flex: 1;
395     overflow-y: auto;
396     overflow-x: hidden;
397     padding: 14px 16px;
398     min-height: 0;
399 }
400 .msg-row {
401     display: flex;
402     margin-bottom: 12px;
403 }
404 .msg-row.user { justify-content: flex-end; }
405 .msg-row.assistant { justify-content: flex-start; }
406 .msg-bubble {
407     max-width: 92%;
408     padding: 10px 12px;
409     border-radius: 14px;
410     font-size: 14px;
411     line-height: 1.65;
412     word-break: break-word;
413 }
```

```

414 .msg-bubble.user {
415     background: #3498db;
416     color: #fff;
417     border-bottom-right-radius: 4px;
418 }
419 .msg-bubble.assistant {
420     background: #ecf0f1;
421     color: #333;
422     border-bottom-left-radius: 4px;
423 }
424 .drawer-md { font-size: 14px; line-height: 1.65; }
425 .drawer-md h1, .drawer-md h2, .drawer-md h3 { margin: 0.6em 0 0.35em; font-size: 1.05em; }
426 .drawer-md p { margin: 0.45em 0; }
427 .drawer-md ul, .drawer-md ol { margin: 0.4em 0; padding-left: 1.2em; }
428 .drawer-md code { background: rgba(0,0,0,0.06); padding: 2px 5px; border-radius: 4px; font-size: 0.9em; }
429 .drawer-foot {
430     flex-shrink: 0;
431     padding: 12px 14px;
432     border-top: 1px solid #eee;
433     background: #fafafa;
434     display: flex;
435     gap: 10px;
436     align-items: center;
437 }
438 .drawer-foot input {
439     flex: 1;
440     padding: 11px 12px;
441     border-radius: 8px;
442     border: 1px solid #ddd;
443     outline: none;
444     font-size: 14px;
445 }
446 .drawer-foot input:focus {
447     border-color: var(--accent);
448     box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
449 }
450 .drawer-foot button {
451     padding: 11px 18px;
452     border-radius: 8px;
453     border: none;
454     background: var(--accent);
455     color: #fff;
456     font-weight: 600;
457     cursor: pointer;
458     white-space: nowrap;
459 }
460 .drawer-foot button:hover { filter: brightness(1.05); }
461 /* AI tab */
462 .ai-hero {
463     max-width: 720px;
464     margin: 0 auto 28px;
465     text-align: center;
466 }
467 .ai-hero input {
468     width: 100%;
469     padding: 16px 20px;
470     font-size: 17px;
471     border-radius: 12px;
472     border: 1px solid #ddd;
473     outline: none;
474     transition: 0.25s;
475 }
476 .ai-hero input:focus {
477     border-color: var(--accent);

```

```

478     box-shadow: 0 0 12px rgba(52, 152, 219, 0.35);
479 }
480 .ai-hero button {
481     margin-top: 14px;
482     padding: 12px 32px;
483     border-radius: 10px;
484     border: none;
485     background: var(--accent);
486     color: #fff;
487     font-size: 16px;
488     font-weight: 700;
489     cursor: pointer;
490     transition: 0.2s;
491 }
492 .ai-hero button:hover { filter: brightness(1.05); transform: translateY(-1px); }
493 .ai-split {
494     display: none;
495     grid-template-columns: 40% 60%;
496     gap: 20px;
497     align-items: stretch;
498     min-height: 60vh;
499 }
500 .ai-split.active { display: grid; }
501 @media (max-width: 900px) {
502     .ai-split.active { grid-template-columns: 1fr; }
503 }
504 .poem-col {
505     overflow: auto;
506     max-height: calc(100vh - 220px);
507 }
508 .poem-card {
509     background: var(--card);
510     border-radius: 10px;
511     padding: 14px 16px;
512     margin-bottom: 12px;
513     box-shadow: 0 4px 20px rgba(0, 0, 0, 0.04);
514     cursor: pointer;
515     transition: transform 0.2s, box-shadow 0.2s;
516     border: 1px solid #f0f0f0;
517 }
518 .poem-card:hover {
519     transform: translateY(-3px);
520     box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
521 }
522 .poem-card h4 { margin: 0 0 6px; color: var(--primary); font-size: 15px; }
523 .poem-card .sub { font-size: 12px; color: #888; margin-bottom: 8px; }
524 .poem-card .body {
525     font-family: 楷体, KaiTi, serif;
526     font-size: 15px;
527     line-height: 1.65;
528     color: #333;
529 }
530 .poem-card .sim {
531     margin-top: 8px;
532     font-size: 12px;
533     color: var(--success);
534     font-weight: 700;
535 }
536 .poem-detail { display: none; margin-top: 10px; font-size: 13px; color: #555; white-space: pre-wrap; }
537 .poem-card.expanded .poem-detail { display: block; }
538 .ai-answer {
539     background: var(--card);
540     border-radius: 10px;
541     padding: 18px;

```



```

542     box-shadow: 0 4px 20px rgba(0, 0, 0, 0.04);
543     display: flex;
544     flex-direction: column;
545     min-height: 420px;
546     border: 1px solid #f0f0f0;
547 }
548 .ai-answer-scroll {
549     flex: 1;
550     overflow: auto;
551     padding-bottom: 12px;
552     white-space: pre-wrap;
553     line-height: 1.75;
554     font-size: 15px;
555 }
556 .ai-foot {
557     margin-top: 10px;
558     border-top: 1px solid #eee;
559     padding-top: 12px;
560     display: flex;
561     gap: 10px;
562 }
563 .ai-foot input {
564     flex: 1;
565     padding: 12px 14px;
566     border-radius: 8px;
567     border: 1px solid #ddd;
568     outline: none;
569 }
570 .ai-foot input:focus {
571     border-color: var(--accent);
572     box-shadow: 0 0 8px rgba(52, 152, 219, 0.35);
573 }
574 .ai-foot button {
575     padding: 12px 18px;
576     border-radius: 8px;
577     border: none;
578     background: var(--primary);
579     color: #fff;
580     font-weight: 600;
581     cursor: pointer;
582 }
583 .warn-text { color: var(--warn); font-size: 13px; margin-top: 8px; }
584 .link-action { color: var(--accent); cursor: pointer; font-weight: 600; }
585 .danger { color: #e74c3c; }
586 .ops-cell { white-space: nowrap; }
587 .ops-cell .btn-link {
588     background: none;
589     border: none;
590     padding: 0 8px;
591     cursor: pointer;
592     font-size: 14px;
593     color: var(--accent);
594     font-weight: 600;
595 }
596 .ops-cell .btn-link:hover { text-decoration: underline; }
597 </style>
598 </head>
599 <body>
600 <nav class="navbar">
601     <div class="brand">
602         <div class="logo">唐</div>
603         <span>唐诗意象智能分析系统</span>
604     </div>
605     <div class="tabs">

```

```
606     <button type="button" class="tab-btn active" data-tab="graph">数据图谱</button>
607     <button type="button" class="tab-btn" data-tab="ai">AI问答</button>
608 </div>
609 </nav>
610 <section id="tab-graph" class="tab-pane visible">
611     <div class="stats-bar-wrap">
612         <div class="stats-bar">
613             <div class="stat-item">净解析诗歌: <span id="stat-poems" class="stat-num">0</span> 首</div>
614             <div class="stat-item">提取真意象: <span id="stat-images" class="stat-num">0</span> 条</div>
615             <div class="stat-item">分类维度: <span class="stat-num">4 层 10 维</span></div>
616         </div>
617         <div style="margin-top:16px;">
618             <button type="button" class="recycle-btn" onclick="openRecycle()">回收站 (<span id="recycle-count">0</span>)</button>
619         </div>
620     </div>
621     <div class="grid-2">
622         <div class="card" id="card-ind">
623             <div class="card-header">
624                 <h2 class="card-title">核心意象 Top50</h2>
625                 <button type="button" class="btn-action" onclick="toggleFullScreen('card-ind')">放大视图</button>
626             </div>
627             <div id="chart-ind" class="chart-container"></div>
628         </div>
629         <div class="card" id="card-cat">
630             <div class="card-header">
631                 <h2 class="card-title">意象分类域分布</h2>
632                 <button type="button" class="btn-action" onclick="toggleFullScreen('card-cat')">放大视图</button>
633             </div>
634             <div id="chart-cat" class="chart-container"></div>
635         </div>
636     </div>
637     <div class="card">
638         <div class="card-header">
639             <h2 class="card-title">意象溯源数据表 <span id="record-count" style="font-size:13px;color:#999;font-weight:400;"></span></h2>
640             <button type="button" class="btn-action" style="background:#e74c3c;color:#fff;display:none;" id="resetBtn" onclick="resetFilters()">清除
641         </div>
642         <div class="filter-bar">
643             <input type="text" id="searchTxt" class="filter-input" placeholder="搜索意象文本（如：山、月）..." onkeyup="applyFilters()">
644             <select id="filterCat" class="filter-select" onchange="applyFilters()">
645                 <option value="">所有分类域</option>
646             </select>
647             <select id="filterGenre" class="filter-select" onchange="applyFilters()">
648                 <option value="">所有体裁/标签</option>
649             </select>
650             <select id="filterEmotion" class="filter-select" onchange="applyFilters()">
651                 <option value="">所有情感倾向</option>
652             </select>
653         </div>
654         <div class="table-wrapper">
655             <table id="dataTable">
656                 <thead>
657                     <tr>
658                         <th style="width:9%;">诗歌编号</th>
659                         <th style="width:11%;">意象文本</th>
660                         <th style="width:20%;">所属诗歌</th>
661                         <th style="width:14%;">体裁/标签</th>
662                         <th style="width:18%;">大类归属</th>
663                         <th style="width:12%;">情感倾向</th>
664                         <th style="width:14%;">操作</th>
665                     </tr>
666                 </thead>
667                 <tbody id="traceback-body"></tbody>
668             </table>
669         </div>
```

```

670     <div class="pager">
671         <span id="page-info"></span>
672         <button type="button" id="page-prev" onclick="changePage(-1)">上一页</button>
673         <button type="button" id="page-next" onclick="changePage(1)">下一页</button>
674     </div>
675 </div>
676 </section>
677 <section id="tab-ai" class="tab-pane">
678     <div class="ai-hero">
679         <input type="text" id="rag-question" placeholder="输入关于唐诗的问题，例如：杜甫写过哪些关于秋天的诗？" onkeydown="if(event.key==='Enter'){"
680     <div>
681         <button type="button" onclick="runRagFirst()">检索并生成回答</button>
682     </div>
683     <p class="warn-text" id="rag-hint" style="display:none;"></p>
684 </div>
685 <div id="ai-split" class="ai-split">
686     <div class="poem-col" id="rag-poems"></div>
687     <div class="ai-answer">
688         <div class="ai-answer-scroll" id="rag-answer"></div>
689         <div class="ai-foot">
690             <input type="text" id="rag-followup" placeholder="多轮追问..." onkeydown="if(event.key==='Enter'){runRagFollowup();}">
691             <button type="button" onclick="runRagFollowup()">发送</button>
692         </div>
693     </div>
694 </div>
695 </section>
696 <div class="modal-overlay" id="detailModal">
697     <div class="modal-content">
698         <button type="button" class="close-btn" onclick="closeModal('detailModal')">×</button>
699         <div id="md-content"></div>
700     </div>
701 </div>
702 <div id="drawer-overlay" class="drawer-overlay" onclick="closeAnalyzeDrawer()"></div>
703 <aside id="analyze-drawer" class="analyze-drawer" aria-hidden="true">
704     <div id="drawer-resizer" class="drawer-resizer"></div>
705     <div class="drawer-head">
706         <button type="button" class="drawer-close" onclick="closeAnalyzeDrawer()" aria-label="关闭">×</button>
707         <div class="drawer-title-row" id="drawer-title-line">意象: <span class="hl" id="drawer-img-text">—</span></div>
708         <div class="drawer-meta" id="drawer-poem-meta">《》</div>
709         <div class="drawer-line" id="drawer-line-text">诗句: —</div>
710     </div>
711     <div class="drawer-tags" id="drawer-tags"></div>
712     <div class="drawer-chat-scroll" id="drawer-chat-scroll">
713         <div id="drawer-messages"></div>
714     </div>
715     <div class="drawer-foot">
716         <input type="text" id="drawer-followup" placeholder="输入追问..." onkeydown="if(event.key==='Enter'){sendDrawerFollowup();}">
717         <button type="button" onclick="sendDrawerFollowup()">发送</button>
718     </div>
719 </aside>
720 <div class="modal-overlay" id="recycleModal">
721     <div class="modal-content">
722         <button type="button" class="close-btn" onclick="closeModal('recycleModal')">×</button>
723         <h2 style="color:var(--warn);margin-top:0;border-bottom:2px solid #eee;padding-bottom:12px;">已删除意象回收站</h2>
724         <div class="table-wrapper" style="flex:1;max-height:62vh;">
725             <table>
726                 <thead>
727                     <tr>
728                         <th style="width:12%;">编号</th>
729                         <th style="width:15%;">被删意象</th>
730                         <th style="width:25%;">所属诗歌</th>
731                         <th style="width:28%;">大类</th>
732                         <th style="width:18%;">操作</th>
733                     </tr>

```

```

734         </thead>
735         <tbody id="recycle-body"></tbody>
736     </table>
737 </div>
738 </div>
739 </div>
740 <script>
741     const PAGE_SIZE = 25;
742     const RENDER_LIMIT = 5000;
743     let ALL_DATA = [];
744     let DELETED_DATA = [];
745     let filteredView = [];
746     let currentPage = 1;
747     let totalPoemsStat = 0;
748     let indChart = null;
749     let catChart = null;
750     const DEFAULT_ANALYZE_PROMPT = '请从认知诗学角度深度解析这个意象在诗歌中的多维功能';
751     /** 意象解析缓存 key = poem_id + '_' + 意象文本 */
752     const ANALYZE_CACHE = {};
753     let currentCacheKey = null;
754     function makeCacheKey(item) {
755         if (!item) return '';
756         const pid = String(item.poem_id ?? '');
757         const tx = String(item.txt ?? item['文本'] ?? '');
758         return pid + '_' + tx;
759     }
760     function isCacheAnalysisComplete(ent) {
761         if (!ent) return false;
762         if (ent.done === true) return true;
763         if (ent.done === false) return false;
764         return Array.isArray(ent.messages) && ent.messages.length >= 2;
765     }
766     let ragPoems = [];
767     let ragConv = [];
768     let ragBusy = false;
769     function ctxHint(item) {
770         const t = (item && (item['文本'] || item.txt)) || '';
771         return ` (语境: ${ (item && item.title) || '' } | 意象: ${t} | 诗句: ${ (item && item.line) || '' }) \n`;
772     }
773     function itemApiPayload(row) {
774         if (!row) return {};
775         const o = JSON.parse(JSON.stringify(row));
776         delete o._uid;
777         return o;
778     }
779     async function loadApiData() {
780         const res = await fetch('/api/data');
781         const data = await res.json();
782         totalPoemsStat = data.total_poems || 0;
783         ALL_DATA = data.traceback || [];
784         document.getElementById('stat-poems').innerText = totalPoemsStat;
785         ALL_DATA.forEach((item, index) => {
786             item._uid = 'img_' + index + '_' + Math.random().toString(36).substr(2, 5);
787         });
788         initCharts();
789         refreshAll();
790     }
791     function initCharts() {
792         const e11 = document.getElementById('chart-ind');
793         const e12 = document.getElementById('chart-cat');
794         if (!indChart) indChart = echarts.init(e11);
795         if (!catChart) catChart = echarts.init(e12);
796     }
797     function updateUIStats() {

```

```

798     document.getElementById('stat-images').innerText = ALL_DATA.length;
799     document.getElementById('recycle-count').innerText = DELETED_DATA.length;
800 }
801 function populateSelects() {
802     const cats = [...new Set(ALL_DATA.map(item => item.cat))].filter(Boolean);
803     const genres = [...new Set(ALL_DATA.map(item => item.genre))].filter(Boolean);
804     const emotions = [...new Set(ALL_DATA.map(item => item.emo_cat))].filter(Boolean);
805     const catSel = document.getElementById('filterCat');
806     const genSel = document.getElementById('filterGenre');
807     const emoSel = document.getElementById('filterEmotion');
808     const curCat = catSel.value, curGen = genSel.value, curEmo = emoSel.value;
809     catSel.innerHTML = '<option value="">所有分类域</option>';
810     genSel.innerHTML = '<option value="">所有体裁/标签</option>';
811     emoSel.innerHTML = '<option value="">所有情感倾向</option>';
812     cats.forEach(c => catSel.add(new Option(c, c)));
813     genres.forEach(g => genSel.add(new Option(g, g)));
814     emotions.forEach(e => emoSel.add(new Option(e, e)));
815     catSel.value = curCat; genSel.value = curGen; emoSel.value = curEmo;
816 }
817 function getFiltered() {
818     const searchText = document.getElementById('searchTxt').value.toLowerCase();
819     const selCat = document.getElementById('filterCat').value;
820     const selGen = document.getElementById('filterGenre').value;
821     const selEmo = document.getElementById('filterEmotion').value;
822     return ALL_DATA.filter(item => {
823         const t = (item.txt || item['文本'] || '').toLowerCase();
824         const matchSearch = t.includes(searchText);
825         const matchCat = selCat === '' || item.cat === selCat;
826         const matchGen = selGen === '' || item.genre === selGen;
827         const matchEmo = selEmo === '' || item.emo_cat === selEmo;
828         return matchSearch && matchCat && matchGen && matchEmo;
829     });
830 }
831 function renderTablePage() {
832     const dataArray = filteredView.slice(0, RENDER_LIMIT);
833     const total = dataArray.length;
834     const pages = Math.max(1, Math.ceil(total / PAGE_SIZE));
835     if (currentPage > pages) currentPage = pages;
836     const start = (currentPage - 1) * PAGE_SIZE;
837     const slice = dataArray.slice(start, start + PAGE_SIZE);
838     const tbody = document.getElementById('tbody-body');
839     let html = '';
840     slice.forEach(item => {
841         const apiPayload = itemApiPayload(item);
842         const dataItem = encodeURIComponent(JSON.stringify(apiPayload));
843         const uid = item._uid;
844         html += `<tr class="data-row" data-uid="${uid}" data-item="${dataItem}">
845             <td>${escapeHtml(item.poem_id)}</td>
846             <td style="color:#2980b9;font-weight:bold;">${escapeHtml(item.txt || item['文本'])}</td>
847             <td><b>${escapeHtml(item.title)}</b></td>
848             <td><span class="tag">${escapeHtml(item.genre)}</span></td>
849             <td>${escapeHtml(item.cat)}</td>
850             <td>${escapeHtml(item.emotion)}</td>
851             <td class="ops-cell">
852                 <button type="button" class="btn-link" onclick="openDetailModal('${uid}', event)">详情</button>
853                 <button type="button" class="btn-link" onclick="openAnalyzeDrawer('${uid}', event)">解析</button>
854                 <a href="javascript:void(0)" class="danger" onclick="event.stopPropagation();deleteItem('${uid}')">删除</a>
855             </td>
856         </tr>`;
857     });
858     if (total > RENDER_LIMIT) {
859         html += `<tr><td colspan="7" style="text-align:center;color:#e74c3c;background:#fff3f3;">仅渲染前 ${RENDER_LIMIT} 条筛选结果，请缩小筛选
860     `;
861     }
862     tbody.innerHTML = html || `<tr><td colspan="7" style="text-align:center;color:#999;">暂无数据</td></tr>`;

```

```

862 document.getElementById('record-count').innerText = `共命中 ${total} 条记录 · 第 ${currentPage}/${pages} 页`;
863 document.getElementById('page-info').innerText = `每页 ${PAGE_SIZE} 条`;
864 document.getElementById('page-prev').disabled = currentPage <= 1;
865 document.getElementById('page-next').disabled = currentPage >= pages;
866 }
867 function openDetailModal(uid, ev) {
868     if (ev) ev.stopPropagation();
869     const item = ALL_DATA.find(i => i._uid === uid);
870     if (!item) return;
871     const w = item['文本'] || item.txt || '';
872     document.getElementById('md-content').innerHTML = `
873         <h2 style="color:var(--primary);margin-top:0;border-bottom:2px solid #eee;padding-bottom:12px;">《${escapeHtml(item.title)}》 — 意象详
874         <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">意象文本</b><span style="color:#e74c3c;font-
875         <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">大类归属</b><span>${escapeHtml(item.cat)}</s
876         <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">结构化标注</b>
877         <div style="margin-top:8px;font-size:13px;line-height:1.7;color:#344;">
878             词性 ${escapeHtml(String(item['词性'] ?? ''))} · 成分 ${escapeHtml(String(item['成分类型'] ?? ''))}<br>
879             感知 ${escapeHtml(String(item['感知通道'] ?? ''))} · 素材 ${escapeHtml(String(item['素材类型'] ?? ''))}<br>
880             内部结构 ${escapeHtml(String(item['内部结构'] ?? ''))} · 指涉 ${escapeHtml(String(item['指涉来源'] ?? ''))}<br>
881             表现功能 ${escapeHtml(String(item['表现功能'] ?? ''))} · 结构组 ${escapeHtml(String(item['结构功能组'] ?? ''))}<br>
882             文化流通 ${escapeHtml(String(item['文化流通性'] ?? ''))} · 跨文化性 ${escapeHtml(String(item['跨文化性'] ?? ''))}<br>
883             认知强度 ${escapeHtml(String(item['认知强度'] ?? ''))} · 核心意象 ${escapeHtml(String(item['核心意象'] ?? ''))}<br>
884             情感极性 ${escapeHtml(String(item['情感极性'] ?? ''))} · 类别 ${escapeHtml(String(item['情感类别'] ?? ''))} · 置信度 ${escapeHtml
885             大类编码 ${escapeHtml(String(item['大类编码'] ?? ''))} · 子类 ${escapeHtml(String(item['子类编码'] ?? ''))}
886         </div>
887     </div>
888     <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">四层摘要</b><span style="color:#16a085;">${e
889     <div style="margin-bottom:10px;"><b style="color:#7f8c8d;width:120px;display:inline-block;">情感倾向</b><span style="color:#e67e22;">${e
890     <div><b style="color:#7f8c8d;display:block;margin-bottom:8px;">完整诗句</b>
891         <div style="background:#f8f9fa;padding:14px;border-radius:8px;font-family:楷体,KaiTi,serif;font-size:17px;line-height:1.65;">${escape
892     </div>`;
893     document.getElementById('detailModal').style.display = 'block';
894 }
895 function mdToHtml(text) {
896     if (!text) return '';
897     if (typeof marked !== 'undefined' && typeof marked.parse === 'function') {
898         try {
899             return marked.parse(text, { breaks: true });
900         } catch (e) {
901             return '<p>' + escapeHtml(text) + '</p>';
902         }
903     }
904     return '<p>' + escapeHtml(text) + '</p>';
905 }
906 function getItemPayloadByUid(uid) {
907     const tr = document.querySelector(`tr.data-row[data-uid="${uid}"]`);
908     try {
909         const raw = tr && tr.getAttribute('data-item');
910         if (raw) return JSON.parse(decodeURIComponent(raw));
911     } catch (e) { console.warn(e); }
912     return itemApiPayload(ALL_DATA.find(i => i._uid === uid));
913 }
914 function buildTagChips(item) {
915     const chips = [];
916     const pc = item['感知通道'];
917     if (pc) chips.push(String(pc));
918     const cat = item.cat || '';
919     const shortCat = cat.includes('-') ? cat.split('-').pop().trim() : cat;
920     if (shortCat) chips.push(shortCat);
921     const fx = item['表现功能'];
922     if (fx) chips.push(String(fx));
923     const cs = item['认知强度'];
924     if (cs !== '' && cs !== null) chips.push('认知强度' + cs);
925     const emo = item['情感类别'] || item.emo_cat || '';

```

```

926     const pol = item['情感极性'];
927     let emoChip = '';
928     if (emo && pol !== '' && pol !== null) emoChip = String(emo) + String(pol);
929     else if (emo) emoChip = String(emo);
930     else if (pol !== '' && pol !== null) emoChip = String(pol);
931     if (emoChip) chips.push(emoChip);
932     return chips.filter(Boolean);
933 }
934 function fillDrawerChrome(item) {
935     const w = item['文本'] || item.txt || '—';
936     document.getElementById('drawer-img-text').textContent = w;
937     document.getElementById('drawer-poem-meta').textContent = `《${item.title || ''}》 ${item.genre || ''}`.trim();
938     document.getElementById('drawer-line-text').textContent = '诗句: ' + (item.line || '—');
939     const tagBox = document.getElementById('drawer-tags');
940     const chips = buildTagChips(item);
941     tagBox.innerHTML = chips.map(c => `<span class="tag-chip">${escapeHtml(c)}</span>`).join('');
942 }
943 function scrollDrawerToBottom() {
944     const el = document.getElementById('drawer-chat-scroll');
945     if (el) el.scrollTop = el.scrollHeight;
946 }
947 function renderDrawerMessagesFromHistory(hist) {
948     const box = document.getElementById('drawer-messages');
949     box.innerHTML = '';
950     if (!hist || !hist.length) return;
951     hist.forEach((m) => {
952         if (m.role === 'user') {
953             const row = document.createElement('div');
954             row.className = 'msg-row user';
955             row.innerHTML = `<div class="msg-bubble user">${escapeHtml(m.content)}</div>`;
956             box.appendChild(row);
957         } else if (m.role === 'assistant') {
958             const row = document.createElement('div');
959             row.className = 'msg-row assistant';
960             const bubble = document.createElement('div');
961             bubble.className = 'msg-bubble assistant drawer-md';
962             bubble.innerHTML = mdToHtml(m.content);
963             row.appendChild(bubble);
964             box.appendChild(row);
965         }
966     });
967     scrollDrawerToBottom();
968 }
969 function openDrawerDom() {
970     document.getElementById('drawer-overlay').classList.add('show');
971     const dr = document.getElementById('analyze-drawer');
972     dr.classList.add('open');
973     dr.setAttribute('aria-hidden', 'false');
974 }
975 function closeAnalyzeDrawer() {
976     document.getElementById('drawer-overlay').classList.remove('show');
977     const dr = document.getElementById('analyze-drawer');
978     dr.classList.remove('open');
979     dr.setAttribute('aria-hidden', 'true');
980     currentCacheKey = null; // 仅仅是视角切走，后台不中断
981 }
982 function restoreFromCache(cacheKey) {
983     const e = ANALYZE_CACHE[cacheKey];
984     if (!e) return;
985     const box = document.getElementById('drawer-messages');
986     if (e.rendered) {
987         box.innerHTML = e.rendered;
988     } else if (e.messages && e.messages.length) {
989         renderDrawerMessagesFromHistory(e.messages);

```

```

990     e.rendered = box.innerHTML;
991 } else {
992     box.innerHTML = '';
993 }
994 scrollDrawerToBottom();
995 }
996 function openAnalyzeDrawer(uid, ev) {
997     if (ev) ev.stopPropagation();
998     const item = getItemPayloadByUid(uid);
999     const cacheKey = makeCacheKey(item);
1000    currentCacheKey = cacheKey;
1001    fillDrawerChrome(item);
1002    openDrawerDom();
1003    let ent = ANALYZE_CACHE[cacheKey];
1004    if (!ent) {
1005        ent = {
1006            item: JSON.parse(JSON.stringify(item)),
1007            messages: [],
1008            rendered: '',
1009            done: false,
1010            isStreaming: false,
1011            abortController: null
1012        };
1013        ANALYZE_CACHE[cacheKey] = ent;
1014    }
1015    if (ent.isStreaming) {
1016        restoreFromCache(cacheKey);
1017        return;
1018    }
1019    if (isCacheAnalysisComplete(ent)) {
1020        if (ent.done !== true) ent.done = true;
1021        restoreFromCache(cacheKey);
1022        return;
1023    }
1024    ent.messages = [];
1025    ent.rendered = '';
1026    ent.done = false;
1027    document.getElementById('drawer-messages').innerHTML = '';
1028    document.getElementById('drawer-followup').value = '';
1029    runDrawerInitialStream(cacheKey, item, ent);
1030 }
1031 async function runDrawerInitialStream(cacheKey, item, ent) {
1032     ent.isStreaming = true;
1033     ent.abortController = new AbortController();
1034     const bubbleId = 'bubble-' + Math.random().toString(36).substr(2, 9);
1035     let acc = '';
1036     const userHtml = `<div class="msg-row user"><div class="msg-bubble user">${escapeHtml(DEFAULT_ANALYZE_PROMPT)}</div></div>`;
1037     const asstHtml = `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">...</div></div>`;
1038     ent.rendered = userHtml + asstHtml;
1039     if (currentCacheKey === cacheKey) {
1040         document.getElementById('drawer-messages').innerHTML = ent.rendered;
1041     }
1042     try {
1043         const res = await fetch('/api/analyze', {
1044             method: 'POST',
1045             headers: { 'Content-Type': 'application/json' },
1046             body: JSON.stringify({ item, history: [], followup: DEFAULT_ANALYZE_PROMPT }),
1047             signal: ent.abortController.signal
1048         });
1049         await readSSE(res, (obj) => {
1050             if (obj.type === 'chunk' && obj.text) {
1051                 acc += obj.text;
1052             } else if (obj.type === 'error') {
1053                 acc += '\n\n【错误】' + (obj.message || '');

```



```

1054     }
1055     ent.rendered = userHtml + `
```

```

1118     } else if (obj.type === 'error') {
1119         acc += '\n\n【错误】' + (obj.message || '');
1120     }
1121     ent.rendered = baseRendered + newQ + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${(
1122     if (currentCacheKey === cacheKey) {
1123         const bubble = document.getElementById(bubbleId);
1124         if (bubble) bubble.innerHTML = mdToHtml(acc);
1125         scrollDrawerToBottom();
1126     }
1127     }, { signal: ent.abortController.signal });
1128     if (!ent.abortController.signal.aborted) {
1129         ent.messages.push({ role: 'user', content: q }, { role: 'assistant', content: acc });
1130     }
1131 } catch (e) {
1132     if (e.name !== 'AbortError') {
1133         acc += '\n\n<p class="danger">请求失败: ' + escapeHtml(String(e)) + '</p>';
1134         ent.rendered = baseRendered + newQ + `<div class="msg-row assistant"><div class="msg-bubble assistant drawer-md" id="${bubbleId}">${(
1135         if (currentCacheKey === cacheKey) {
1136             const bubble = document.getElementById(bubbleId);
1137             if (bubble) bubble.innerHTML = mdToHtml(acc);
1138         }
1139         }
1140     } finally {
1141         ent.isStreaming = false;
1142         ent.abortController = null;
1143         if (currentCacheKey === cacheKey) scrollDrawerToBottom();
1144     }
1145 }
1146 function escapeHtml(s) {
1147     if (!s) return '';
1148     return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
1149 }
1150 function changePage(delta) {
1151     const dataArray = filteredView.slice(0, RENDER_LIMIT);
1152     const pages = Math.max(1, Math.ceil(dataArray.length / PAGE_SIZE));
1153     currentPage = Math.min(pages, Math.max(1, currentPage + delta));
1154     renderTablePage();
1155 }
1156 function applyFilters() {
1157     filteredView = getFiltered();
1158     currentPage = 1;
1159     const searchText = document.getElementById('searchTxt').value;
1160     const selCat = document.getElementById('filterCat').value;
1161     const selGen = document.getElementById('filterGenre').value;
1162     const selEmo = document.getElementById('filterEmotion').value;
1163     const isFiltered = !(searchText || selCat || selGen || selEmo);
1164     document.getElementById('resetBtn').style.display = isFiltered ? 'inline-flex' : 'none';
1165     renderTablePage();
1166 }
1167 function updateCharts() {
1168     let individualCounts = {};
1169     let categoryCounts = {};
1170     ALL_DATA.forEach(item => {
1171         const tk = item.txt || item['文本'] || '';
1172         individualCounts[tk] = (individualCounts[tk] || 0) + 1;
1173         categoryCounts[item.cat] = (categoryCounts[item.cat] || 0) + 1;
1174     });
1175     const sortedInd = Object.entries(individualCounts).sort((a, b) => b[1] - a[1]).slice(0, 50);
1176     const sortedCat = Object.entries(categoryCounts).sort((a, b) => b[1] - a[1]);
1177     indChart.setOption({
1178         tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
1179         grid: { top: '10%', left: '5%', right: '5%', bottom: '25%' },
1180         xAxis: { type: 'category', data: sortedInd.map(x => x[0]), axisLabel: { interval: 0, rotate: 30 } },
1181         yAxis: { type: 'value' },

```

```

1182     dataZoom: [{ type: 'slider', show: true, xAxisIndex: [0], start: 0, end: 40, bottom: 0 }],
1183     series: [{ name: '频次', type: 'bar', barWidth: '60%', data: sortedInd.map(x => x[1]), itemStyle: { color: '#3498db' } }],
1184   }, true);
1185   catChart.setOption({
1186     tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
1187     grid: { top: '10%', left: '5%', right: '5%', bottom: '15%' },
1188     xAxis: { type: 'category', data: sortedCat.map(x => x[0]), axisLabel: { interval: 0, rotate: 15 } },
1189     yAxis: { type: 'value' },
1190     series: [{ name: '频次', type: 'bar', barWidth: '50%', data: sortedCat.map(x => x[1]), itemStyle: { color: '#9b59b6' } }],
1191   }, true);
1192 }
1193 function refreshAll() {
1194   updateUIStats();
1195   populateSelects();
1196   filteredView = getFiltered();
1197   renderTablePage();
1198   updateCharts();
1199 }
1200 function deleteItem(uid) {
1201   const targetIndex = ALL_DATA.findIndex(item => item._uid === uid);
1202   if (targetIndex === -1) return;
1203   const targetItem = ALL_DATA.splice(targetIndex, 1)[0];
1204   DELETED_DATA.push(targetItem);
1205   const ck = makeCacheKey(targetItem);
1206   if (ck && ANALYZE_CACHE[ck]) {
1207     const ent = ANALYZE_CACHE[ck];
1208     if (ent.isStreaming && ent.abortController) {
1209       ent.abortController.abort();
1210     }
1211     delete ANALYZE_CACHE[ck];
1212   }
1213   if (currentCacheKey === ck) {
1214     closeAnalyzeDrawer();
1215   }
1216   refreshAll();
1217 }
1218 function restoreItem(uid) {
1219   const targetIndex = DELETED_DATA.findIndex(item => item._uid === uid);
1220   if (targetIndex === -1) return;
1221   const targetItem = DELETED_DATA.splice(targetIndex, 1)[0];
1222   ALL_DATA.push(targetItem);
1223   refreshAll();
1224   renderRecycleBin();
1225 }
1226 function renderRecycleBin() {
1227   const tbody = document.getElementById('recycle-body');
1228   let html = '';
1229   if (DELETED_DATA.length === 0) {
1230     html = '<tr><td colspan="5" style="text-align:center;padding:24px;color:#999;">回收站为空</td></tr>';
1231   } else {
1232     DELETED_DATA.forEach(item => {
1233       html += `<tr>
1234         <td>${escapeHtml(item.poem_id)}</td>
1235         <td style="color:#e74c3c;font-weight:bold;text-decoration:line-through;">${escapeHtml(item.txt)}</td>
1236         <td>${escapeHtml(item.title)}</td>
1237         <td>${escapeHtml(item.cat)}</td>
1238         <td><button type="button" class="btn-action" style="background:var(--success);color:#fff;padding:6px 10px;font-size:12px;" onclick=
1239       </tr>`;
1240     });
1241   }
1242   tbody.innerHTML = html;
1243 }
1244 function openRecycle() {
1245   renderRecycleBin();

```

```

1246     document.getElementById('recycleModal').style.display = 'block';
1247 }
1248 function closeModal(id) {
1249     document.getElementById(id).style.display = 'none';
1250 }
1251 function resetFilters() {
1252     document.getElementById('searchTxt').value = '';
1253     document.getElementById('filterCat').value = '';
1254     document.getElementById('filterGenre').value = '';
1255     document.getElementById('filterEmotion').value = '';
1256     applyFilters();
1257 }
1258 function filterFromChart(text, isCategory) {
1259     resetFilters();
1260     if (isCategory) document.getElementById('filterCat').value = text;
1261     else document.getElementById('searchTxt').value = text;
1262     applyFilters();
1263     document.getElementById('dataTable').scrollIntoView({ behavior:'smooth', block:'start' });
1264 }
1265 function toggleFullScreen(elemId) {
1266     const elem = document.getElementById(elemId);
1267     if (!document.fullscreenElement) {
1268         elem.requestFullscreen().catch(err => alert('全屏请求失败: ' + err.message));
1269     } else {
1270         document.exitFullscreen();
1271     }
1272 }
1273 document.addEventListener('fullscreenchange', () => {
1274     const isFull = !!document.fullscreenElement;
1275     document.querySelectorAll('.chart-container').forEach(el => {
1276         el.style.height = isFull ? 'calc(100vh - 80px)' : '400px';
1277     });
1278     if (indChart) indChart.resize();
1279     if (catChart) catChart.resize();
1280 });
1281 async function readSSE(response, onJson, opts) {
1282     const signal = opts && opts.signal;
1283     if (signal && signal.aborted) return;
1284     const reader = response.body.getReader();
1285     const dec = new TextDecoder();
1286     let buf = '';
1287     while (true) {
1288         if (signal && signal.aborted) {
1289             try { await reader.cancel(); } catch (e) { /* ignore */ }
1290             break;
1291         }
1292         let readResult;
1293         try {
1294             readResult = await reader.read();
1295         } catch (err) {
1296             if (signal && signal.aborted) break;
1297             throw err;
1298         }
1299         const { value, done } = readResult;
1300         if (done) break;
1301         buf += dec.decode(value, { stream: true });
1302         const parts = buf.split('\n\n');
1303         buf = parts.pop() || '';
1304         for (const block of parts) {
1305             const lines = block.split('\n');
1306             for (const line of lines) {
1307                 if (line.startsWith('data: ')) {
1308                     try {
1309                         const obj = JSON.parse(line.slice(6));

```

```

1310         onJson(obj);
1311     } catch (e) { console.warn(e); }
1312 }
1313 }
1314 }
1315 }
1316 }
1317 async function runRagFirst() {
1318     const q = (document.getElementById('rag-question').value || '').trim();
1319     const hint = document.getElementById('rag-hint');
1320     hint.style.display = 'none';
1321     if (!q || ragBusy) return;
1322     ragBusy = true;
1323     ragConv = [];
1324     document.getElementById('rag-answer').innerText = '';
1325     document.getElementById('rag-poems').innerHTML = '';
1326     document.getElementById('ai-split').classList.add('active');
1327     let ans = '';
1328     try {
1329         const res = await fetch('/api/ask', {
1330             method: 'POST',
1331             headers: { 'Content-Type': 'application/json' },
1332             body: JSON.stringify({ question: q, history: [] })
1333         });
1334         await readSSE(res, (obj) => {
1335             if (obj.type === 'poems') {
1336                 ragPoems = obj.poems || [];
1337                 renderRagPoems();
1338             } else if (obj.type === 'chunk' && obj.text) {
1339                 ans += obj.text;
1340                 document.getElementById('rag-answer').innerText = ans;
1341                 const el = document.getElementById('rag-answer');
1342                 el.scrollTop = el.scrollHeight;
1343             } else if (obj.type === 'error') {
1344                 hint.innerText = obj.message || '出错';
1345                 hint.style.display = 'block';
1346             }
1347         });
1348         if (ans) ragConv = [
1349             { role: 'user', content: q },
1350             { role: 'assistant', content: ans }
1351         ];
1352     } catch (e) {
1353         hint.innerText = '请求失败: ' + e;
1354         hint.style.display = 'block';
1355     } finally {
1356         ragBusy = false;
1357     }
1358 }
1359 function renderRagPoems() {
1360     const box = document.getElementById('rag-poems');
1361     if (!ragPoems.length) {
1362         box.innerHTML = '<p style="color:#999;">暂无检索结果</p>';
1363         return;
1364     }
1365     box.innerHTML = ragPoems.map((p) => `
1366         <div class="poem-card" onclick="this.classList.toggle('expanded')">
1367             <h4>《${escapeHtml(p['标题'])}》 ${escapeHtml(p['作者'])}</h4>
1368             <div class="sub">相似度: ${p['相似度']}</div>
1369             <div class="body">${escapeHtml((p['原文'] || '').slice(0, 120))}${(p['原文'] || '').length>120?'...':''}</div>
1370             <div class="poem-detail">${escapeHtml(p['原文'] || '')}</div>
1371             <div class="sim">点击卡片展开全文</div>
1372         </div>
1373     `).join('');

```

```

1374 }
1375 async function runRagFollowup() {
1376     const q = (document.getElementById('rag-followup').value || '').trim();
1377     const firstQ = document.getElementById('rag-question').value.trim();
1378     if (!q || ragBusy || !firstQ || !ragConv.length) return;
1379     ragBusy = true;
1380     document.getElementById('rag-followup').value = '';
1381     const ansEl = document.getElementById('rag-answer');
1382     ansEl.innerHTML += `\\n\\n【问】${q}\\n\\n【答】`;
1383     let acc = '';
1384     const markPos = ansEl.innerHTML.length;
1385     const hist = ragConv.map(m => ({ role: m.role, content: m.content }));
1386     try {
1387         const res = await fetch('/api/ask', {
1388             method: 'POST',
1389             headers: { 'Content-Type': 'application/json' },
1390             body: JSON.stringify({ question: q, history: hist })
1391         });
1392         await readSSE(res, (obj) => {
1393             if (obj.type === 'chunk' && obj.text) {
1394                 acc += obj.text;
1395                 ansEl.innerHTML = ansEl.innerHTML.slice(0, markPos) + acc;
1396                 ansEl.scrollTop = ansEl.scrollHeight;
1397             } else if (obj.type === 'error') {
1398                 ansEl.innerHTML += `\\n【错误】` + (obj.message || '');
1399             }
1400         });
1401         ragConv.push({ role: 'user', content: q });
1402         ragConv.push({ role: 'assistant', content: acc });
1403     } catch (e) {
1404         ansEl.innerHTML += `\\n请求失败: ` + e;
1405     } finally {
1406         ragBusy = false;
1407     }
1408 }
1409 /* Tab switch */
1410 document.querySelectorAll('.tab-btn').forEach(btn => {
1411     btn.addEventListener('click', () => {
1412         document.querySelectorAll('.tab-btn').forEach(b => b.classList.remove('active'));
1413         btn.classList.add('active');
1414         const tab = btn.getAttribute('data-tab');
1415         document.querySelectorAll('.tab-pane').forEach(p => {
1416             p.classList.remove('visible');
1417         });
1418         if (tab === 'graph') {
1419             document.getElementById('tab-graph').classList.add('visible');
1420             setTimeout(() => { if (indChart) indChart.resize(); if (catChart) catChart.resize(); }, 200);
1421         } else {
1422             document.getElementById('tab-ai').classList.add('visible');
1423         }
1424     });
1425 });
1426 window.addEventListener('resize', () => {
1427     if (indChart) indChart.resize();
1428     if (catChart) catChart.resize();
1429 });
1430 loadApiData().then(() => {
1431     if (indChart) indChart.on('click', p => filterFromChart(p.name, false));
1432     if (catChart) catChart.on('click', p => filterFromChart(p.name, true));
1433 });
1434 /* --- 抽屉宽度拖拽 (IDE Style) --- */
1435 const drw = document.getElementById('analyze-drawer');
1436 const resizer = document.getElementById('drawer-resizer');
1437 let isResizing = false;

```

```

1438     resizer.addEventListener('mousedown', (e) => {
1439         isResizing = true;
1440         resizer.classList.add('active');
1441         drw.classList.add('no-transition');
1442         document.body.style.cursor = 'ew-resize';
1443         document.body.style.userSelect = 'none';
1444     });
1445     document.addEventListener('mousemove', (e) => {
1446         if (!isResizing) return;
1447         let newWidth = window.innerWidth - e.clientX;
1448         if (newWidth < 320) newWidth = 320;
1449         if (newWidth > window.innerWidth - 80) newWidth = window.innerWidth - 80;
1450         drw.style.width = newWidth + 'px';
1451     });
1452     document.addEventListener('mouseup', () => {
1453         if (isResizing) {
1454             isResizing = false;
1455             resizer.classList.remove('active');
1456             drw.classList.remove('no-transition');
1457             document.body.style.cursor = '';
1458             document.body.style.userSelect = '';
1459         }
1460     });
1461 </script>
1462 </body>
1463 </html>

```

-- 文件结束: templates/index.html --

文件: templates/admin.html (363 行, 12.3 KB)

```

1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>系统管理 — 唐诗意象智能分析系统</title>
7   <script src="https://cdn.jsdelivr.net/npm/echarts@5.5.0/dist/echarts.min.js"></script>
8   <style>
9     :root {
10       --primary: #2c3e50;
11       --accent: #3498db;
12       --bg: #f4f7f6;
13       --card: #ffffff;
14       --success: #2ecc71;
15       --warn: #e67e22;
16       --danger: #e74c3c;
17     }
18     * { box-sizing: border-box; }
19     body {
20       font-family: 'Helvetica Neue', Arial, sans-serif;
21       background: var(--bg);
22       margin: 0;
23       padding: 0;
24       color: #333;
25     }
26     .navbar {
27       background: var(--primary);
28       color: #fff;
29       padding: 16px 28px;
30       display: flex;
31       align-items: center;
32       justify-content: space-between;
33     }
34     .navbar h1 { margin: 0; font-size: 20px; }
35     .navbar a { color: #bdc3c7; text-decoration: none; font-size: 14px; }
36     .navbar a:hover { color: #fff; }

```

```

37 .container { max-width: 1400px; margin: 24px auto; padding: 0 20px; }
38 .stats-grid {
39     display: grid;
40     grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
41     gap: 16px;
42     margin-bottom: 24px;
43 }
44 .stat-card {
45     background: var(--card);
46     border-radius: 10px;
47     padding: 20px;
48     box-shadow: 0 2px 8px rgba(0,0,0,0.06);
49     text-align: center;
50 }
51 .stat-card .num { font-size: 32px; font-weight: 800; color: var(--accent); }
52 .stat-card .label { font-size: 13px; color: #888; margin-top: 6px; }
53 .card {
54     background: var(--card);
55     border-radius: 10px;
56     padding: 24px;
57     margin-bottom: 24px;
58     box-shadow: 0 2px 8px rgba(0,0,0,0.06);
59 }
60 .card-title {
61     margin: 0 0 16px;
62     font-size: 18px;
63     color: var(--primary);
64     border-left: 4px solid var(--accent);
65     padding-left: 12px;
66 }
67 .grid-2 {
68     display: grid;
69     grid-template-columns: 1fr 1fr;
70     gap: 24px;
71 }
72 @media (max-width: 800px) { .grid-2 { grid-template-columns: 1fr; } }
73 .chart-box { height: 340px; width: 100%; }
74 .prop-table { width: 100%; border-collapse: collapse; font-size: 14px; }
75 .prop-table th, .prop-table td {
76     border: 1px solid #eee;
77     padding: 10px 14px;
78     text-align: left;
79 }
80 .prop-table th { background: #fafafa; font-weight: 600; }
81 .prop-table tr:hover { background: #f8f9fa; }
82 .btn {
83     padding: 10px 22px;
84     border-radius: 8px;
85     border: none;
86     font-weight: 600;
87     cursor: pointer;
88     transition: 0.2s;
89     font-size: 14px;
90 }
91 .btn-primary { background: var(--accent); color: #fff; }
92 .btn-primary:hover { filter: brightness(1.05); }
93 .btn-success { background: var(--success); color: #fff; }
94 .btn-success:hover { filter: brightness(1.05); }
95 .btn-warn { background: var(--warn); color: #fff; }
96 .btn-warn:hover { filter: brightness(1.05); }
97 .btn-danger { background: var(--danger); color: #fff; }
98 .btn-danger:hover { filter: brightness(1.05); }
99 .btn-sm { padding: 6px 14px; font-size: 12px; }
100 .action-bar {

```



```
101     display: flex;
102     gap: 12px;
103     flex-wrap: wrap;
104     margin-bottom: 16px;
105 }
106 .tag {
107     background: #ecf0f1;
108     padding: 3px 10px;
109     border-radius: 20px;
110     font-size: 12px;
111     display: inline-block;
112 }
113 .tag-green { background: #d5f5e3; color: #27ae60; }
114 .tag-red { background: #fadbd8; color: #c0392b; }
115 .status-dot {
116     display: inline-block;
117     width: 10px; height: 10px;
118     border-radius: 50%;
119     margin-right: 6px;
120 }
121 .status-dot.online { background: var(--success); }
122 .export-list { max-height: 300px; overflow-y: auto; }
123 .toast {
124     position: fixed;
125     bottom: 30px;
126     right: 30px;
127     background: var(--primary);
128     color: #fff;
129     padding: 14px 24px;
130     border-radius: 10px;
131     font-size: 14px;
132     opacity: 0;
133     transform: translateY(20px);
134     transition: 0.3s;
135     z-index: 999;
136 }
137 .toast.show { opacity: 1; transform: translateY(0); }
138 </style>
139 </head>
140 <body>
141     <nav class="navbar">
142         <h1> 系统管理后台</h1>
143         <div>
144             <a href="/">← 返回前台</a>
145         </div>
146     </nav>
147     <div class="container">
148         <div id="stats-panel" class="stats-grid"></div>
149         <div class="grid-2">
150             <!-- 分类分布 -->
151             <div class="card">
152                 <h2 class="card-title">意象分类域分布</h2>
153                 <div id="chart-cat" class="chart-box"></div>
154             </div>
155             <!-- 情感分布 -->
156             <div class="card">
157                 <h2 class="card-title">情感类别分布</h2>
158                 <div id="chart-emo" class="chart-box"></div>
159             </div>
160         </div>
161         <!-- 系统配置 -->
162         <div class="card">
163             <h2 class="card-title">系统配置</h2>
164             <table class="prop-table" id="config-table"></table>
```

```

165     </div>
166     <!-- 操作区 -->
167     <div class="card">
168         <h2 class="card-title">操作面板</h2>
169         <div class="action-bar">
170             <button class="btn btn-primary" onclick="refreshData()"> 刷新数据缓存</button>
171             <button class="btn btn-success" onclick="exportData(' csv')"> 导出 CSV</button>
172             <button class="btn btn-success" onclick="exportData(' json')"> 导出 JSON</button>
173             <button class="btn btn-success" onclick="exportData(' report')"> 导出分析报告</button>
174             <button class="btn btn-warn" onclick="clearCache()"> 清理缓存</button>
175         </div>
176     </div>
177     <!-- 导出文件列表 -->
178     <div class="card">
179         <h2 class="card-title">导出文件列表</h2>
180         <div class="action-bar">
181             <button class="btn btn-danger btn-sm" onclick="clearExports()">清空全部</button>
182         </div>
183         <div class="export-list" id="export-list"><p style="color:#999;">加载中...</p></div>
184     </div>
185 </div>
186 <div id="toast" class="toast"></div>
187 <script>
188     let catChart = null;
189     let emoChart = null;
190     async function fetchJSON(url) {
191         const res = await fetch(url);
192         if (!res.ok) throw new Error(` HTTP ${res.status}`);
193         return res.json();
194     }
195     function showToast(msg, isError) {
196         const el = document.getElementById(' toast');
197         el.textContent = msg;
198         el.style.background = isError ? '#e74c3c' : '#2c3e50';
199         el.classList.add(' show');
200         setTimeout(() => el.classList.remove(' show'), 3000);
201     }
202     async function loadStatus() {
203         try {
204             const s = await fetchJSON('/admin/api/status');
205             const panel = document.getElementById(' stats-panel');
206             panel.innerHTML = `
207                 <div class="stat-card"><div class="num">${s.data.total_poems}</div><div class="label">净解析诗歌</div></div>
208                 <div class="stat-card"><div class="num">${s.data.total_images}</div><div class="label">提取意象条目</div></div>
209                 <div class="stat-card"><div class="num">${s.data.category_count}</div><div class="label">分类维度</div></div>
210                 <div class="stat-card"><div class="num">${s.data.known_authors_count}</div><div class="label">已知诗人</div></div>
211                 <div class="stat-card">
212                     <div class="num">${s.rag_database.exists ? '<span style="color:#2ecc71;">V</span>' : '<span style="color:#e74c3c;">X</span>'}</div>
213                     <div class="label">向量数据库 (${s.rag_database.file_count || 0} 文件)</div>
214                 </div>
215                 <div class="stat-card">
216                     <div class="num">${s.cache.memory_entries}</div>
217                     <div class="label">内存缓存条目</div>
218                 </div>
219                 <div class="stat-card">
220                     <div class="num">${s.exports.file_count}</div>
221                     <div class="label">导出文件</div>
222                 </div>
223                 <div class="stat-card">
224                     <div class="num">${s.poem_json.file_count}</div>
225                     <div class="label">诗歌数据文件</div>
226                 </div>
227             `;
228             const configTable = document.getElementById(' config-table');

```

```

229     configTable.innerHTML = Object.entries(s.config).map(([k, v]) =>
230       `<tr><td>${k}</td><td>${v}</td></tr>`
231     ).join('');
232   } catch (e) {
233     showToast('加载状态失败: ' + e.message, true);
234   }
235 }
236 async function loadStats() {
237   try {
238     const stats = await fetchJSON('/admin/api/stats');
239     const catData = (stats.category_stats || []).slice(0, 30);
240     const emoData = (stats.emotion_stats || []).slice(0, 20);
241     if (!catChart) {
242       catChart = echarts.init(document.getElementById('chart-cat'));
243     }
244     catChart.setOption({
245       tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
246       grid: { top: '8%', left: '15%', right: '5%', bottom: '20%' },
247       xAxis: { type: 'value' },
248       yAxis: { type: 'category', data: catData.map(d => d.category).reverse(), axisLabel: { fontSize: 11 } },
249       series: [{ type: 'bar', data: catData.map(d => d.count).reverse(), itemStyle: { color: '#9b59b6' }, barWidth: '60%' }]
250     }, true);
251     if (!emoChart) {
252       emoChart = echarts.init(document.getElementById('chart-emo'));
253     }
254     emoChart.setOption({
255       tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
256       grid: { top: '8%', left: '5%', right: '5%', bottom: '20%' },
257       xAxis: { type: 'category', data: emoData.map(d => d.emotion), axisLabel: { interval: 0, rotate: 20, fontSize: 11 } },
258       yAxis: { type: 'value' },
259       series: [{ type: 'bar', data: emoData.map(d => d.count), itemStyle: { color: '#e67e22' }, barWidth: '50%' }]
260     }, true);
261     window.addEventListener('resize', () => { catChart && catChart.resize(); emoChart && emoChart.resize(); });
262   } catch (e) {
263     showToast('加载统计数据失败: ' + e.message, true);
264   }
265 }
266 async function loadExports() {
267   try {
268     const data = await fetchJSON('/admin/api/exports');
269     const list = document.getElementById('export-list');
270     if (!data.exports || !data.exports.length) {
271       list.innerHTML = '<p style="color:#999;">暂无导出文件</p>';
272       return;
273     }
274     list.innerHTML = '<table class="prop-table">' +
275       '<thead><tr><th>文件名</th><th>大小</th><th>修改时间</th></tr></thead><tbody>' +
276     data.exports.map(f => {
277       const size = f.size > 1024 ? (f.size / 1024).toFixed(1) + ' KB' : f.size + ' B';
278       const time = new Date(f.mtime * 1000).toLocaleString();
279       return `<tr><td>${f.name}</td><td>${size}</td><td>${time}</td></tr>`;
280     }).join('') + '</tbody></table>';
281   } catch (e) {
282     document.getElementById('export-list').innerHTML = '<p style="color:#e74c3c;">加载失败</p>';
283   }
284 }
285 async function refreshData() {
286   try {
287     const res = await fetchJSON('/admin/api/refresh');
288     showToast(`数据已刷新: ${res.total_poems} 首诗歌, ${res.total_images} 条意象`);
289     loadStatus();
290     loadStats();
291   } catch (e) {
292     showToast('刷新失败: ' + e.message, true);

```

```

293     }
294 }
295 async function exportData(fmt) {
296     try {
297         const res = await fetch('/admin/api/exports', {
298             method: 'POST',
299             headers: { 'Content-Type': 'application/json' },
300             body: JSON.stringify({ format: fmt })
301         });
302         const data = await res.json();
303         if (data.error) { showToast(data.error, true); return; }
304         showToast(`导出成功: ${data.file} (${(data.size / 1024).toFixed(1)} KB)`);
305         loadExports();
306     } catch (e) {
307         showToast('导出失败: ' + e.message, true);
308     }
309 }
310 async function clearCache() {
311     try {
312         const res = await fetch('/admin/api/cache/clear', {
313             method: 'POST',
314             headers: { 'Content-Type': 'application/json' },
315             body: JSON.stringify({ type: 'all' })
316         });
317         const data = await res.json();
318         showToast(data.message);
319         loadStatus();
320     } catch (e) {
321         showToast('清理失败: ' + e.message, true);
322     }
323 }
324 async function clearExports() {
325     if (!confirm('确定要清空所有导出文件吗?')) return;
326     try {
327         const res = await fetch('/admin/api/exports/clear', { method: 'POST' });
328         const data = await res.json();
329         showToast(data.message);
330         loadExports();
331     } catch (e) {
332         showToast('清空失败: ' + e.message, true);
333     }
334 }
335 // 初始化
336 loadStatus();
337 loadStats();
338 loadExports();
339 </script>
340 </body>
341 </html>

```

-- 文件结束: templates/admin.html --

文件: json_to_3csv.py (163 行, 7.0 KB)

```

1 import json
2 import csv
3 import os
4 import glob
5 def aggregate_jsons_to_csv():
6     base_dir = os.path.dirname(os.path.abspath(__file__))
7     json_folder = os.path.join(base_dir, 'json_data')
8     if not os.path.exists(json_folder):
9         print(f" 找不到文件夹: {json_folder}, 请创建并放入JSON文件!")
10        return
11    json_files = glob.glob(os.path.join(json_folder, '*.json'))
12    if not json_files:
13        print(" 文件夹里没有找到任何 .json 文件!")

```

```

14         return
15     print(f" 发现 {len(json_files)} 个 JSON 文件，正在执行安全解析与聚合...")
16     # --- 1. 定义三张表的表头 ---
17     header_a = [
18         "诗歌编号", "标题", "作者", "分类标签", "诗行编号",
19         "单元编号", "文本", "行内位置", "词性", "成分类型",
20         "是否意象", "大类编码", "子类编码", "感知通道", "素材类型",
21         "内部结构", "指涉来源", "表现功能", "文化流通性", "跨文化性",
22         "认知强度", "核心意象", "结构功能组", "情感极性", "情感类别", "情感置信度"
23     ]
24     header_b = [
25         "诗歌编号", "标题", "作者", "分类标签",
26         "诗行编号", "诗行原文",
27         "平均情感极性", "主导情感", "情感波动值"
28     ]
29     header_c = [
30         "诗歌编号", "标题", "作者", "关系编号",
31         "来源单元编号", "来源文本",
32         "目标单元编号", "目标文本",
33         "关系类型", "关系强度"
34     ]
35     rows_a, rows_b, rows_c = [], [], []
36     # 全局计数器（解决所有文件都从 P01 和 U001 开始的 Bug）
37     global_poem_idx = 1
38     global_unit_idx = 1
39     global_rel_idx = 1
40     global_traj_idx = 1
41     # --- 2. 遍历所有 JSON 文件 ---
42     for file_path in json_files:
43         try:
44             with open(file_path, 'r', encoding='utf-8') as f:
45                 data = json.load(f)
46         except Exception as e:
47             print(f" 解析失败，跳过损坏的 JSON 文件 {os.path.basename(file_path)}: {e}")
48             continue
49         poems = data.get("诗歌集", []) if isinstance(data, dict) else data
50         # --- 3. 遍历文件内的诗歌 ---
51         for p in poems:
52             # 分配全局唯一的诗歌 ID
53             new_p_id = f"P{global_poem_idx:03d}"
54             global_poem_idx += 1
55             title = p.get("标题", "")
56             author = p.get("作者", "")
57             tag = p.get("分类标签", "")
58             base_meta = [new_p_id, title, author, tag]
59             # 建立旧 ID 到新 ID 的映射字典，防止指针错乱
60             line_id_map = {}
61             unit_id_map = {}
62             unit_text_map = {} # 用于关系表反查文本
63             # 3.1 提取诗行并映射新 ID
64             line_text_map = {}
65             for i, line in enumerate(p.get("诗行", []), 1):
66                 old_l_id = line.get("诗行编号", "")
67                 new_l_id = f"{new_p_id}_L{i:02d}"
68                 line_id_map[old_l_id] = new_l_id
69                 line_text_map[new_l_id] = line.get("原文", "")
70             # 3.2 解析表 A: 分析单元（并映射全局单元 ID）
71             for u in p.get("分析单元", []):
72                 old_u_id = u.get("单元编号", "")
73                 new_u_id = f"U{global_unit_idx:05d}"
74                 global_unit_idx += 1
75                 unit_id_map[old_u_id] = new_u_id
76                 text_val = u.get("文本", "")
77                 unit_text_map[new_u_id] = text_val

```

```

78         # 转换对应的诗行 ID
79         old_u_line = u.get("诗行编号", "")
80         mapped_l_id = line_id_map.get(old_u_line, old_u_line)
81         rows_a.append([
82             new_p_id, title, author, tag, mapped_l_id,
83             new_u_id, text_val, u.get("行内位置", ""),
84             u.get("词性", ""), u.get("成分类型", ""), u.get("是否意象", ""),
85             u.get("大类编码", ""), u.get("子类编码", ""), u.get("感知通道", ""),
86             u.get("素材类型", ""), u.get("内部结构", ""), u.get("指涉来源", ""),
87             u.get("表现功能", ""), u.get("文化流通性", ""), u.get("跨文化性", ""),
88             u.get("认知强度", ""), u.get("核心意象", ""), u.get("结构功能组", ""),
89             u.get("情感极性", ""), u.get("情感类别", ""), u.get("情感置信度", "")
90         ])
91     # 3.3 解析表 B: 情感轨迹
92     for traj in p.get("情感轨迹", []):
93         old_t_line = traj.get("诗行编号", "")
94         mapped_t_line = line_id_map.get(old_t_line, old_t_line)
95         rows_b.append(base_meta + [
96             mapped_t_line, line_text_map.get(mapped_t_line, ""),
97             traj.get("平均情感极性", ""), traj.get("主导情感", ""), traj.get("情感波动值", "")
98         ])
99     # 3.4 解析表 C: 意象关系网络 (核心: 修复指针)
100    for r in p.get("意象关系", []):
101        new_r_id = f"R{global_rel_idx:04d}"
102        global_rel_idx += 1
103        old_src = r.get("来源单元", "")
104        old_tgt = r.get("目标单元", "")
105        # 指针映射到新的全局 ID
106        new_src = unit_id_map.get(old_src, old_src)
107        new_tgt = unit_id_map.get(old_tgt, old_tgt)
108        rows_c.append([new_p_id, title, author] + [
109            new_r_id,
110            new_src, unit_text_map.get(new_src, "未知"),
111            new_tgt, unit_text_map.get(new_tgt, "未知"),
112            r.get("关系类型", ""), r.get("关系强度", "")
113        ])
114    # --- 4. 写入 CSV 文件的标准函数 ---
115    def save_csv(filename, header, rows):
116        output_path = os.path.join(base_dir, filename)
117        # utf-8-sig 防止 Excel 打开中文乱码
118        with open(output_path, 'w', encoding='utf-8-sig', newline='') as f:
119            writer = csv.writer(f)
120            writer.writerow(header)
121            writer.writerows(rows)
122        return output_path
123    if rows_a:
124        path_a = save_csv("Table_A_Atomic.csv", header_a, rows_a)
125        path_b = save_csv("Table_B_Lines.csv", header_b, rows_b)
126        path_c = save_csv("Table_C_Relations.csv", header_c, rows_c)
127        print("\n 数据转换与全局重排成功! 文件已生成至当前目录:")
128        print(f"  Table A (共 {len(rows_a)} 个分析单元): {path_a}")
129        print(f"  Table B (共 {len(rows_b)} 条诗行轨迹): {path_b}")
130        print(f"  Table C (共 {len(rows_c)} 对意象关系): {path_c}")
131    else:
132        print("\n 警告: 没有从 JSON 文件中提取到任何有效数据。")
133    if __name__ == "__main__":
134        aggregate_jsons_to_csv()
-- 文件结束: json_to_3csv.py --
文件: split_lkb.py (55 行, 2.3 KB)
1 import os
2 def split_poetry_lkb_extreme(input_file, max_bytes=1024):
3     if not os.path.exists(input_file):
4         print(f" 找不到文件: {input_file}")
5     return

```

```

6     with open(input_file, 'r', encoding='utf-8') as f:
7         content = f.read()
8     poems = [p.strip() for p in content.split('=====') if p.strip()]
9     output_dir = "api_chunks_1kb"
10    if not os.path.exists(output_dir):
11        os.makedirs(output_dir)
12    print(f" 共读取到 {len(poems)} 首诗歌。启动【1KB 极限防爆防截断】切分引擎...")
13    current_chunk = []
14    current_bytes = 0
15    file_index = 1
16    for poem in poems:
17        poem_text = f"{poem}\n\n=====\\n\\n"
18        # 核心：计算真实的 UTF-8 字节数 (中文字符占 3 字节)
19        poem_bytes = len(poem_text.encode('utf-8'))
20        # 如果加入这首诗会超过 1KB (1024 bytes)，且当前块里已经有数据了，立刻封包！
21        if current_bytes + poem_bytes > max_bytes and current_chunk:
22            output_path = os.path.join(output_dir, f"chunk_{file_index:03d}.txt")
23            with open(output_path, 'w', encoding='utf-8') as out_f:
24                out_f.write("".join(current_chunk))
25            print(f" 生成切片: chunk_{file_index:03d}.txt (大小: {current_bytes} 字节)")
26            # 重置状态，准备下一个文件
27            file_index += 1
28            current_chunk = []
29            current_bytes = 0
30            current_chunk.append(poem_text)
31            current_bytes += poem_bytes
32    # 把最后剩下的一点点尾料封包
33    if current_chunk:
34        output_path = os.path.join(output_dir, f"chunk_{file_index:03d}.txt")
35        with open(output_path, 'w', encoding='utf-8') as out_f:
36            out_f.write("".join(current_chunk))
37        print(f" 生成收尾切片: chunk_{file_index:03d}.txt (大小: {current_bytes} 字节)")
38    print(f" 全部切分完成！文件已安全存放在目录: {os.path.abspath(output_dir)}")
39    print(f" 警告：拿着这些 1KB 的碎片去请求 API，绝对不会再触发截断！")
40 if __name__ == "__main__":
41     # 确保你的古诗源文件名为 source.txt
42     split_poetry_1kb_extreme('source.txt', max_bytes=1024)

```

-- 文件结束: split_1kb.py --

文件: tests/test_config.py (63 行, 1.4 KB)

```

1  #-*- coding: utf-8 -*-
2  """测试配置模块"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  import config
7  def test_base_dir():
8      assert os.path.isdir(config.BASE_DIR)
9  def test_api_urls():
10     assert config.EMBED_API_URL.startswith("http")
11     assert config.CHAT_API_URL.startswith("http")
12 def test_models():
13     assert config.EMBED_MODEL
14     assert config.CHAT_MODEL
15 def test_directories():
16     assert config.POEM_JSON_DIR
17     assert config.RAG_DB_DIR
18     assert config.TEMPLATES_DIR
19 def test_category_map():
20     assert len(config.CATEGORY_NAME_MAP) == 11
21     assert config.CATEGORY_NAME_MAP["1-1"] == "自然意象-天文意象"
22     assert config.CATEGORY_NAME_MAP["3-4"] == "人文意象-文化意象"
23 def test_major_category_map():
24     assert config.CATEGORY_MAJOR_MAP["1"] == "自然意象"
25     assert config.CATEGORY_MAJOR_MAP["2"] == "社会意象"

```

```

26     assert config.CATEGORY_MAJOR_MAP["3"] == "人文意象"
27 def test_known_authors():
28     assert "杜甫" in config.KNOWN_AUTHORS
29     assert "李白" in config.KNOWN_AUTHORS
30     assert len(config.KNOWN_AUTHORS) >= 20
31 def test_flask_config():
32     assert config.FLASK_PORT == 5000
33     assert config.FLASK_HOST == "0.0.0.0"
34     assert config.SECRET_KEY
35 def test_pagination():
36     assert config.PAGE_SIZE > 0
37     assert config.RENDER_LIMIT > 0
38 def test_cache_config():
39     assert config.CACHE_DEFAULT_TTL > 0
40     assert isinstance(config.CACHE_ENABLED, bool)

```

-- 文件结束: tests/test_config.py --

文件: tests/test_validators.py (119 行, 3.4 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试校验模块"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  import pytest
7  from validators import (
8      validate_question,
9      validate_item,
10     validate_history,
11     validate_poem_id,
12     validate_filename,
13     sanitize_html,
14 )
15 from errors import ValidationError
16 class TestValidateQuestion:
17     def test_valid_question(self):
18         assert validate_question("杜甫写过哪些诗? ") == "杜甫写过哪些诗? "
19     def test_empty_question(self):
20         with pytest.raises(ValidationError):
21             validate_question("")
22         with pytest.raises(ValidationError):
23             validate_question(None)
24         with pytest.raises(ValidationError):
25             validate_question(" ")
26     def test_question_stripped(self):
27         assert validate_question(" 秋天 ") == "秋天"
28 class TestValidateItem:
29     def test_valid_item_with_text(self):
30         item = {"文本": "明月", "line": "床前明月光"}
31         assert validate_item(item) == item
32     def test_valid_item_with_txt(self):
33         item = {"txt": "青山", "line": "青山一道同云雨"}
34         assert validate_item(item) == item
35     def test_invalid_item(self):
36         with pytest.raises(ValidationError):
37             validate_item({})
38         with pytest.raises(ValidationError):
39             validate_item(None)
40         with pytest.raises(ValidationError):
41             validate_item("not a dict")
42 class TestValidateHistory:
43     def test_empty_history(self):
44         assert validate_history(None) == []
45         assert validate_history([]) == []
46     def test_valid_history(self):
47         history = [

```



```

48         {"role": "user", "content": "你好"},
49         {"role": "assistant", "content": "你好! "},
50     ]
51     result = validate_history(history)
52     assert len(result) == 2
53     def test_filters_invalid_roles(self):
54         history = [
55             {"role": "user", "content": "你好"},
56             {"role": "system", "content": "system msg"},
57             {"role": "invalid", "content": "msg"},
58         ]
59         result = validate_history(history)
60         assert len(result) == 1
61     def test_filters_empty_content(self):
62         history = [
63             {"role": "user", "content": ""},
64             {"role": "assistant", "content": None},
65         ]
66         result = validate_history(history)
67         assert len(result) == 0
68     class TestValidatePoemId:
69         def test_valid_ids(self):
70             assert validate_poem_id("P001") == "P001"
71             assert validate_poem_id("12345") == "12345"
72         def test_invalid_ids(self):
73             with pytest.raises(ValidationError):
74                 validate_poem_id("")
75             with pytest.raises(ValidationError):
76                 validate_poem_id(None)
77     class TestValidateFilename:
78         def test_default_name(self):
79             assert validate_filename(None) == "export"
80             assert validate_filename("") == "export"
81         def test_valid_name(self):
82             assert validate_filename("output.csv") == "output.csv"
83             assert validate_filename("data.json") == "data.json"
84         def test_invalid_name(self):
85             with pytest.raises(ValidationError):
86                 validate_filename("bad.txt", allowed_exts=[".csv", ".json"])
87     class TestSanitizeHtml:
88         def test_sanitize(self):
89             assert sanitize_html("<script>") == "&lt;script&gt;"
90             assert sanitize_html("a&b") == "a&amp;b"
91         def test_empty(self):
92             assert sanitize_html("") == ""
93             assert sanitize_html(None) == ""

```

-- 文件结束: tests/test_validators.py --

文件: tests/test_analytics.py (161 行, 6.2 KB)

```

1  #-*- coding: utf-8 -*-
2  """测试统计分析服务"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  from analytics import StatsService
7  SAMPLE_DATA = [
8      {"poem_id": "P001", "title": "静夜思", "author": "李白",
9       "txt": "明月", "文本": "明月", "cat": "自然意象-天文意象",
10      "genre": "五言绝句", "emo_cat": "思乡", "情感类别": "思乡",
11      "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",
12      "感知通道": "视觉"},
13      {"poem_id": "P001", "title": "静夜思", "author": "李白",
14       "txt": "月光", "文本": "月光", "cat": "自然意象-天文意象",
15      "genre": "五言绝句", "emo_cat": "思乡", "情感类别": "思乡",
16      "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",

```

```

17     "感知通道": "视觉"},
18 {"poem_id": "P002", "title": "春晓", "author": "孟浩然",
19     "txt": "春鸟", "文本": "春鸟", "cat": "自然意象-动物意象",
20     "genre": "五言绝句", "emo_cat": "闲适", "情感类别": "闲适",
21     "情感极性": "正面", "大类编码": "1", "子类编码": "1-4",
22     "感知通道": "听觉"},
23 {"poem_id": "P002", "title": "春晓", "author": "孟浩然",
24     "txt": "风雨", "文本": "风雨", "cat": "自然意象-天文意象",
25     "genre": "五言绝句", "emo_cat": "闲适", "情感类别": "闲适",
26     "情感极性": "中性", "大类编码": "1", "子类编码": "1-1",
27     "感知通道": "听觉"},
28 {"poem_id": "P003", "title": "出塞", "author": "王昌龄",
29     "txt": "明月", "文本": "明月", "cat": "自然意象-天文意象",
30     "genre": "七言绝句", "emo_cat": "悲壮", "情感类别": "悲壮",
31     "情感极性": "负面", "大类编码": "1", "子类编码": "1-1",
32     "感知通道": "视觉"},
33 ]
34 class TestStatsService:
35     def setup_method(self):
36         self.service = StatsService(SAMPLE_DATA)
37     def test_total_images(self):
38         assert self.service.total_images() == 5
39     def test_total_poems(self):
40         assert self.service.total_poems() == 3
41     def test_total_authors(self):
42         assert self.service.total_authors() == 3
43     def test_top_images(self):
44         top = self.service.top_images(5)
45         assert len(top) > 0
46         assert top[0]["text"] == "明月"
47         assert top[0]["count"] == 2
48     def test_category_distribution(self):
49         cats = self.service.category_distribution()
50         assert len(cats) == 2
51         cat_map = {c["category"]: c["count"] for c in cats}
52         assert cat_map["自然意象-天文意象"] == 4
53         assert cat_map["自然意象-动物意象"] == 1
54     def test_major_category_distribution(self):
55         majors = self.service.major_category_distribution()
56         assert len(majors) == 1
57         assert majors[0]["category"] == "自然意象"
58         assert majors[0]["count"] == 5
59     def test_emotion_distribution(self):
60         emos = self.service.emotion_distribution()
61         emo_map = {e["emotion"]: e["count"] for e in emos}
62         assert emo_map["思乡"] == 2
63         assert emo_map["闲适"] == 2
64         assert emo_map["悲壮"] == 1
65     def test_emotion_polarity_distribution(self):
66         pols = self.service.emotion_polarity_distribution()
67         pol_map = {p["polarity"]: p["count"] for p in pols}
68         assert pol_map["负面"] == 3
69         assert pol_map["正面"] == 1
70         assert pol_map["中性"] == 1
71     def test_perception_channel_distribution(self):
72         chs = self.service.perception_channel_distribution()
73         ch_map = {c["channel"]: c["count"] for c in chs}
74         assert ch_map["视觉"] == 3
75         assert ch_map["听觉"] == 2
76     def test_summary_report(self):
77         report = self.service.summary_report()
78         assert report["total_images"] == 5
79         assert report["total_poems"] == 3
80         assert report["total_authors"] == 3

```

```

81     assert len(report["top_images"]) > 0
82     assert len(report["category_stats"]) > 0
83     def test_author_statistics(self):
84         authors = self.service.author_statistics()
85         author_map = {a["author"]: a for a in authors}
86         assert "李白" in author_map
87         assert author_map["李白"]["image_count"] == 2
88     def test_cross_analysis(self):
89         cross = self.service.cross_analysis()
90         assert "自然意象-天文意象" in cross
91         assert len(cross["自然意象-天文意象"]) > 0
92     def test_empty_data(self):
93         empty_service = StatsService([])
94         assert empty_service.total_images() == 0
95         assert empty_service.total_poems() == 0
96         assert empty_service.top_images(10) == []
97         assert empty_service.category_distribution() == []
98         assert empty_service.emotion_distribution() == []
99     def test_set_data(self):
100         new_service = StatsService()
101         assert new_service.total_images() == 0
102         new_service.set_data(SAMPLE_DATA[:1])
103         assert new_service.total_images() == 1
104     def test_genre_distribution(self):
105         genres = self.service.genre_distribution()
106         genre_map = {g["genre"]: g["count"] for g in genres}
107         assert genre_map["五言绝句"] == 4
108         assert genre_map["七言绝句"] == 1
109     def test_internal_structure_distribution(self):
110         stats = self.service.internal_structure_distribution()
111         assert len(stats) > 0
112         all_structures = [s["structure"] for s in stats]
113         assert "未标注" in all_structures
114     def test_function_distribution(self):
115         stats = self.service.function_distribution()
116         assert len(stats) > 0
117     def test_core_image_distribution(self):
118         stats = self.service.core_image_distribution()
119         assert len(stats) > 0
120         assert stats[0]["core_image"] == "未标注"
121     def test_reference_source_distribution(self):
122         stats = self.service.reference_source_distribution()
123         assert len(stats) > 0
124     def test_cognitive_intensity_distribution(self):
125         stats = self.service.cognitive_intensity_distribution()
126         assert len(stats) > 0
127     def test_material_type_distribution(self):
128         stats = self.service.material_type_distribution()
129         assert len(stats) > 0
130     def test_cross_cultural_distribution(self):
131         stats = self.service.cross_cultural_distribution()
132         assert len(stats) > 0

```

-- 文件结束: tests/test_analytics.py --

文件: tests/test_cache.py (115 行, 3.2 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试缓存模块"""
3  import os
4  import sys
5  import time
6  import tempfile
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  import pytest
9  from cache import MemoryCache, FileCache
10 class TestMemoryCache:

```

```

11     def setup_method(self):
12         self.cache = MemoryCache(default_ttl=60)
13     def test_set_and_get(self):
14         self.cache.set("key1", "value1")
15         assert self.cache.get("key1") == "value1"
16     def test_get_nonexistent(self):
17         assert self.cache.get("nonexistent") is None
18     def test_delete(self):
19         self.cache.set("key_del", "to_delete")
20         self.cache.delete("key_del")
21         assert self.cache.get("key_del") is None
22     def test_clear(self):
23         self.cache.set("a", 1)
24         self.cache.set("b", 2)
25         self.cache.clear()
26         assert self.cache.size() == 0
27     def test_ttl_expiry(self):
28         cache = MemoryCache(default_ttl=1)
29         cache.set("expire_key", "will_expire")
30         assert cache.get("expire_key") == "will_expire"
31         time.sleep(1.5)
32         assert cache.get("expire_key") is None
33     def test_custom_ttl(self):
34         self.cache.set("short", "quick", ttl=1)
35         assert self.cache.get("short") == "quick"
36         time.sleep(1.5)
37         assert self.cache.get("short") is None
38     def test_no_ttl(self):
39         cache = MemoryCache(default_ttl=0)
40         cache.set("no_ttl", "forever")
41         assert cache.get("no_ttl") == "forever"
42     def test_size(self):
43         self.cache.set("s1", 1)
44         self.cache.set("s2", 2)
45         self.cache.set("s3", 3)
46         assert self.cache.size() == 3
47     def test_keys(self):
48         self.cache.set("k1", 1)
49         self.cache.set("k2", 2)
50         keys = self.cache.keys()
51         assert "k1" in keys
52         assert "k2" in keys
53     def test_complex_objects(self):
54         obj = {"a": [1, 2, 3], "b": {"nested": "value"}}
55         self.cache.set("complex", obj)
56         assert self.cache.get("complex") == obj
57     class TestFileCache:
58     def setup_method(self):
59         self.tmpdir = tempfile.mkdtemp()
60         self.cache = FileCache(cache_dir=self.tmpdir, default_ttl=60)
61     def teardown_method(self):
62         import shutil
63         shutil.rmtree(self.tmpdir, ignore_errors=True)
64     def test_set_and_get(self):
65         self.cache.set("fkey", "fvalue")
66         assert self.cache.get("fkey") == "fvalue"
67     def test_get_nonexistent(self):
68         assert self.cache.get("no_such_key") is None
69     def test_delete(self):
70         self.cache.set("del_key", "delete_me")
71         self.cache.delete("del_key")
72         assert self.cache.get("del_key") is None
73     def test_clear(self):
74         self.cache.set("a", 1)

```

```

75         self.cache.set("b", 2)
76         self.cache.clear()
77         assert self.cache.size() == 0
78     def test_special_chars_in_key(self):
79         self.cache.set("path/to/file", "content")
80         assert self.cache.get("path/to/file") == "content"
81     def test_dict_value(self):
82         data = {"name": "test", "values": [1, 2, 3]}
83         self.cache.set("dict_key", data)
84         assert self.cache.get("dict_key") == data
85     def test_ttl_expiry(self):
86         cache = FileCache(cache_dir=self.tmpdir, default_ttl=1)
87         cache.set("expire", "gone")
88         time.sleep(1.5)
89         assert cache.get("expire") is None

```

-- 文件结束: tests/test_cache.py --

文件: tests/test_export.py (89 行, 3.0 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试导出服务"""
3  import os
4  import sys
5  import tempfile
6  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
7  import pytest
8  # 重定向 EXPORT_DIR 到临时目录
9  import export_service
10 import config
11 from export_service import ExportService
12 SAMPLE_DATA = [
13     {"poem_id": "P001", "title": "静夜思", "txt": "明月",
14      "cat": "自然意象-天文意象", "genre": "五言绝句",
15      "emotion": "思乡(负面)", "line": "床前明月光"},
16     {"poem_id": "P002", "title": "春晓", "txt": "春鸟",
17      "cat": "自然意象-动物意象", "genre": "五言绝句",
18      "emotion": "闲适(正面)", "line": "处处闻啼鸟"},
19 ]
20 class TestExportService:
21     def setup_method(self):
22         self.tmpdir = tempfile.mkdtemp()
23         self.original_export_dir = config.EXPORT_DIR
24         config.EXPORT_DIR = self.tmpdir
25         # 重新加载模块级配置
26         import importlib
27         importlib.reload(export_service)
28         # 重新导入以使用新配置
29         from export_service import ExportService
30         self.service = ExportService
31     def teardown_method(self):
32         import shutil
33         shutil.rmtree(self.tmpdir, ignore_errors=True)
34         config.EXPORT_DIR = self.original_export_dir
35     def test_export_csv(self):
36         path = self.service.export_traceback_to_csv(SAMPLE_DATA, "test.csv")
37         assert os.path.exists(path)
38         with open(path, "r", encoding="utf-8-sig") as f:
39             content = f.read()
40             assert "poem_id" in content
41             assert "P001" in content
42             assert "静夜思" in content
43     def test_export_json(self):
44         path = self.service.export_to_json(SAMPLE_DATA, "test.json")
45         assert os.path.exists(path)
46         with open(path, "r", encoding="utf-8") as f:
47             data = f.read()

```

```

48     assert "P001" in data
49     assert "静夜思" in data
50     def test_export_summary_report(self):
51         report = {"total": 100, "items": ["a", "b"]}
52         path = self.service.export_summary_report(report, "report.json")
53         assert os.path.exists(path)
54     def test_export_filtered_slice(self):
55         path = self.service.export_filtered_slice(SAMPLE_DATA, 0, 1, "slice.json")
56         assert os.path.exists(path)
57         with open(path, "r", encoding="utf-8") as f:
58             import json
59             data = json.load(f)
60             assert data["count"] == 1
61             assert data["total"] == 2
62     def test_list_exports(self):
63         self.service.export_traceback_to_csv(SAMPLE_DATA, "list_test.csv")
64         files = self.service.list_exports()
65         assert len(files) >= 1
66         assert any(f["name"] == "list_test.csv" for f in files)
67     def test_clear_exports(self):
68         self.service.export_traceback_to_csv(SAMPLE_DATA, "clear_me.csv")
69         count = self.service.clear_exports()
70         assert count >= 1
71         assert len(self.service.list_exports()) == 0
72     def test_export_empty_data_raises(self):
73         with pytest.raises(Exception):
74             self.service.export_traceback_to_csv([], "empty.csv")

```

-- 文件结束: tests/test_export.py --

文件: tests/test_models.py (109 行, 3.5 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试数据模型"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  from models import Poem, PoemLine, AnalysisUnit, EmotionTrajectory, ImageRelation
7  class TestPoemLine:
8      def test_defaults(self):
9          line = PoemLine()
10         assert line.诗行编号 == ""
11         assert line.原文 == ""
12     def test_to_dict(self):
13         line = PoemLine(诗行编号="L01", 原文="床前明月光")
14         d = line.to_dict()
15         assert d["诗行编号"] == "L01"
16         assert d["原文"] == "床前明月光"
17     class TestAnalysisUnit:
18     def test_is_image_true(self):
19         unit = AnalysisUnit(是否意象="1")
20         assert unit.is_image is True
21     def test_is_image_false(self):
22         unit = AnalysisUnit(是否意象="0")
23         assert unit.is_image is False
24     def test_to_dict(self):
25         unit = AnalysisUnit(文本="明月", 词性="名词")
26         d = unit.to_dict()
27         assert d["文本"] == "明月"
28         assert d["词性"] == "名词"
29     class TestPoem:
30     def test_empty_poem(self):
31         poem = Poem()
32         assert poem.诗歌编号 == ""
33         assert poem.image_count == 0
34     def test_image_units_filter(self):
35         poem = Poem(

```

```

36         诗歌编号="P001",
37         标题="静夜思",
38         作者="李白",
39         分析单元=[
40             AnalysisUnit(文本="明月", 是否意象="1"),
41             AnalysisUnit(文本="床", 是否意象="0"),
42             AnalysisUnit(文本="霜", 是否意象="1"),
43         ],
44     )
45     assert poem.image_count == 2
46     assert len(poem.image_units) == 2
47     assert poem.image_units[0].文本 == "明月"
48     def test_summary(self):
49         poem = Poem(标题="静夜思", 作者="李白", 诗歌编号="P001",
50                     分析单元=[AnalysisUnit(是否意象="1")])
51         assert "静夜思" in poem.summary
52         assert "李白" in poem.summary
53         assert "1个意象" in poem.summary
54     def test_from_dict_full(self):
55         data = {
56             "诗歌编号": "P001",
57             "标题": "静夜思",
58             "作者": "李白",
59             "分类标签": "五言绝句",
60             "原文": "床前明月光",
61             "诗行": [{"诗行编号": "L01", "原文": "床前明月光"}],
62             "分析单元": [{"单元编号": "U001", "文本": "明月", "是否意象": "1"}],
63             "情感轨迹": [{"诗行编号": "L01", "主导情感": "思乡"}],
64             "意象关系": [{"关系类型": "并列", "来源文本": "明月", "目标文本": "床"}],
65         }
66         poem = Poem.from_dict(data)
67         assert poem.诗歌编号 == "P001"
68         assert poem.标题 == "静夜思"
69         assert len(poem.诗行) == 1
70         assert len(poem.分析单元) == 1
71         assert len(poem.情感轨迹) == 1
72         assert len(poem.意象关系) == 1
73     def test_to_dict(self):
74         poem = Poem(诗歌编号="P001", 标题="静夜思")
75         d = poem.to_dict()
76         assert d["诗歌编号"] == "P001"
77         assert d["标题"] == "静夜思"
78     class TestEmotionTrajectory:
79         def test_to_dict(self):
80             t = EmotionTrajectory(诗行编号="L01", 主导情感="思乡")
81             d = t.to_dict()
82             assert d["诗行编号"] == "L01"
83             assert d["主导情感"] == "思乡"
84     class TestImageRelation:
85         def test_to_dict(self):
86             r = ImageRelation(关系类型="并列", 关系强度="强")
87             d = r.to_dict()
88             assert d["关系类型"] == "并列"
89             assert d["关系强度"] == "强"

```

-- 文件结束: tests/test_models.py --

文件: tests/test_preprocessor.py (124 行, 4.1 KB)

```

1  #-*- coding: utf-8 -*-
2  """测试数据预处理模块"""
3  import os
4  import sys
5  import json
6  import tempfile
7  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
8  from preprocessor import (

```

```

9     clean_json_content,
10    validate_poem_json,
11    parse_json_file,
12    extract_image_statistics,
13 )
14 from models import Poem, AnalysisUnit
15 class TestCleanJsonContent:
16     def test_remove_code_block(self):
17         content = "`json\n{ \"key\": \"value\" }\n`"
18         result = clean_json_content(content)
19         assert result == '{"key": "value"}'
20     def test_no_block(self):
21         content = '{"key": "value"}'
22         result = clean_json_content(content)
23         assert result == content
24     def test_trailing_commas(self):
25         content = '{"key": "value",}'
26         result = clean_json_content(content)
27         assert "value" in result
28         assert result.endswith("}")
29     def test_strip(self):
30         content = " \n {'key': 'val'} \n "
31         result = clean_json_content(content)
32         assert result == '{"key": "val"}'
33 class TestValidatePoemJson:
34     def test_valid_basic(self):
35         data = {"诗歌集": [{"标题": "静夜思", "诗歌编号": "001", "原文": "床前明月光"}]}
36         errors = validate_poem_json(data)
37         assert len(errors) == 0
38     def test_missing_title(self):
39         data = {"诗歌集": [{"诗歌编号": "001"}]}
40         errors = validate_poem_json(data)
41         assert any("标题" in e for e in errors)
42     def test_missing_id(self):
43         data = {"诗歌集": [{"标题": "静夜思"}]}
44         errors = validate_poem_json(data)
45         assert any("编号" in e for e in errors)
46     def test_analysis_unit_validation(self):
47         data = {
48             "诗歌集": [{
49                 "标题": "静夜思",
50                 "诗歌编号": "001",
51                 "原文": "床前明月光",
52                 "诗行": [{"诗行编号": "1", "原文": "床前明月光"}],
53                 "分析单元": [
54                     {"文本": "明月", "诗行编号": "1"},
55                     {"诗行编号": "2"}, # missing text
56                 ],
57             }]
58         }
59         errors = validate_poem_json(data)
60         assert len(errors) == 1
61         assert "分析单元" in errors[0]
62         assert "文本" in errors[0]
63 class TestParseJsonFile:
64     def test_valid_file(self):
65         with tempfile.NamedTemporaryFile(mode="w", encoding="utf-8",
66                                         suffix=".json", delete=False) as f:
67             json.dump({"诗歌集": [{"标题": "静夜思", "诗歌编号": "001",
68                                   "原文": "床前明月光"}]}, f, ensure_ascii=False)
69         fpath = f.name
70         data, errors = parse_json_file(fpath)
71         assert data is not None
72         assert len(errors) == 0

```



```

73         os.unlink(fpath)
74     def test_invalid_json(self):
75         with tempfile.NamedTemporaryFile(mode="w", encoding="utf-8",
76                                           suffix=".json", delete=False) as f:
77             f.write("{bad json}")
78             fpath = f.name
79             data, errors = parse_json_file(fpath)
80             assert data is None
81             assert len(errors) > 0
82             os.unlink(fpath)
83     def test_file_not_found(self):
84         data, errors = parse_json_file("/nonexistent/file.json")
85         assert data is None
86         assert any("读取" in e for e in errors)
87 class TestExtractImageStatistics:
88     def test_basic_stats(self):
89         poems = [
90             Poem(标题="A", 作者="李白",
91                  分析单元=[AnalysisUnit(文本="明月", 是否意象="1"),
92                                AnalysisUnit(文本="床", 是否意象="0")]),
93             Poem(标题="B", 作者="杜甫",
94                  分析单元=[AnalysisUnit(文本="落日", 是否意象="1")]),
95         ]
96         stats = extract_image_statistics(poems)
97         assert stats["total_poems"] == 2
98         assert stats["total_images"] == 2
99         assert stats["unique_images"] == 2
100        assert stats["authors"] == 2
101    def test_empty_poems(self):
102        stats = extract_image_statistics([])
103        assert stats["total_poems"] == 0
104        assert stats["total_images"] == 0

```

-- 文件结束: tests/test_preprocessor.py --

文件: tests/test_middleware.py (91 行, 2.5 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试中间件模块"""
3  import os
4  import sys
5  import time
6  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
7  from middleware import (
8      RequestContext,
9      PerformanceTracker,
10     timer,
11 )
12 class TestRequestContext:
13     def test_defaults(self):
14         ctx = RequestContext()
15         assert ctx.request_id is None
16         assert ctx.start_time is None
17     def test_elapsed(self):
18         ctx = RequestContext()
19         ctx.start_time = time.time() - 1.0
20         elapsed = ctx.elapsed()
21         assert elapsed >= 900
22     def test_to_dict(self):
23         ctx = RequestContext()
24         ctx.request_id = "abc123"
25         ctx.client_ip = "127.0.0.1"
26         ctx.endpoint = "test"
27         ctx.method = "GET"
28         ctx.start_time = time.time()
29         d = ctx.to_dict()
30         assert d["request_id"] == "abc123"

```

```

31         assert d["method"] == "GET"
32     class TestPerformanceTracker:
33     def setup_method(self):
34         self.tracker = PerformanceTracker()
35     def test_record(self):
36         self.tracker.record("/api/test", 100.0, 200)
37         summary = self.tracker.summary()
38         assert "/api/test" in summary
39         assert summary["/api/test"]["calls"] == 1
40         assert summary["/api/test"]["avg_ms"] == 100.0
41     def test_multiple_records(self):
42         for i in range(5):
43             self.tracker.record("/api/test", float(i * 10), 200)
44         summary = self.tracker.summary()
45         assert summary["/api/test"]["calls"] == 5
46         assert summary["/api/test"]["max_ms"] == 40.0
47         assert summary["/api/test"]["min_ms"] == 0.0
48     def test_error_counting(self):
49         self.tracker.record("/api/test", 50.0, 200)
50         self.tracker.record("/api/test", 50.0, 500)
51         self.tracker.record("/api/test", 50.0, 400)
52         summary = self.tracker.summary()
53         assert summary["/api/test"]["errors"] == 2
54         assert summary["/api/test"]["error_rate_pct"] == 66.7
55     def test_reset(self):
56         self.tracker.record("/api/test", 100.0, 200)
57         self.tracker.reset()
58         assert self.tracker.summary() == {}
59     def test_multiple_endpoints(self):
60         self.tracker.record("/api/a", 100.0, 200)
61         self.tracker.record("/api/b", 200.0, 200)
62         summary = self.tracker.summary()
63         assert len(summary) == 2
64     class TestTimerDecorator:
65     def test_timer_basic(self):
66         @timer
67         def my_func():
68             return 42
69         assert my_func() == 42
70     def test_timer_with_args(self):
71         @timer
72         def add(a, b):
73             return a + b
74         assert add(3, 4) == 7

```

-- 文件结束: tests/test_middleware.py --

文件: tests/test_cli.py (32 行, 0.7 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试 CLI 模块"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  from cli import cmd_status, cmd_scan, cmd_clear_cache, cmd_list_exports
7  class MockArgs:
8      def __init__(self, **kwargs):
9          for k, v in kwargs.items():
10              setattr(self, k, v)
11  class TestCliCommands:
12      def test_status_runs(self):
13          """至少不抛出异常"""
14          args = MockArgs()
15          try:
16              cmd_status(args)
17          except SystemExit:
18              pass

```

```

19     def test_clear_cache_runs(self):
20         args = MockArgs()
21         try:
22             cmd_clear_cache(args)
23         except Exception:
24             pass

```

-- 文件结束: tests/test_cli.py --

文件: tests/test_utils.py (173 行, 4.1 KB)

```

1  # -*- coding: utf-8 -*-
2  """测试通用工具函数"""
3  import os
4  import sys
5  sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
6  from utils import (
7      truncate,
8      is_chinese_char,
9      chinese_ratio,
10     split_sentences,
11     format_file_size,
12     safe_get,
13     dict_pick,
14     dict_omit,
15     md5_hash,
16     safe_json_loads,
17     list_chunk,
18     deduplicate_by_key,
19     normalize_whitespace,
20     extract_numbers,
21     merge_dicts,
22 )
23 class TestTruncate:
24     def test_short_string(self):
25         assert truncate("hello") == "hello"
26     def test_long_string(self):
27         result = truncate("hello world", max_len=5)
28         assert result == "hello..."
29     def test_empty(self):
30         assert truncate("") == ""
31         assert truncate(None) == ""
32 class TestIsChineseChar:
33     def test_chinese(self):
34         assert is_chinese_char("中") is True
35         assert is_chinese_char("国") is True
36     def test_not_chinese(self):
37         assert is_chinese_char("a") is False
38         assert is_chinese_char("1") is False
39         assert is_chinese_char("") is False
40 class TestChineseRatio:
41     def test_all_chinese(self):
42         assert chinese_ratio("中文") == 1.0
43     def test_mixed(self):
44         ratio = chinese_ratio("hello中文")
45         assert abs(ratio - 0.2857) < 0.001
46     def test_empty(self):
47         assert chinese_ratio("") == 0
48 class TestSplitSentences:
49     def test_basic(self):
50         result = split_sentences("句一。句二! 句三? ")
51         assert len(result) == 3
52     def test_no_punctuation(self):
53         assert split_sentences("一句话") == ["一句话"]
54     def test_empty(self):
55         assert split_sentences("") == []
56 class TestFormatFileSize:

```

```

57     def test_bytes(self):
58         assert "B" in format_file_size(500)
59     def test_kb(self):
60         assert "KB" in format_file_size(2048)
61     def test_mb(self):
62         assert "MB" in format_file_size(1048576)
63     def test_zero(self):
64         assert format_file_size(0) == "0 B"
65 class TestSafeGet:
66     def test_existing_key(self):
67         assert safe_get({"a": 1}, "a") == 1
68     def test_missing_key(self):
69         assert safe_get({"a": 1}, "b") == ""
70     def test_custom_default(self):
71         assert safe_get({"a": 1}, "b", default=None) is None
72     def test_none_value(self):
73         assert safe_get({"a": None}, "a") == ""
74 class TestDictPick:
75     def test_basic(self):
76         result = dict_pick({"a": 1, "b": 2, "c": 3}, ["a", "c"])
77         assert result == {"a": 1, "c": 3}
78 class TestDictOmit:
79     def test_basic(self):
80         result = dict_omit({"a": 1, "b": 2, "c": 3}, ["b"])
81         assert result == {"a": 1, "c": 3}
82 class TestMd5Hash:
83     def test_basic(self):
84         h = md5_hash("hello")
85         assert len(h) == 32
86         assert h == md5_hash("hello")
87     def test_different(self):
88         assert md5_hash("a") != md5_hash("b")
89 class TestSafeJsonLoads:
90     def test_valid(self):
91         result = safe_json_loads('{"a": 1}')
92         assert result == {"a": 1}
93     def test_invalid(self):
94         result = safe_json_loads("{bad}", default={})
95         assert result == {}
96 class TestListChunk:
97     def test_basic(self):
98         result = list_chunk([1, 2, 3, 4, 5], 2)
99         assert result == [[1, 2], [3, 4], [5]]
100     def test_empty(self):
101         assert list_chunk([], 2) == []
102 class TestDeduplicateByKey:
103     def test_basic(self):
104         data = [{"id": 1, "v": "a"}, {"id": 2, "v": "b"}, {"id": 1, "v": "c"}]
105         result = deduplicate_by_key(data, "id")
106         assert len(result) == 2
107         assert result[0]["v"] == "a"
108 class TestNormalizeWhitespace:
109     def test_basic(self):
110         assert normalize_whitespace(" a b c ") == "a b c"
111     def test_empty(self):
112         assert normalize_whitespace("") == ""
113         assert normalize_whitespace(None) == ""
114 class TestExtractNumbers:
115     def test_basic(self):
116         assert extract_numbers("abc123def456") == [123, 456]
117     def test_no_numbers(self):
118         assert extract_numbers("abc") == []
119 class TestMergeDicts:
120     def test_basic(self):

```

```
121         result = merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4})
122         assert result == {"a": 1, "b": 3, "c": 4}
-- 文件结束: tests/test_utils.py --
```